01

NOTEBOOK

Elastic Net regression

01_elastic_net.pdf

Elastic Net

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)


from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')         # adjust if need
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v2'
RUN_DIR.mkdir(parents=True, exist_ok=True)

for f in ['movie_features_v6.csv',
          'movie_features_v6_synthetic.csv',
          'scene_movie_metadata_v6.csv',
          'scene_movie_metadata_v6_synthetic.csv']:
    assert (DATA_DIR / f).exists(), f'Missing: {f}'

print('Data dir:', DATA_DIR)
print('Run dir: ', RUN_DIR)
```

```
Mounted at /content/drive
Data dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6
Run dir:  /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6/colab_runs_v2
```

In [2]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, StratifiedKFold, GridSearchCV, cross_val_predict,
)
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
)
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
pd.set_option('display.width', 180)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

2 · Data loading

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')

# Drop target / leakage columns
LEAKAGE_AND_OLD_TARGETS = [
    'budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_mov = real_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')

# Columns from metadata to use as features (excluding IDs, text, leakage)
META_KEEP = [
    'movie_id',
    # Scene properties (will be encoded below)
    'targeted_emotion', 'clip_duration_s',
    'cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
    'silence_ratio', 'music_presence', 'dialogue_density',
    'face_screen_time_ratio', 'lead_screen_time_ratio',
    # Movie metadata
    'release_year', 'genre_primary', 'genre_secondary',
    'country_of_origin', 'budget_categorical',
    # Targets (kept to extract y; not used as features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]

real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left')
real_mov['is_synthetic'] = 0
syn_mov = syn_mov.merge(syn_meta_sub, on='movie_id', how='left')
syn_mov['is_synthetic'] = 1

df_all = pd.concat([real_mov, syn_mov], ignore_index=True)
print(f'Real:      {len(real_mov)} movies')
print(f'Synthetic: {len(syn_mov)} movies')
print(f'Combined:  {len(df_all)} movies × {len(df_all.columns)} columns (pre
print()
print('Target distributions (pre-encoding):')
display(df_all[['is_synthetic', 'imdb_rating', 'wom_multiplier_log']]
        .groupby('is_synthetic').describe().round(2))
```

```
Real:      10 movies
Synthetic: 40 movies
Combined:  50 movies × 343 columns (pre-encoding)

Target distributions (pre-encoding):
```

	imdb_rating											
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min
is_synthetic												
0	10.0	7.41	0.73	6.5	6.8	7.45	8.02	8.5	10.0	1.77	0.64	0.75
1	40.0	7.48	0.55	6.5	7.0	7.50	7.82	8.4	40.0	1.70	0.54	0.34

In [ ]:

```
# Encode metadata features and build the final X / y
DROP = {
    'movie_id', 'condition', 'n_participants',
    # Targets (cannot be features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
    # Control flag (not a feature)
    'is_synthetic',
}

df_feat = df_all.copy()

#Ordinal encoding: low / moderate / high → 1 / 2 / 3
ORD_MAP = {'low': 1, 'moderate': 2, 'high': 3}
ORD_COLS = ['cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
            'silence_ratio', 'music_presence', 'dialogue_density',
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'budget_categorical']
for c in ORD_COLS:
    if c in df_feat.columns:
        df_feat[c] = df_feat[c].map(ORD_MAP)

#One-hot encode multi-category text columns
OH_COLS = ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country_
OH_COLS = [c for c in OH_COLS if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH_COLS, prefix_sep='_',
                         dummy_na=False, dtype=int)

#Build feature matrix
feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
groups_all = df_feat['movie_id'].values if 'movie_id' in df_feat.columns els
synth_mask_all = df_feat['is_synthetic'].values.astype(bool)

#Drop all-NaN columns
all_nan_cols = X_all.columns[X_all.isna().all()].tolist()
if all_nan_cols:
    print(f'Dropping {len(all_nan_cols)} all-NaN columns')
    X_all = X_all.drop(columns=all_nan_cols)

#Drop zero-variance columns
zero_var_cols = X_all.columns[X_all.std() == 0].tolist()
if zero_var_cols:
    print(f'Dropping {len(zero_var_cols)} zero-variance columns')
```

```
    X_all = X_all.drop(columns=zero_var_cols)

feature_cols = list(X_all.columns)

#Categorise final features for transparency
phys_cols = [c for c in feature_cols if c.endswith('__mean') or c.endswith('
ord_in_X  = [c for c in feature_cols if c in ORD_COLS]
oh_in_X   = [c for c in feature_cols
             if any(c.startswith(p + '_') for p in
                    ['targeted_emotion', 'genre_primary', 'genre_secondary',
                     'country_of_origin'])]
other_cols = [c for c in feature_cols
              if c not in phys_cols + ord_in_X + oh_in_X]

print(f'\nFinal feature matrix: {X_all.shape}')
print(f'  Physiological (mean/std):  {len(phys_cols):>4d}')
print(f'  Scene/movie ordinal:       {len(ord_in_X):>4d}  ({ord_in_X})')
print(f'  Scene/movie one-hot:       {len(oh_in_X):>4d}')
print(f'  Other numeric (year, dur): {len(other_cols):>4d}  ({other_cols})')
print(f'  TOTAL features:            {len(feature_cols):>4d}')
```

```
Final feature matrix: (50, 360)
  Physiological (mean/std):   320
  Scene/movie ordinal:         10  (['cut_count', 'brightness', 'motion_inte
nsity', 'audio_loudness', 'silence_ratio', 'music_presence', 'dialogue_densi
ty', 'face_screen_time_ratio', 'lead_screen_time_ratio', 'budget_categorica
l'])
  Scene/movie one-hot:         28
  Other numeric (year, dur):    2  (['clip_duration_s', 'release_year'])
  TOTAL features:             360
```

In [ ]:

```
#Evaluation helpers

def regression_metrics(y_true, y_pred):
    """Compute R², MAE, RMSE, Spearman, Pearson."""
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    if len(yt) < 3:
        return {'n': len(yt), 'r2': np.nan, 'mae': np.nan, 'rmse': np.nan,
                'spearman': np.nan, 'pearson': np.nan}
    return {
        'n': len(yt),
        'r2': r2_score(yt, yp),
        'mae': mean_absolute_error(yt, yp),
        'rmse': np.sqrt(mean_squared_error(yt, yp)),
        'spearman': spearmanr(yt, yp).correlation,
        'pearson': pearsonr(yt, yp)[0],
    }


def loo_predict(estimator, X, y):
    """Leave-one-out cross-validated predictions."""
    loo = LeaveOneOut()
    return cross_val_predict(estimator, X, y, cv=loo, n_jobs=-1)
```

```
def kfold_predict(estimator, X, y, n_splits=5, random_state=42):
    """K-fold CV predictions (k=5 default)."""
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    return cross_val_predict(estimator, X, y, cv=kf, n_jobs=-1)


def report(metrics, title=''):
    print(f'\n——— {title} ———')
    for k, v in metrics.items():
        if k == 'n':
            print(f'  {k:>10s}: {v}')
        else:
            print(f'  {k:>10s}: {v:.3f}' if not np.isnan(v) else f'  {k:>10s
```

```
def plot_pred_vs_actual(y_true, y_pred, title, ax=None):
    if ax is None:
        fig, ax = plt.subplots(figsize=(5, 5))
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    ax.scatter(yt, yp, alpha=0.6, s=40, edgecolor='k', linewidth=0.5)
    lo, hi = min(yt.min(), yp.min()), max(yt.max(), yp.max())
    ax.plot([lo, hi], [lo, hi], 'r--', alpha=0.5, label='y = x')
    r2 = r2_score(yt, yp)
    rho = spearmanr(yt, yp).correlation
    ax.set_xlabel('Actual')
    ax.set_ylabel('Predicted')
    ax.set_title(f'{title}\nR² = {r2:.2f}, Spearman ρ = {rho:.2f}')
    ax.legend()
    ax.grid(alpha=0.3)
    return ax
```

Real-only (Leave-One-Out CV, n = 10)

In [6]:

```
from sklearn.linear_model import ElasticNet

# Real-only subsets
mask_real = ~synth_mask_all
X_real = X_all[mask_real].reset_index(drop=True)
y_imdb_real = y_imdb_all[mask_real].reset_index(drop=True)
y_wom_real = y_wom_all[mask_real].reset_index(drop=True)

print(f'Real-only data: {X_real.shape}')

# Pipeline: impute → scale → ElasticNet
def make_elastic_pipeline(alpha=1.0, l1_ratio=0.5):
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('model', ElasticNet(alpha=alpha, l1_ratio=l1_ratio, max_iter=10000,
    ])
```

```
Real-only data: (10, 360)
```

Baseline (default hyperparameters)

In [7]:
```
# IMDb rating
pipe = make_elastic_pipeline(alpha=1.0, l1_ratio=0.5)
y_pred_real_imdb = loo_predict(pipe, X_real, y_imdb_real)
metrics_real_imdb_baseline = regression_metrics(y_imdb_real, y_pred_real_imc
report(metrics_real_imdb_baseline, 'Real-only / IMDb / baseline')

# WOM multiplier
pipe = make_elastic_pipeline(alpha=1.0, l1_ratio=0.5)
y_pred_real_wom = loo_predict(pipe, X_real, y_wom_real)
metrics_real_wom_baseline = regression_metrics(y_wom_real, y_pred_real_wom)
report(metrics_real_wom_baseline, 'Real-only / WOM-log / baseline')
```

```
——— Real-only / IMDb / baseline ———
          n: 10
         r2: -0.271
        mae: 0.701
       rmse: 0.785
   spearman: -0.750
    pearson: -0.779

——— Real-only / WOM-log / baseline ———
          n: 10
         r2: -0.248
        mae: 0.578
       rmse: 0.675
   spearman: -0.964
    pearson: -0.981
```

Hyperparameter tuning (LOO grid search)

In [ ]:
```
# Wider alpha grid (log-space) and denser l1_ratio
param_grid = {
    'model__alpha':    np.logspace(-4, 2, 25).tolist(),
    'model__l1_ratio': [0.05, 0.1, 0.2, 0.3, 0.5, 0.7, 0.85, 0.95, 0.99],
}

# IMDb
search = GridSearchCV(make_elastic_pipeline(), param_grid,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
                      n_jobs=-1, refit=True)
search.fit(X_real, y_imdb_real)
best_params_imdb_real = search.best_params_
y_pred_real_imdb_tuned = loo_predict(search.best_estimator_, X_real, y_imdb_
metrics_real_imdb_tuned = regression_metrics(y_imdb_real, y_pred_real_imdb_t
print(f'\nBest IMDb params (real): {best_params_imdb_real}')
report(metrics_real_imdb_tuned, 'Real-only / IMDb / tuned')

# WOM
search = GridSearchCV(make_elastic_pipeline(), param_grid,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
```

```
                      n_jobs=-1, refit=True)
search.fit(X_real, y_wom_real)
best_params_wom_real = search.best_params_
y_pred_real_wom_tuned = loo_predict(search.best_estimator_, X_real, y_wom_re
metrics_real_wom_tuned = regression_metrics(y_wom_real, y_pred_real_wom_tune
print(f'\nBest WOM params (real): {best_params_wom_real}')
report(metrics_real_wom_tuned, 'Real-only / WOM-log / tuned')
```

```
Best IMDb params (real): {'model__alpha': 0.0001, 'model__l1_ratio': 0.05}

──── Real-only / IMDb / tuned ────
           n: 10
          r2: 0.115
         mae: 0.533
        rmse: 0.655
    spearman: 0.463
     pearson: 0.471

Best WOM params (real): {'model__alpha': 0.1, 'model__l1_ratio': 0.99}

──── Real-only / WOM-log / tuned ────
           n: 10
          r2: 0.431
         mae: 0.355
        rmse: 0.456
    spearman: 0.685
     pearson: 0.743
```

Augmented (5-fold CV, n = 50)

In [9]:

```
# Use full augmented dataset (n=50)
print(f'Augmented data: {X_all.shape}')

# Baseline
pipe = make_elastic_pipeline(alpha=1.0, l1_ratio=0.5)
y_pred_aug_imdb = kfold_predict(pipe, X_all, y_imdb_all, n_splits=5)
metrics_aug_imdb_baseline = regression_metrics(y_imdb_all, y_pred_aug_imdb)
report(metrics_aug_imdb_baseline, 'Augmented / IMDb / baseline')

pipe = make_elastic_pipeline(alpha=1.0, l1_ratio=0.5)
y_pred_aug_wom = kfold_predict(pipe, X_all, y_wom_all, n_splits=5)
metrics_aug_wom_baseline = regression_metrics(y_wom_all, y_pred_aug_wom)
report(metrics_aug_wom_baseline, 'Augmented / WOM-log / baseline')
```

```
Augmented data: (50, 360)

──── Augmented / IMDb / baseline ────
           n: 50
          r2: -0.013
         mae: 0.500
        rmse: 0.579
    spearman: -0.171
     pearson: -0.155

──── Augmented / WOM-log / baseline ────
           n: 50
          r2: -0.028
         mae: 0.460
        rmse: 0.556
    spearman: -0.219
     pearson: -0.224
```

In [10]:

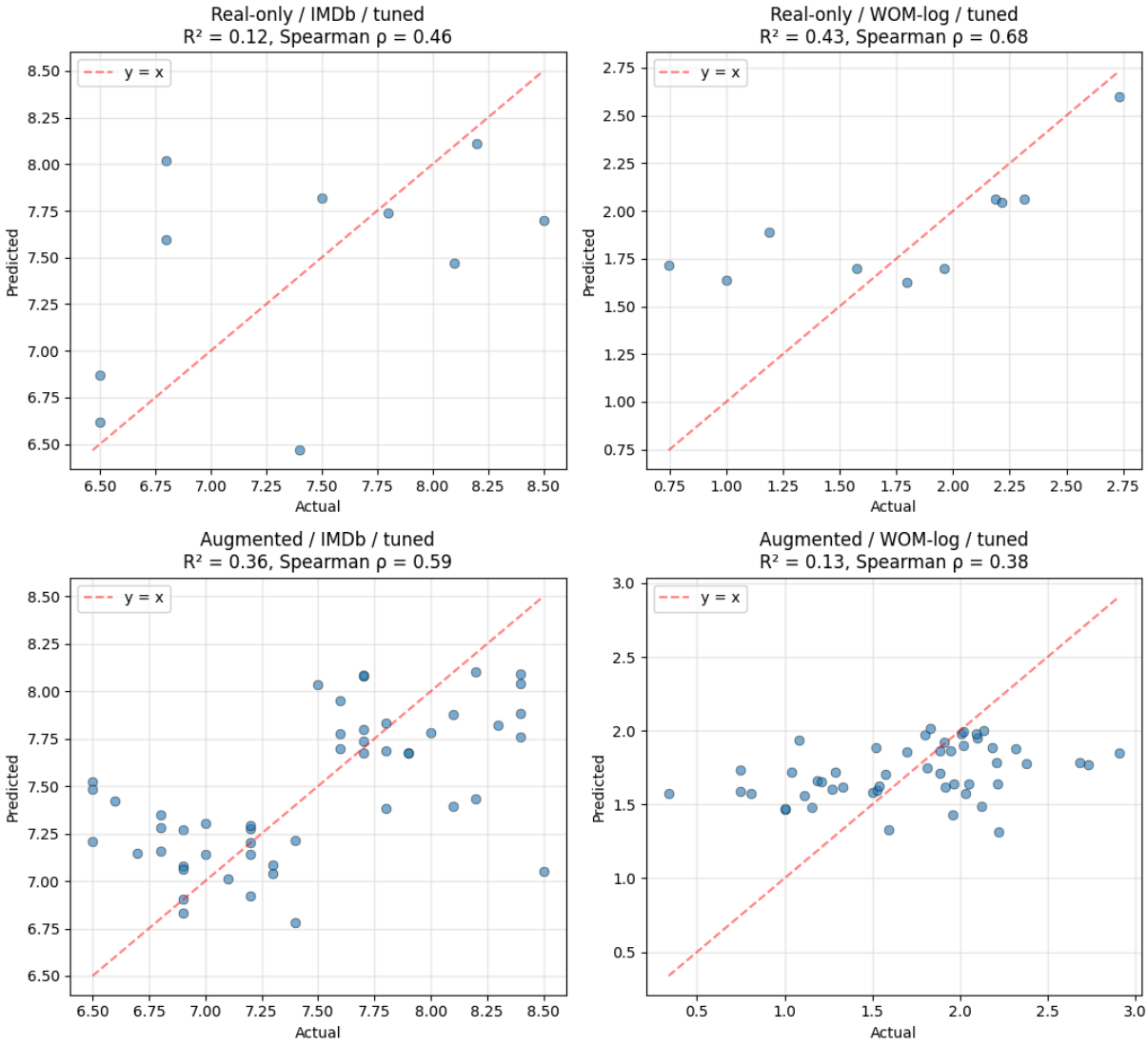
```python
# Augmented — same wider grid
search = GridSearchCV(make_elastic_pipeline(), param_grid,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_imdb_all)
best_params_imdb = search.best_params_
y_pred_aug_imdb_tuned = kfold_predict(search.best_estimator_, X_all, y_imdb_
metrics_aug_imdb_tuned = regression_metrics(y_imdb_all, y_pred_aug_imdb_tune
print(f'\nBest IMDb params (augmented): {best_params_imdb}')
report(metrics_aug_imdb_tuned, 'Augmented / IMDb / tuned')

search = GridSearchCV(make_elastic_pipeline(), param_grid,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_wom_all)
best_params_wom = search.best_params_
y_pred_aug_wom_tuned = kfold_predict(search.best_estimator_, X_all, y_wom_al
metrics_aug_wom_tuned = regression_metrics(y_wom_all, y_pred_aug_wom_tuned)
print(f'\nBest WOM params (augmented): {best_params_wom}')
report(metrics_aug_wom_tuned, 'Augmented / WOM-log / tuned')
```

```
Best IMDb params (augmented): {'model__alpha': 1.0, 'model__l1_ratio': 0.05}

──── Augmented / IMDb / tuned ────
           n: 50
          r2: 0.365
         mae: 0.347
        rmse: 0.458
    spearman: 0.589
     pearson: 0.606

Best WOM params (augmented): {'model__alpha': 0.1778279410038923, 'model__l1
_ratio': 0.85}

──── Augmented / WOM-log / tuned ────
           n: 50
          r2: 0.131
         mae: 0.406
        rmse: 0.511
    spearman: 0.379
     pearson: 0.363
```

Visuals

In [11]:

```
fig, axes = plt.subplots(2, 2, figsize=(11, 10))
plot_pred_vs_actual(y_imdb_real, y_pred_real_imdb_tuned,
                    'Real-only / IMDb / tuned', axes[0, 0])
plot_pred_vs_actual(y_wom_real, y_pred_real_wom_tuned,
                    'Real-only / WOM-log / tuned', axes[0, 1])
plot_pred_vs_actual(y_imdb_all, y_pred_aug_imdb_tuned,
                    'Augmented / IMDb / tuned', axes[1, 0])
plot_pred_vs_actual(y_wom_all, y_pred_aug_wom_tuned,
                    'Augmented / WOM-log / tuned', axes[1, 1])
plt.tight_layout()
plt.show()
```



Coefficient analysis and top features

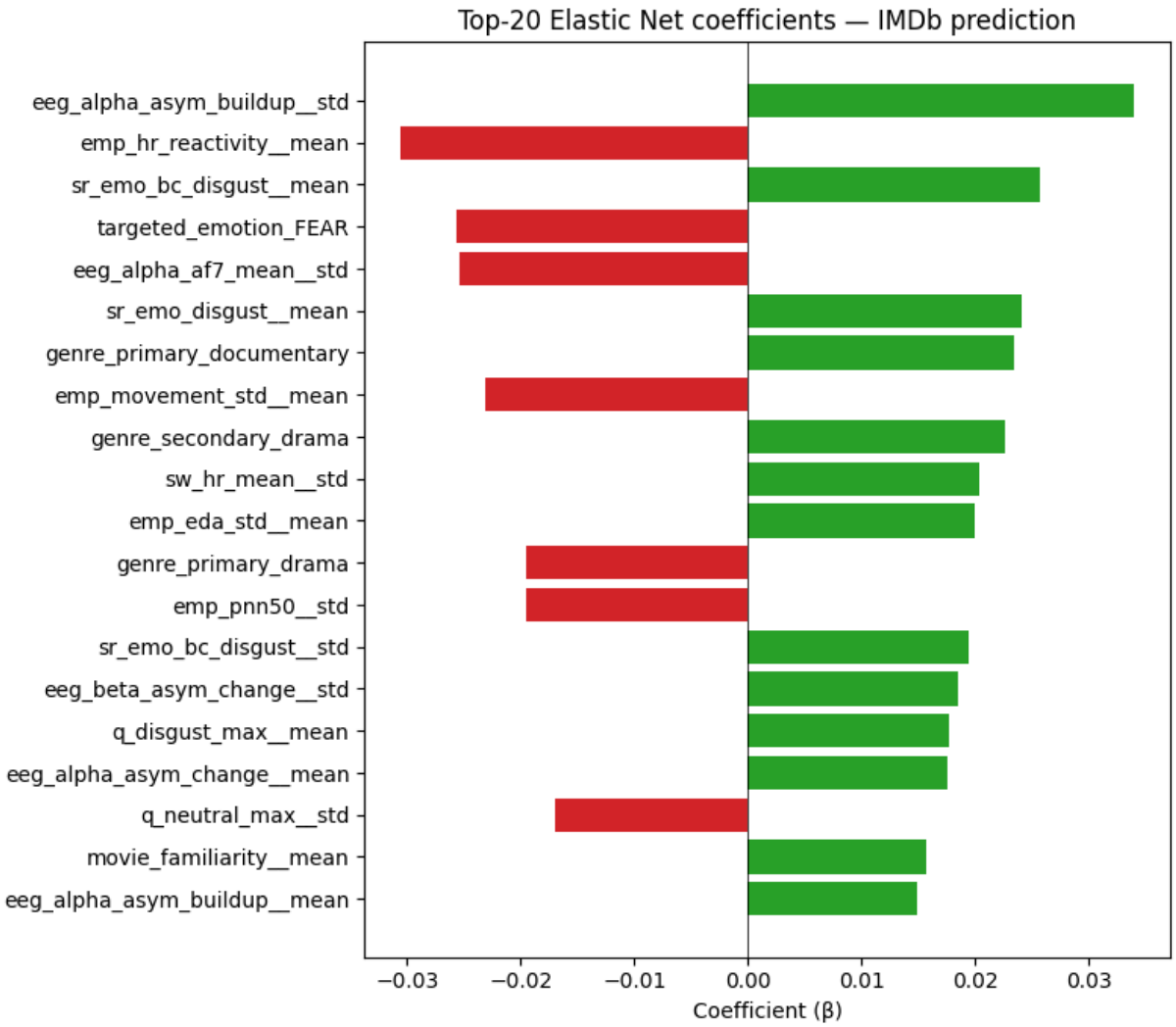
In [ ]:

```
# Refit on full augmented data with tuned params, and extract coefficients
final_pipe = make_elastic_pipeline(**{k.replace('model__', ''): v for k, v i
final_pipe.fit(X_all, y_imdb_all)
coefs = final_pipe.named_steps['model'].coef_

coef_df = (pd.DataFrame({'feature': feature_cols, 'coef': coefs})
           .assign(abs_coef=lambda d: d.coef.abs())
           .sort_values('abs_coef', ascending=False))

print(f'Non-zero coefficients: {(coefs != 0).sum()} / {len(coefs)}')
print(f'\nTop 30 features by |β|:')
display(coef_df.head(30).round(4))

# Visualise top 20
top = coef_df.head(20).iloc[::-1]
fig, ax = plt.subplots(figsize=(8, 7))
colors = ['#2ca02c' if c > 0 else '#d62728' for c in top['coef']]
ax.barh(top['feature'], top['coef'], color=colors)
```

```
ax.axvline(0, color='black', linewidth=0.5)
ax.set_xlabel('Coefficient (β)')
ax.set_title('Top-20 Elastic Net coefficients — IMDb prediction')
plt.tight_layout()
plt.show()
```

```
Non-zero coefficients: 78 / 360

Top 30 features by |β|:
```

	feature	coef	abs_coef
9	eeg_alpha_asym_buildup__std	0.0339	0.0339
60	emp_hr_reactivity__mean	-0.0305	0.0305
256	sr_emo_bc_disgust__mean	0.0257	0.0257
337	targeted_emotion_FEAR	-0.0256	0.0256
1	eeg_alpha_af7_mean__std	-0.0253	0.0253
268	sr_emo_disgust__mean	0.0241	0.0241
345	genre_primary_documentary	0.0234	0.0234
70	emp_movement_std__mean	-0.0230	0.0230
352	genre_secondary_drama	0.0226	0.0226
289	sw_hr_mean__std	0.0204	0.0204
50	emp_eda_std__mean	0.0199	0.0199
346	genre_primary_drama	-0.0195	0.0195
73	emp_pnn50__std	-0.0194	0.0194
257	sr_emo_bc_disgust__std	0.0194	0.0194
29	eeg_beta_asym_change__std	0.0185	0.0185
192	q_disgust_max__mean	0.0177	0.0177
10	eeg_alpha_asym_change__mean	0.0175	0.0175
221	q_neutral_max__std	-0.0169	0.0169
110	movie_familiarity__mean	0.0157	0.0157
8	eeg_alpha_asym_buildup__mean	0.0149	0.0149
139	of_duchenne_fraction_bc__std	-0.0142	0.0142
333	targeted_emotion_ANGER	0.0139	0.0139
220	q_neutral_max__mean	0.0136	0.0136
51	emp_eda_std__std	0.0134	0.0134
111	movie_familiarity__std	0.0133	0.0133
288	sw_hr_mean__mean	0.0131	0.0131
28	eeg_beta_asym_change__mean	0.0127	0.0127
246	sr_emo_anger__mean	0.0126	0.0126
261	sr_emo_bc_fear__std	-0.0125	0.0125
293	sw_hr_peak_timing__std	-0.0119	0.0119



Save results

In [ ]:

```python
# Save results
out_path = RUN_DIR / 'results_elastic_net.json'
results_to_save = {
    'model': 'elastic_net',
    'feature_count': len(feature_cols),
    'best_params_imdb': best_params_imdb,
    'best_params_wom':  best_params_wom,
    'metrics_real_imdb_baseline': metrics_real_imdb_baseline,
    'metrics_real_wom_baseline':  metrics_real_wom_baseline,
    'metrics_real_imdb_tuned':    metrics_real_imdb_tuned,
    'metrics_real_wom_tuned':     metrics_real_wom_tuned,
    'metrics_aug_imdb_baseline':  metrics_aug_imdb_baseline,
    'metrics_aug_wom_baseline':   metrics_aug_wom_baseline,
    'metrics_aug_imdb_tuned':     metrics_aug_imdb_tuned,
    'metrics_aug_wom_tuned':      metrics_aug_wom_tuned,
}
with open(out_path, 'w') as f:
    json.dump(results_to_save, f, indent=2, default=str)
```

```
print(f'\nSaved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v2/results_elastic_net.json
```

02

NOTEBOOK

Ridge regression

02_ridge.pdf

Ridge Regression

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)

from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')         # adjust if need
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v2'
RUN_DIR.mkdir(parents=True, exist_ok=True)


for f in ['movie_features_v6.csv',
          'movie_features_v6_synthetic.csv',
          'scene_movie_metadata_v6.csv',
          'scene_movie_metadata_v6_synthetic.csv']:
    assert (DATA_DIR / f).exists(), f'Missing: {f}'

print('Data dir:', DATA_DIR)
print('Run dir: ', RUN_DIR)
```

```
Mounted at /content/drive
Data dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6
Run dir:  /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6/colab_runs_v2
```

In [ ]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, StratifiedKFold, GridSearchCV, cross_val_predict,
)
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
)
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
pd.set_option('display.width', 180)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

Data loading

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')


LEAKAGE_AND_OLD_TARGETS = [
    'budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_mov = real_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')


META_KEEP = [
    'movie_id',
    # Scene properties
    'targeted_emotion', 'clip_duration_s',
    'cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
    'silence_ratio', 'music_presence', 'dialogue_density',
    'face_screen_time_ratio', 'lead_screen_time_ratio',
    # Movie metadata
    'release_year', 'genre_primary', 'genre_secondary',
    'country_of_origin', 'budget_categorical',
    # Targets (kept to extract y; not used as features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]

real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left')
real_mov['is_synthetic'] = 0
syn_mov = syn_mov.merge(syn_meta_sub, on='movie_id', how='left')
syn_mov['is_synthetic'] = 1

df_all = pd.concat([real_mov, syn_mov], ignore_index=True)
print(f'Real:      {len(real_mov)} movies')
print(f'Synthetic: {len(syn_mov)} movies')
print(f'Combined:  {len(df_all)} movies × {len(df_all.columns)} columns (pre
print()
print('Target distributions (pre-encoding):')
display(df_all[['is_synthetic', 'imdb_rating', 'wom_multiplier_log']]
        .groupby('is_synthetic').describe().round(2))
```

```
Real:      10 movies
Synthetic: 40 movies
Combined:  50 movies × 343 columns (pre-encoding)

Target distributions (pre-encoding):
```

	imdb_rating											
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min
is_synthetic												
0	10.0	7.41	0.73	6.5	6.8	7.45	8.02	8.5	10.0	1.77	0.64	0.75
1	40.0	7.48	0.55	6.5	7.0	7.50	7.82	8.4	40.0	1.70	0.54	0.34

Feature preparation

In [ ]:

```
DROP = {
    'movie_id', 'condition', 'n_participants',
    # Targets (cannot be features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
    # Control flag (not a feature)
    'is_synthetic',
}

df_feat = df_all.copy()

ORD_MAP = {'low': 1, 'moderate': 2, 'high': 3}
ORD_COLS = ['cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
            'silence_ratio', 'music_presence', 'dialogue_density',
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'budget_categorical']
for c in ORD_COLS:
    if c in df_feat.columns:
        df_feat[c] = df_feat[c].map(ORD_MAP)

OH_COLS = ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country_
OH_COLS = [c for c in OH_COLS if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH_COLS, prefix_sep='_',
                         dummy_na=False, dtype=int)


feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
groups_all = df_feat['movie_id'].values if 'movie_id' in df_feat.columns els
synth_mask_all = df_feat['is_synthetic'].values.astype(bool)


all_nan_cols = X_all.columns[X_all.isna().all()].tolist()
if all_nan_cols:
    print(f'Dropping {len(all_nan_cols)} all-NaN columns')
    X_all = X_all.drop(columns=all_nan_cols)


zero_var_cols = X_all.columns[X_all.std() == 0].tolist()
if zero_var_cols:
```

```
    print(f'Dropping {len(zero_var_cols)} zero-variance columns')
    X_all = X_all.drop(columns=zero_var_cols)

feature_cols = list(X_all.columns)

phys_cols = [c for c in feature_cols if c.endswith('__mean') or c.endswith('
ord_in_X  = [c for c in feature_cols if c in ORD_COLS]
oh_in_X   = [c for c in feature_cols
             if any(c.startswith(p + '_') for p in
                    ['targeted_emotion', 'genre_primary', 'genre_secondary',
                     'country_of_origin'])]
other_cols = [c for c in feature_cols
              if c not in phys_cols + ord_in_X + oh_in_X]

print(f'\nFinal feature matrix: {X_all.shape}')
print(f'  Physiological (mean/std):  {len(phys_cols):>4d}')
print(f'  Scene/movie ordinal:       {len(ord_in_X):>4d}  ({ord_in_X})')
print(f'  Scene/movie one-hot:       {len(oh_in_X):>4d}')
print(f'  Other numeric (year, dur): {len(other_cols):>4d}  ({other_cols})')
print(f'  TOTAL features:            {len(feature_cols):>4d}')
```

```
Final feature matrix: (50, 360)
  Physiological (mean/std):   320
  Scene/movie ordinal:         10  (['cut_count', 'brightness', 'motion_inte
nsity', 'audio_loudness', 'silence_ratio', 'music_presence', 'dialogue_densi
ty', 'face_screen_time_ratio', 'lead_screen_time_ratio', 'budget_categorica
l'])
  Scene/movie one-hot:         28
  Other numeric (year, dur):    2  (['clip_duration_s', 'release_year'])
  TOTAL features:             360
```

Evaluation helpers

In [ ]:

```
def regression_metrics(y_true, y_pred):
    """Compute R², MAE, RMSE, Spearman, Pearson."""
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    if len(yt) < 3:
        return {'n': len(yt), 'r2': np.nan, 'mae': np.nan, 'rmse': np.nan,
                'spearman': np.nan, 'pearson': np.nan}
    return {
        'n': len(yt),
        'r2': r2_score(yt, yp),
        'mae': mean_absolute_error(yt, yp),
        'rmse': np.sqrt(mean_squared_error(yt, yp)),
        'spearman': spearmanr(yt, yp).correlation,
        'pearson': pearsonr(yt, yp)[0],
    }


def loo_predict(estimator, X, y):
    """Leave-one-out cross-validated predictions."""
    loo = LeaveOneOut()
    return cross_val_predict(estimator, X, y, cv=loo, n_jobs=-1)
```

```
def kfold_predict(estimator, X, y, n_splits=5, random_state=42):
    """K-fold CV predictions (k=5 default)."""
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    return cross_val_predict(estimator, X, y, cv=kf, n_jobs=-1)


def report(metrics, title=''):
    print(f'\n——— {title} ———')
    for k, v in metrics.items():
        if k == 'n':
            print(f'  {k:>10s}: {v}')
        else:
            print(f'  {k:>10s}: {v:.3f}' if not np.isnan(v) else f'  {k:>10s
```

```
def plot_pred_vs_actual(y_true, y_pred, title, ax=None):
    if ax is None:
        fig, ax = plt.subplots(figsize=(5, 5))
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    ax.scatter(yt, yp, alpha=0.6, s=40, edgecolor='k', linewidth=0.5)
    lo, hi = min(yt.min(), yp.min()), max(yt.max(), yp.max())
    ax.plot([lo, hi], [lo, hi], 'r--', alpha=0.5, label='y = x')
    r2 = r2_score(yt, yp)
    rho = spearmanr(yt, yp).correlation
    ax.set_xlabel('Actual')
    ax.set_ylabel('Predicted')
    ax.set_title(f'{title}\nR² = {r2:.2f}, Spearman ρ = {rho:.2f}')
    ax.legend()
    ax.grid(alpha=0.3)
    return ax
```

Real-only (LOO CV, n = 10)

In [ ]:

```
from sklearn.linear_model import Ridge

mask_real = ~synth_mask_all
X_real = X_all[mask_real].reset_index(drop=True)
y_imdb_real = y_imdb_all[mask_real].reset_index(drop=True)
y_wom_real = y_wom_all[mask_real].reset_index(drop=True)

def make_ridge_pipeline(alpha=1.0):
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('model', Ridge(alpha=alpha, random_state=42)),
    ])

# Baseline
pipe = make_ridge_pipeline(alpha=1.0)
y_pred = loo_predict(pipe, X_real, y_imdb_real)
metrics_real_imdb_baseline = regression_metrics(y_imdb_real, y_pred)
```

```
report(metrics_real_imdb_baseline, 'Real / IMDb / baseline')

pipe = make_ridge_pipeline(alpha=1.0)
y_pred = loo_predict(pipe, X_real, y_wom_real)
metrics_real_wom_baseline = regression_metrics(y_wom_real, y_pred)
report(metrics_real_wom_baseline, 'Real / WOM-log / baseline')
```

```
──── Real / IMDb / baseline ────
           n: 10
          r2: -0.973
         mae: 0.898
        rmse: 0.978
    spearman: -0.713
     pearson: -0.678

──── Real / WOM-log / baseline ────
           n: 10
          r2: -0.759
         mae: 0.718
        rmse: 0.802
    spearman: -0.770
     pearson: -0.760
```

Tuned

In [ ]:
```
# Wider alpha grid in log-space.
param_grid = {'model__alpha': np.logspace(-4, 4, 30).tolist()}

search = GridSearchCV(make_ridge_pipeline(), param_grid,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
                      n_jobs=-1, refit=True)
search.fit(X_real, y_imdb_real)
best_params_imdb_real = search.best_params_
y_pred_real_imdb_tuned = loo_predict(search.best_estimator_, X_real, y_imdb_
metrics_real_imdb_tuned = regression_metrics(y_imdb_real, y_pred_real_imdb_t
print(f'\nBest IMDb params: {best_params_imdb_real}')
report(metrics_real_imdb_tuned, 'Real / IMDb / tuned')

search = GridSearchCV(make_ridge_pipeline(), param_grid,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
                      n_jobs=-1, refit=True)
search.fit(X_real, y_wom_real)
best_params_wom_real = search.best_params_
y_pred_real_wom_tuned = loo_predict(search.best_estimator_, X_real, y_wom_re
metrics_real_wom_tuned = regression_metrics(y_wom_real, y_pred_real_wom_tune
print(f'\nBest WOM params: {best_params_wom_real}')
report(metrics_real_wom_tuned, 'Real / WOM-log / tuned')
```

```
Best IMDb params: {'model__alpha': 10000.0}

——— Real / IMDb / tuned ———
           n: 10
          r2: -0.260
         mae: 0.685
        rmse: 0.782
    spearman: -0.970
     pearson: -0.991

Best WOM params: {'model__alpha': 10000.0}

——— Real / WOM-log / tuned ———
           n: 10
          r2: -0.254
         mae: 0.578
        rmse: 0.677
    spearman: -1.000
     pearson: -0.996
```

Augmented (5-fold CV, n = 50)

In [ ]:
```python
# Baseline
pipe = make_ridge_pipeline(alpha=1.0)
y_pred_aug_imdb = kfold_predict(pipe, X_all, y_imdb_all, n_splits=5)
metrics_aug_imdb_baseline = regression_metrics(y_imdb_all, y_pred_aug_imdb)
report(metrics_aug_imdb_baseline, 'Augmented / IMDb / baseline')

pipe = make_ridge_pipeline(alpha=1.0)
y_pred_aug_wom = kfold_predict(pipe, X_all, y_wom_all, n_splits=5)
metrics_aug_wom_baseline = regression_metrics(y_wom_all, y_pred_aug_wom)
report(metrics_aug_wom_baseline, 'Augmented / WOM-log / baseline')
```

```
——— Augmented / IMDb / baseline ———
           n: 50
          r2: 0.380
         mae: 0.354
        rmse: 0.453
    spearman: 0.632
     pearson: 0.646

——— Augmented / WOM-log / baseline ———
           n: 50
          r2: 0.051
         mae: 0.406
        rmse: 0.534
    spearman: 0.347
     pearson: 0.369
```

In [ ]:
```python
# Augmented — same wider alpha grid
search = GridSearchCV(make_ridge_pipeline(), param_grid,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_imdb_all)
```

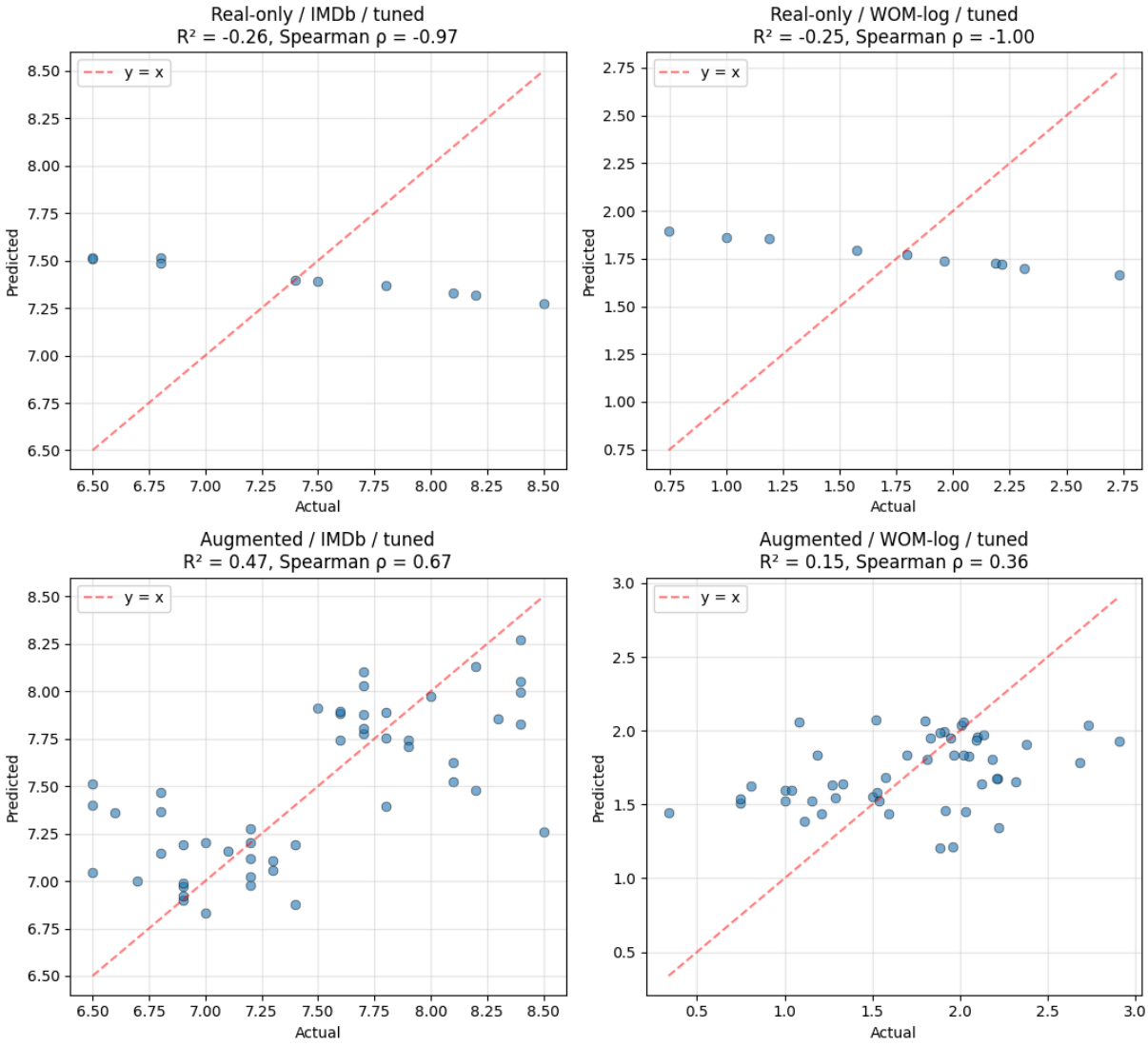
```
best_params_imdb = search.best_params_
y_pred_aug_imdb_tuned = kfold_predict(search.best_estimator_, X_all, y_imdb_
metrics_aug_imdb_tuned = regression_metrics(y_imdb_all, y_pred_aug_imdb_tune
print(f'\nBest IMDb params: {best_params_imdb}')
report(metrics_aug_imdb_tuned, 'Augmented / IMDb / tuned')

search = GridSearchCV(make_ridge_pipeline(), param_grid,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_wom_all)
best_params_wom = search.best_params_
y_pred_aug_wom_tuned = kfold_predict(search.best_estimator_, X_all, y_wom_al
metrics_aug_wom_tuned = regression_metrics(y_wom_all, y_pred_aug_wom_tuned)
print(f'\nBest WOM params: {best_params_wom}')
report(metrics_aug_wom_tuned, 'Augmented / WOM-log / tuned')
```

```
Best IMDb params: {'model__alpha': 117.21022975334793}

──── Augmented / IMDb / tuned ────
           n: 50
          r2: 0.472
         mae: 0.317
        rmse: 0.418
    spearman: 0.668
     pearson: 0.687

Best WOM params: {'model__alpha': 417.53189365604004}

──── Augmented / WOM-log / tuned ────
           n: 50
          r2: 0.147
         mae: 0.406
        rmse: 0.507
    spearman: 0.358
     pearson: 0.385
```

Visuals

In [ ]:

```
fig, axes = plt.subplots(2, 2, figsize=(11, 10))
plot_pred_vs_actual(y_imdb_real, y_pred_real_imdb_tuned,
                    'Real-only / IMDb / tuned', axes[0, 0])
plot_pred_vs_actual(y_wom_real, y_pred_real_wom_tuned,
                    'Real-only / WOM-log / tuned', axes[0, 1])
plot_pred_vs_actual(y_imdb_all, y_pred_aug_imdb_tuned,
                    'Augmented / IMDb / tuned', axes[1, 0])
plot_pred_vs_actual(y_wom_all, y_pred_aug_wom_tuned,
                    'Augmented / WOM-log / tuned', axes[1, 1])
plt.tight_layout()
plt.show()
```



Coefficients, top features

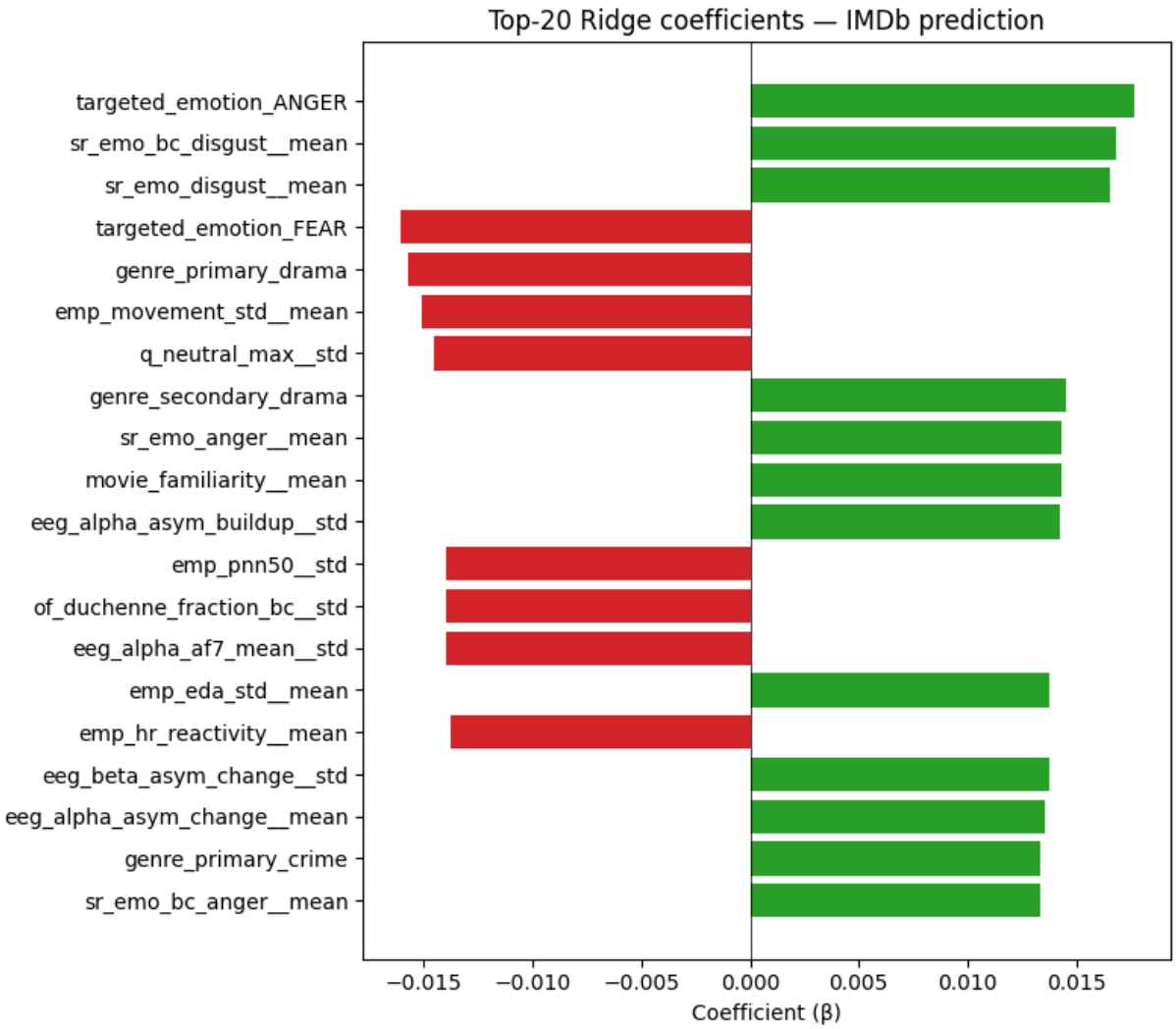
In [ ]:

```
final_pipe = make_ridge_pipeline(**{k.replace('model__', ''): v for k, v in
final_pipe.fit(X_all, y_imdb_all)
coefs = final_pipe.named_steps['model'].coef_

coef_df = (pd.DataFrame({'feature': feature_cols, 'coef': coefs})
           .assign(abs_coef=lambda d: d.coef.abs())
           .sort_values('abs_coef', ascending=False))
print('Top 30 ridge coefficients:')
display(coef_df.head(30).round(4))

top = coef_df.head(20).iloc[::-1]
fig, ax = plt.subplots(figsize=(8, 7))
colors = ['#2ca02c' if c > 0 else '#d62728' for c in top['coef']]
ax.barh(top['feature'], top['coef'], color=colors)
ax.axvline(0, color='black', linewidth=0.5)
ax.set_xlabel('Coefficient (β)')
ax.set_title('Top-20 Ridge coefficients — IMDb prediction')
```

```
plt.tight_layout()
plt.show()
```

```
Top 30 ridge coefficients:
```

	feature	coef	abs_coef
333	targeted_emotion_ANGER	0.0177	0.0177
256	sr_emo_bc_disgust__mean	0.0169	0.0169
268	sr_emo_disgust__mean	0.0166	0.0166
337	targeted_emotion_FEAR	-0.0161	0.0161
346	genre_primary_drama	-0.0157	0.0157
70	emp_movement_std__mean	-0.0151	0.0151
221	q_neutral_max__std	-0.0145	0.0145
352	genre_secondary_drama	0.0145	0.0145
246	sr_emo_anger__mean	0.0143	0.0143
110	movie_familiarity__mean	0.0143	0.0143
9	eeg_alpha_asym_buildup__std	0.0142	0.0142
73	emp_pnn50__std	-0.0140	0.0140
139	of_duchenne_fraction_bc__std	-0.0140	0.0140
1	eeg_alpha_af7_mean__std	-0.0140	0.0140
50	emp_eda_std__mean	0.0138	0.0138
60	emp_hr_reactivity__mean	-0.0138	0.0138
29	eeg_beta_asym_change__std	0.0137	0.0137
10	eeg_alpha_asym_change__mean	0.0136	0.0136
344	genre_primary_crime	0.0134	0.0134
252	sr_emo_bc_anger__mean	0.0133	0.0133
51	emp_eda_std__std	0.0128	0.0128
220	q_neutral_max__mean	0.0126	0.0126
313	sw_ppi_rmssd__std	0.0126	0.0126
8	eeg_alpha_asym_buildup__mean	0.0125	0.0125
289	sw_hr_mean__std	0.0125	0.0125
27	eeg_alpha_variability__std	-0.0124	0.0124
141	of_gaze_stability__std	-0.0124	0.0124
345	genre_primary_documentary	0.0121	0.0121
123	of_blink_rate_bc__std	0.0119	0.0119
229	q_peak_emotion_timing__std	-0.0113	0.0113



Save results

In [ ]:
```python
# Save results
out_path = RUN_DIR / 'results_ridge.json'
results_to_save = {
    'model': 'ridge',
    'feature_count': len(feature_cols),
    'best_params_imdb': best_params_imdb,
    'best_params_wom':  best_params_wom,
    'metrics_real_imdb_baseline': metrics_real_imdb_baseline,
    'metrics_real_wom_baseline':  metrics_real_wom_baseline,
    'metrics_real_imdb_tuned':    metrics_real_imdb_tuned,
    'metrics_real_wom_tuned':     metrics_real_wom_tuned,
    'metrics_aug_imdb_baseline':  metrics_aug_imdb_baseline,
    'metrics_aug_wom_baseline':   metrics_aug_wom_baseline,
    'metrics_aug_imdb_tuned':     metrics_aug_imdb_tuned,
    'metrics_aug_wom_tuned':      metrics_aug_wom_tuned,
}
with open(out_path, 'w') as f:
    json.dump(results_to_save, f, indent=2, default=str)
```

```
print(f'\nSaved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v2/results_ridge.json
```

03

NOTEBOOK

Random Forest regression

03_random_forest.pdf

Random Forest

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)

from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')         # adjust if need
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v2'
RUN_DIR.mkdir(parents=True, exist_ok=True)

# Verify data files exist
for f in ['movie_features_v6.csv',
          'movie_features_v6_synthetic.csv',
          'scene_movie_metadata_v6.csv',
          'scene_movie_metadata_v6_synthetic.csv']:
    assert (DATA_DIR / f).exists(), f'Missing: {f}'

print('Data dir:', DATA_DIR)
print('Run dir: ', RUN_DIR)
```

```
Mounted at /content/drive
Data dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6
Run dir:  /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6/colab_runs_v2
```

In [2]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, StratifiedKFold, GridSearchCV, cross_val_predict,
)
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
)
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
pd.set_option('display.width', 180)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

Data loading

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')


LEAKAGE_AND_OLD_TARGETS = [
    'budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_mov = real_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')


META_KEEP = [
    'movie_id',
    # Scene properties (will be encoded below)
    'targeted_emotion', 'clip_duration_s',
    'cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
    'silence_ratio', 'music_presence', 'dialogue_density',
    'face_screen_time_ratio', 'lead_screen_time_ratio',
    # Movie metadata
    'release_year', 'genre_primary', 'genre_secondary',
    'country_of_origin', 'budget_categorical',
    # Targets (kept to extract y; not used as features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]

real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left')
real_mov['is_synthetic'] = 0
syn_mov = syn_mov.merge(syn_meta_sub, on='movie_id', how='left')
syn_mov['is_synthetic'] = 1

df_all = pd.concat([real_mov, syn_mov], ignore_index=True)
print(f'Real:      {len(real_mov)} movies')
print(f'Synthetic: {len(syn_mov)} movies')
print(f'Combined:  {len(df_all)} movies × {len(df_all.columns)} columns (pre
print()
print('Target distributions (pre-encoding):')
display(df_all[['is_synthetic', 'imdb_rating', 'wom_multiplier_log']]
        .groupby('is_synthetic').describe().round(2))
```

```
Real:      10 movies
Synthetic: 40 movies
Combined:  50 movies × 343 columns (pre-encoding)

Target distributions (pre-encoding):
```

	imdb_rating											
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min
is_synthetic												
0	10.0	7.41	0.73	6.5	6.8	7.45	8.02	8.5	10.0	1.77	0.64	0.75
1	40.0	7.48	0.55	6.5	7.0	7.50	7.82	8.4	40.0	1.70	0.54	0.34

3 · Feature preparation

In [ ]:

```
DROP = {
    'movie_id', 'condition', 'n_participants',
    # Targets (cannot be features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
    # Control flag (not a feature)
    'is_synthetic',
}

df_feat = df_all.copy()

# ── Ordinal encoding: low / moderate / high → 1 / 2 / 3
ORD_MAP = {'low': 1, 'moderate': 2, 'high': 3}
ORD_COLS = ['cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
            'silence_ratio', 'music_presence', 'dialogue_density',
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'budget_categorical']
for c in ORD_COLS:
    if c in df_feat.columns:
        df_feat[c] = df_feat[c].map(ORD_MAP)


OH_COLS = ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country_
OH_COLS = [c for c in OH_COLS if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH_COLS, prefix_sep='_',
                         dummy_na=False, dtype=int)


feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
groups_all = df_feat['movie_id'].values if 'movie_id' in df_feat.columns els
synth_mask_all = df_feat['is_synthetic'].values.astype(bool)

# Drop all-NaN columns
all_nan_cols = X_all.columns[X_all.isna().all()].tolist()
if all_nan_cols:
    print(f'Dropping {len(all_nan_cols)} all-NaN columns')
    X_all = X_all.drop(columns=all_nan_cols)

# Drop zero-variance columns
zero_var_cols = X_all.columns[X_all.std() == 0].tolist()
```

```
if zero_var_cols:
    print(f'Dropping {len(zero_var_cols)} zero-variance columns')
    X_all = X_all.drop(columns=zero_var_cols)

feature_cols = list(X_all.columns)

# Categorise final features for transparency
phys_cols = [c for c in feature_cols if c.endswith('__mean') or c.endswith('
ord_in_X  = [c for c in feature_cols if c in ORD_COLS]
oh_in_X   = [c for c in feature_cols
             if any(c.startswith(p + '_') for p in
                    ['targeted_emotion', 'genre_primary', 'genre_secondary',
                     'country_of_origin'])]
other_cols = [c for c in feature_cols
              if c not in phys_cols + ord_in_X + oh_in_X]

print(f'\nFinal feature matrix: {X_all.shape}')
print(f'  Physiological (mean/std):  {len(phys_cols):>4d}')
print(f'  Scene/movie ordinal:       {len(ord_in_X):>4d}  ({ord_in_X})')
print(f'  Scene/movie one-hot:       {len(oh_in_X):>4d}')
print(f'  Other numeric (year, dur): {len(other_cols):>4d}  ({other_cols})')
print(f'  TOTAL features:            {len(feature_cols):>4d}')
```

```
Final feature matrix: (50, 360)
  Physiological (mean/std):   320
  Scene/movie ordinal:         10  (['cut_count', 'brightness', 'motion_inte
nsity', 'audio_loudness', 'silence_ratio', 'music_presence', 'dialogue_densi
ty', 'face_screen_time_ratio', 'lead_screen_time_ratio', 'budget_categorica
l'])
  Scene/movie one-hot:         28
  Other numeric (year, dur):    2  (['clip_duration_s', 'release_year'])
  TOTAL features:             360
```

Evaluation helpers

In [ ]:

```
def regression_metrics(y_true, y_pred):
    """Compute R², MAE, RMSE, Spearman, Pearson."""
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    if len(yt) < 3:
        return {'n': len(yt), 'r2': np.nan, 'mae': np.nan, 'rmse': np.nan,
                'spearman': np.nan, 'pearson': np.nan}
    return {
        'n': len(yt),
        'r2': r2_score(yt, yp),
        'mae': mean_absolute_error(yt, yp),
        'rmse': np.sqrt(mean_squared_error(yt, yp)),
        'spearman': spearmanr(yt, yp).correlation,
        'pearson': pearsonr(yt, yp)[0],
    }


def loo_predict(estimator, X, y):
    """Leave-one-out cross-validated predictions."""
```

```
    loo = LeaveOneOut()
    return cross_val_predict(estimator, X, y, cv=loo, n_jobs=-1)


def kfold_predict(estimator, X, y, n_splits=5, random_state=42):
    """K-fold CV predictions (k=5 default)."""
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    return cross_val_predict(estimator, X, y, cv=kf, n_jobs=-1)


def report(metrics, title=''):
    print(f'\n——— {title} ———')
    for k, v in metrics.items():
        if k == 'n':
            print(f'  {k:>10s}: {v}')
        else:
            print(f'  {k:>10s}: {v:.3f}' if not np.isnan(v) else f'  {k:>10s
```

```
def plot_pred_vs_actual(y_true, y_pred, title, ax=None):
    if ax is None:
        fig, ax = plt.subplots(figsize=(5, 5))
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    ax.scatter(yt, yp, alpha=0.6, s=40, edgecolor='k', linewidth=0.5)
    lo, hi = min(yt.min(), yp.min()), max(yt.max(), yp.max())
    ax.plot([lo, hi], [lo, hi], 'r--', alpha=0.5, label='y = x')
    r2 = r2_score(yt, yp)
    rho = spearmanr(yt, yp).correlation
    ax.set_xlabel('Actual')
    ax.set_ylabel('Predicted')
    ax.set_title(f'{title}\nR² = {r2:.2f}, Spearman ρ = {rho:.2f}')
    ax.legend()
    ax.grid(alpha=0.3)
    return ax
```

Real-only (LOO CV, n = 10)

In [12]:

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance

mask_real = ~synth_mask_all
X_real = X_all[mask_real].reset_index(drop=True)
y_imdb_real = y_imdb_all[mask_real].reset_index(drop=True)
y_wom_real = y_wom_all[mask_real].reset_index(drop=True)

def make_rf_pipeline(n_estimators=300, max_depth=None,
                     min_samples_leaf=1, max_features='sqrt',
                     min_samples_split=2):
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('model', RandomForestRegressor(
            n_estimators=n_estimators, max_depth=max_depth,
            min_samples_leaf=min_samples_leaf, max_features=max_features,
```

```
            min_samples_split=min_samples_split, # Added this line
            random_state=42, n_jobs=-1)),
    ])

# Baseline (default)
pipe = make_rf_pipeline()
y_pred = loo_predict(pipe, X_real, y_imdb_real)
metrics_real_imdb_baseline = regression_metrics(y_imdb_real, y_pred)
report(metrics_real_imdb_baseline, 'Real / IMDb / baseline')

pipe = make_rf_pipeline()
y_pred = loo_predict(pipe, X_real, y_wom_real)
metrics_real_wom_baseline = regression_metrics(y_wom_real, y_pred)
report(metrics_real_wom_baseline, 'Real / WOM-log / baseline')
```

```
──── Real / IMDb / baseline ────
         n: 10
        r2: -0.458
       mae: 0.734
      rmse: 0.841
  spearman: -0.841
   pearson: -0.830

──── Real / WOM-log / baseline ────
         n: 10
        r2: -0.378
       mae: 0.614
      rmse: 0.709
  spearman: -0.636
   pearson: -0.747
```

Tuned (constrained grid for small n)

In [ ]:

```
param_grid = {
    'model__n_estimators':     [200, 500],
    'model__max_depth':        [3, 5, 7, 10, None],
    'model__min_samples_leaf': [1, 2, 4],
    'model__max_features':    ['sqrt', 0.2, 0.3, 0.5],
    'model__min_samples_split':[2, 5],
}

search = GridSearchCV(make_rf_pipeline(), param_grid,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
                      n_jobs=-1, refit=True)
search.fit(X_real, y_imdb_real)
best_params_imdb_real = search.best_params_
y_pred_real_imdb_tuned = loo_predict(search.best_estimator_, X_real, y_imdb_
metrics_real_imdb_tuned = regression_metrics(y_imdb_real, y_pred_real_imdb_t
print(f'\nBest IMDb params: {best_params_imdb_real}')
report(metrics_real_imdb_tuned, 'Real / IMDb / tuned')

search = GridSearchCV(make_rf_pipeline(), param_grid,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
                      n_jobs=-1, refit=True)
```

```
search.fit(X_real, y_wom_real)
best_params_wom_real = search.best_params_
y_pred_real_wom_tuned = loo_predict(search.best_estimator_, X_real, y_wom_re
metrics_real_wom_tuned = regression_metrics(y_wom_real, y_pred_real_wom_tune
print(f'\nBest WOM params: {best_params_wom_real}')
report(metrics_real_wom_tuned, 'Real / WOM-log / tuned')
```

```
Best IMDb params: {'model__max_depth': None, 'model__max_features': 'sqrt',
'model__min_samples_leaf': 4, 'model__min_samples_split': 2, 'model__n_estim
ators': 500}

──── Real / IMDb / tuned ────
           n: 10
          r2: -0.252
         mae: 0.685
        rmse: 0.779
    spearman: -0.994
     pearson: -0.999

Best WOM params: {'model__max_depth': 5, 'model__max_features': 0.5, 'model_
_min_samples_leaf': 4, 'model__min_samples_split': 5, 'model__n_estimators':
200}

──── Real / WOM-log / tuned ────
           n: 10
          r2: -0.244
         mae: 0.570
        rmse: 0.674
    spearman: -0.988
     pearson: -0.977
```

Augmented (5-fold CV, n = 50)

In [8]:

```
# Baseline
pipe = make_rf_pipeline()
y_pred_aug_imdb = kfold_predict(pipe, X_all, y_imdb_all, n_splits=5)
metrics_aug_imdb_baseline = regression_metrics(y_imdb_all, y_pred_aug_imdb)
report(metrics_aug_imdb_baseline, 'Augmented / IMDb / baseline')

pipe = make_rf_pipeline()
y_pred_aug_wom = kfold_predict(pipe, X_all, y_wom_all, n_splits=5)
metrics_aug_wom_baseline = regression_metrics(y_wom_all, y_pred_aug_wom)
report(metrics_aug_wom_baseline, 'Augmented / WOM-log / baseline')
```

```
──── Augmented / IMDb / baseline ────
           n: 50
          r2: 0.498
         mae: 0.319
        rmse: 0.407
    spearman: 0.678
     pearson: 0.748

──── Augmented / WOM-log / baseline ────
           n: 50
          r2: 0.198
         mae: 0.392
        rmse: 0.491
    spearman: 0.402
     pearson: 0.459
```

In [9]:

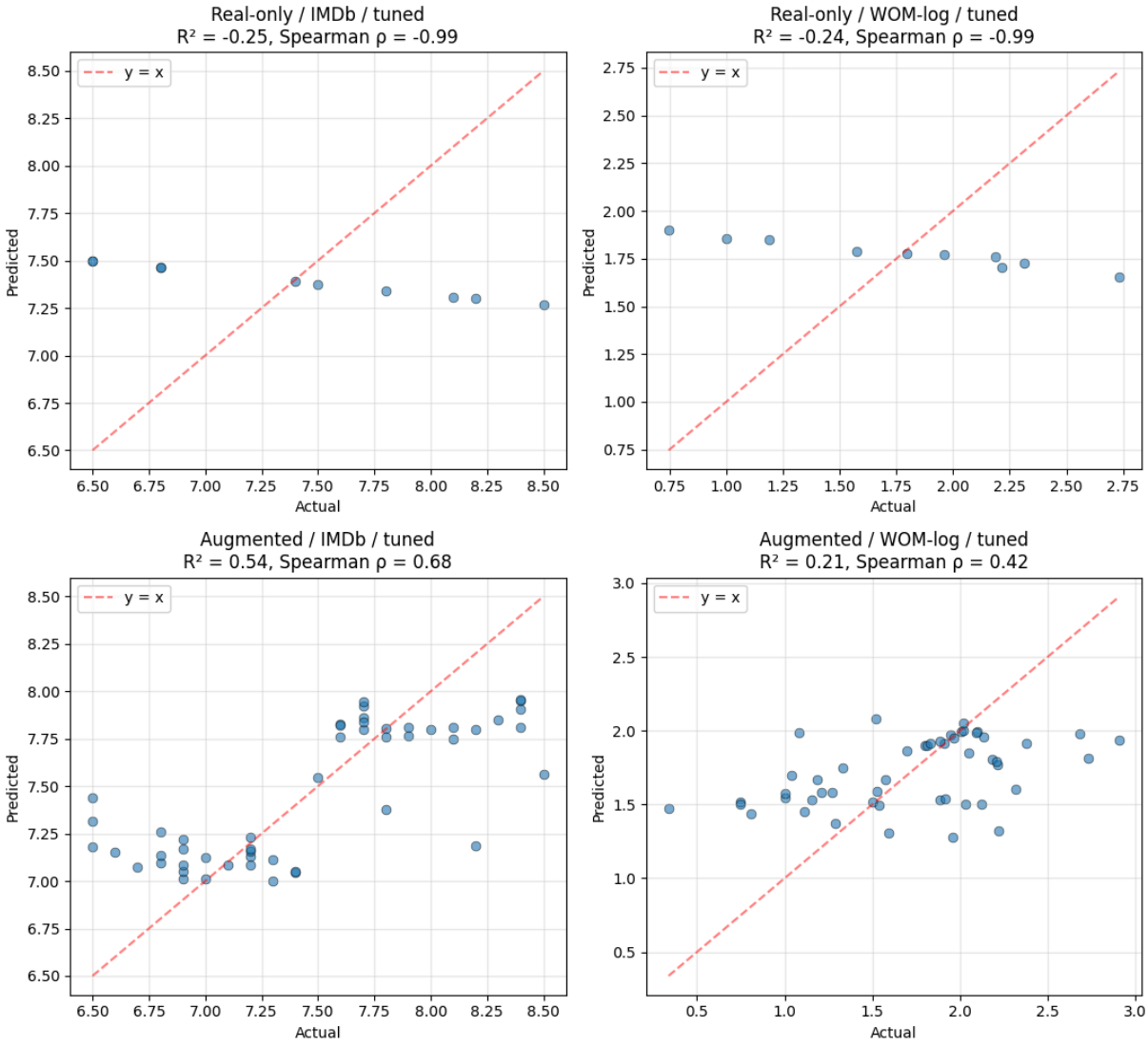
```python
# Augmented — same wider grid
search = GridSearchCV(make_rf_pipeline(), param_grid,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_imdb_all)
best_params_imdb = search.best_params_
y_pred_aug_imdb_tuned = kfold_predict(search.best_estimator_, X_all, y_imdb_
metrics_aug_imdb_tuned = regression_metrics(y_imdb_all, y_pred_aug_imdb_tune
print(f'\nBest IMDb params: {best_params_imdb}')
report(metrics_aug_imdb_tuned, 'Augmented / IMDb / tuned')

search = GridSearchCV(make_rf_pipeline(), param_grid,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_wom_all)
best_params_wom = search.best_params_
y_pred_aug_wom_tuned = kfold_predict(search.best_estimator_, X_all, y_wom_al
metrics_aug_wom_tuned = regression_metrics(y_wom_all, y_pred_aug_wom_tuned)
print(f'\nBest WOM params: {best_params_wom}')
report(metrics_aug_wom_tuned, 'Augmented / WOM-log / tuned')
```

```
Best IMDb params: {'model__max_depth': 7, 'model__max_features': 0.2, 'model
__min_samples_leaf': 1, 'model__min_samples_split': 2, 'model__n_estimator
s': 200}

──── Augmented / IMDb / tuned ────
           n: 50
          r2: 0.544
         mae: 0.299
        rmse: 0.388
    spearman: 0.683
     pearson: 0.753

Best WOM params: {'model__max_depth': 10, 'model__max_features': 0.2, 'model
__min_samples_leaf': 1, 'model__min_samples_split': 2, 'model__n_estimator
s': 200}

──── Augmented / WOM-log / tuned ────
           n: 50
          r2: 0.207
         mae: 0.381
        rmse: 0.488
    spearman: 0.415
     pearson: 0.458
```

Visuals

In [10]:

```python
fig, axes = plt.subplots(2, 2, figsize=(11, 10))
plot_pred_vs_actual(y_imdb_real, y_pred_real_imdb_tuned,
                    'Real-only / IMDb / tuned', axes[0, 0])
plot_pred_vs_actual(y_wom_real, y_pred_real_wom_tuned,
                    'Real-only / WOM-log / tuned', axes[0, 1])
plot_pred_vs_actual(y_imdb_all, y_pred_aug_imdb_tuned,
                    'Augmented / IMDb / tuned', axes[1, 0])
plot_pred_vs_actual(y_wom_all, y_pred_aug_wom_tuned,
                    'Augmented / WOM-log / tuned', axes[1, 1])
plt.tight_layout()
plt.show()
```



Permutation importance

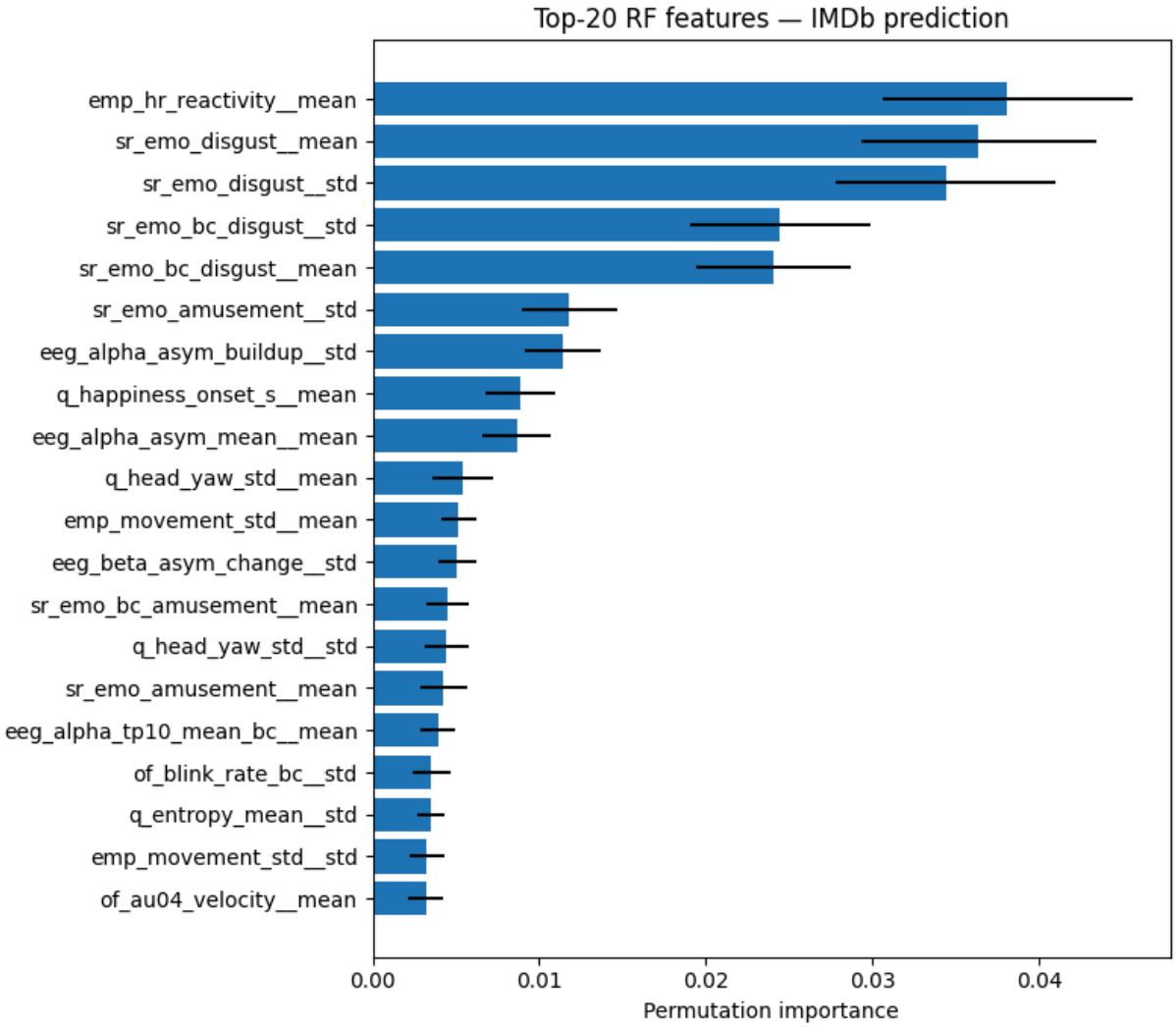
In [13]:

```
final_pipe = make_rf_pipeline(**{k.replace('model__', ''): v for k, v in bes
final_pipe.fit(X_all, y_imdb_all)

# Compute permutation importance on full data
X_imputed = final_pipe.named_steps['imputer'].transform(X_all)
perm = permutation_importance(
    final_pipe.named_steps['model'], X_imputed, y_imdb_all,
    n_repeats=20, random_state=42, n_jobs=-1, scoring='neg_mean_absolute_err
)

imp_df = (pd.DataFrame({
    'feature': feature_cols,
    'imp_mean': perm.importances_mean,
    'imp_std':  perm.importances_std,
}).sort_values('imp_mean', ascending=False))
print('Top 30 features by permutation importance:')
display(imp_df.head(30).round(4))
```

```
top = imp_df.head(20).iloc[::-1]
fig, ax = plt.subplots(figsize=(8, 7))
ax.barh(top['feature'], top['imp_mean'], xerr=top['imp_std'])
ax.set_xlabel('Permutation importance')
ax.set_title('Top-20 RF features — IMDb prediction')
plt.tight_layout()
plt.show()
```

```
Top 30 features by permutation importance:
```

	feature	imp_mean	imp_std
60	emp_hr_reactivity__mean	0.0381	0.0075
268	sr_emo_disgust__mean	0.0364	0.0071
269	sr_emo_disgust__std	0.0344	0.0066
257	sr_emo_bc_disgust__std	0.0245	0.0054
256	sr_emo_bc_disgust__mean	0.0241	0.0047
245	sr_emo_amusement__std	0.0118	0.0029
9	eeg_alpha_asym_buildup__std	0.0114	0.0023
208	q_happiness_onset_s__mean	0.0089	0.0021
12	eeg_alpha_asym_mean__mean	0.0086	0.0020
214	q_head_yaw_std__mean	0.0054	0.0018
70	emp_movement_std__mean	0.0052	0.0011
29	eeg_beta_asym_change__std	0.0051	0.0012
250	sr_emo_bc_amusement__mean	0.0045	0.0013
215	q_head_yaw_std__std	0.0044	0.0013
244	sr_emo_amusement__mean	0.0043	0.0014
20	eeg_alpha_tp10_mean_bc__mean	0.0039	0.0010
123	of_blink_rate_bc__std	0.0035	0.0011
199	q_entropy_mean__std	0.0035	0.0008
71	emp_movement_std__std	0.0033	0.0011
112	of_au04_velocity__mean	0.0032	0.0010
289	sw_hr_mean__std	0.0032	0.0009
220	q_neutral_max__mean	0.0025	0.0006
247	sr_emo_anger__std	0.0024	0.0006
206	q_happiness_mean_bc__mean	0.0023	0.0006
2	eeg_alpha_af7_mean_bc__mean	0.0023	0.0008
120	of_blink_interval_std__mean	0.0022	0.0008
151	of_lip_depress_mean_bc__std	0.0021	0.0008
190	q_anger_mean_bc__mean	0.0021	0.0004
192	q_disgust_max__mean	0.0020	0.0005
207	q_happiness_mean_bc__std	0.0019	0.0008



In [ ]:
```
out_path = RUN_DIR / 'results_random_forest.json'
results_to_save = {
    'model': 'random_forest',
    'feature_count': len(feature_cols),
    'best_params_imdb': best_params_imdb,
    'best_params_wom':  best_params_wom,
    'metrics_real_imdb_baseline': metrics_real_imdb_baseline,
    'metrics_real_wom_baseline':  metrics_real_wom_baseline,
    'metrics_real_imdb_tuned':    metrics_real_imdb_tuned,
    'metrics_real_wom_tuned':     metrics_real_wom_tuned,
    'metrics_aug_imdb_baseline':  metrics_aug_imdb_baseline,
    'metrics_aug_wom_baseline':   metrics_aug_wom_baseline,
    'metrics_aug_imdb_tuned':     metrics_aug_imdb_tuned,
    'metrics_aug_wom_tuned':      metrics_aug_wom_tuned,
}
with open(out_path, 'w') as f:
    json.dump(results_to_save, f, indent=2, default=str)

print(f'\nSaved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v2/results_random_forest.json
```

04

NOTEBOOK

Gradient Boosting regression

04_gradient_boosting.pdf

Gradient Boosting

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)

from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')         # adjust if need
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v2'
RUN_DIR.mkdir(parents=True, exist_ok=True)


for f in ['movie_features_v6.csv',
          'movie_features_v6_synthetic.csv',
          'scene_movie_metadata_v6.csv',
          'scene_movie_metadata_v6_synthetic.csv']:
    assert (DATA_DIR / f).exists(), f'Missing: {f}'

print('Data dir:', DATA_DIR)
print('Run dir: ', RUN_DIR)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, cal
l drive.mount("/content/drive", force_remount=True).
Data dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6
Run dir:  /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6/colab_runs_v2
```

In [14]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, StratifiedKFold, GridSearchCV, cross_val_predict,
)
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
)
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
pd.set_option('display.width', 180)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

Data loading

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')

LEAKAGE_AND_OLD_TARGETS = [
    'budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_mov = real_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')


META_KEEP = [
    'movie_id',
    # Scene properties (will be encoded below)
    'targeted_emotion', 'clip_duration_s',
    'cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
    'silence_ratio', 'music_presence', 'dialogue_density',
    'face_screen_time_ratio', 'lead_screen_time_ratio',
    # Movie metadata
    'release_year', 'genre_primary', 'genre_secondary',
    'country_of_origin', 'budget_categorical',
    # Targets (kept to extract y; not used as features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]

real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left')
real_mov['is_synthetic'] = 0
syn_mov = syn_mov.merge(syn_meta_sub, on='movie_id', how='left')
syn_mov['is_synthetic'] = 1

df_all = pd.concat([real_mov, syn_mov], ignore_index=True)
print(f'Real:      {len(real_mov)} movies')
print(f'Synthetic: {len(syn_mov)} movies')
print(f'Combined:  {len(df_all)} movies × {len(df_all.columns)} columns (pre
print()
print('Target distributions (pre-encoding):')
display(df_all[['is_synthetic', 'imdb_rating', 'wom_multiplier_log']]
        .groupby('is_synthetic').describe().round(2))
```

```
Real:      10 movies
Synthetic: 40 movies
Combined:  50 movies × 343 columns (pre-encoding)

Target distributions (pre-encoding):
```

	imdb_rating											
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min
is_synthetic												
0	10.0	7.41	0.73	6.5	6.8	7.45	8.02	8.5	10.0	1.77	0.64	0.75
1	40.0	7.48	0.55	6.5	7.0	7.50	7.82	8.4	40.0	1.70	0.54	0.34

3 · Feature preparation

In [ ]:

```
DROP = {
    'movie_id', 'condition', 'n_participants',
    # Targets (cannot be features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
    # Control flag (not a feature)
    'is_synthetic',
}

df_feat = df_all.copy()

# Ordinal encoding: low / moderate / high → 1 / 2 / 3
ORD_MAP = {'low': 1, 'moderate': 2, 'high': 3}
ORD_COLS = ['cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
            'silence_ratio', 'music_presence', 'dialogue_density',
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'budget_categorical']
for c in ORD_COLS:
    if c in df_feat.columns:
        df_feat[c] = df_feat[c].map(ORD_MAP)

# One-hot encode multi-category text columns
OH_COLS = ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country_
OH_COLS = [c for c in OH_COLS if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH_COLS, prefix_sep='_',
                         dummy_na=False, dtype=int)

# Build feature matrix
feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
groups_all = df_feat['movie_id'].values if 'movie_id' in df_feat.columns els
synth_mask_all = df_feat['is_synthetic'].values.astype(bool)

# Drop all-NaN columns
all_nan_cols = X_all.columns[X_all.isna().all()].tolist()
if all_nan_cols:
    print(f'Dropping {len(all_nan_cols)} all-NaN columns')
    X_all = X_all.drop(columns=all_nan_cols)

# Drop zero-variance columns
zero_var_cols = X_all.columns[X_all.std() == 0].tolist()
```

```
if zero_var_cols:
    print(f'Dropping {len(zero_var_cols)} zero-variance columns')
    X_all = X_all.drop(columns=zero_var_cols)

feature_cols = list(X_all.columns)

#Categorise final features for transparency
phys_cols = [c for c in feature_cols if c.endswith('__mean') or c.endswith('
ord_in_X  = [c for c in feature_cols if c in ORD_COLS]
oh_in_X   = [c for c in feature_cols
             if any(c.startswith(p + '_') for p in
                    ['targeted_emotion', 'genre_primary', 'genre_secondary',
                     'country_of_origin'])]
other_cols = [c for c in feature_cols
              if c not in phys_cols + ord_in_X + oh_in_X]

print(f'\nFinal feature matrix: {X_all.shape}')
print(f'  Physiological (mean/std):  {len(phys_cols):>4d}')
print(f'  Scene/movie ordinal:       {len(ord_in_X):>4d}  ({ord_in_X})')
print(f'  Scene/movie one-hot:       {len(oh_in_X):>4d}')
print(f'  Other numeric (year, dur): {len(other_cols):>4d}  ({other_cols})')
print(f'  TOTAL features:            {len(feature_cols):>4d}')
```

```
Final feature matrix: (50, 360)
  Physiological (mean/std):   320
  Scene/movie ordinal:         10  (['cut_count', 'brightness', 'motion_inte
nsity', 'audio_loudness', 'silence_ratio', 'music_presence', 'dialogue_densi
ty', 'face_screen_time_ratio', 'lead_screen_time_ratio', 'budget_categorica
l'])
  Scene/movie one-hot:         28
  Other numeric (year, dur):    2  (['clip_duration_s', 'release_year'])
  TOTAL features:             360
```

Evaluation helpers

In [ ]:

```
def regression_metrics(y_true, y_pred):
    """Compute R², MAE, RMSE, Spearman, Pearson."""
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    if len(yt) < 3:
        return {'n': len(yt), 'r2': np.nan, 'mae': np.nan, 'rmse': np.nan,
                'spearman': np.nan, 'pearson': np.nan}
    return {
        'n': len(yt),
        'r2': r2_score(yt, yp),
        'mae': mean_absolute_error(yt, yp),
        'rmse': np.sqrt(mean_squared_error(yt, yp)),
        'spearman': spearmanr(yt, yp).correlation,
        'pearson': pearsonr(yt, yp)[0],
    }


def loo_predict(estimator, X, y):
    """Leave-one-out cross-validated predictions."""
```

```
    loo = LeaveOneOut()
    return cross_val_predict(estimator, X, y, cv=loo, n_jobs=-1)


def kfold_predict(estimator, X, y, n_splits=5, random_state=42):
    """K-fold CV predictions (k=5 default)."""
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    return cross_val_predict(estimator, X, y, cv=kf, n_jobs=-1)


def report(metrics, title=''):
    print(f'\n——— {title} ———')
    for k, v in metrics.items():
        if k == 'n':
            print(f'  {k:>10s}: {v}')
        else:
            print(f'  {k:>10s}: {v:.3f}' if not np.isnan(v) else f'  {k:>10s
```

```
def plot_pred_vs_actual(y_true, y_pred, title, ax=None):
    if ax is None:
        fig, ax = plt.subplots(figsize=(5, 5))
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    ax.scatter(yt, yp, alpha=0.6, s=40, edgecolor='k', linewidth=0.5)
    lo, hi = min(yt.min(), yp.min()), max(yt.max(), yp.max())
    ax.plot([lo, hi], [lo, hi], 'r--', alpha=0.5, label='y = x')
    r2 = r2_score(yt, yp)
    rho = spearmanr(yt, yp).correlation
    ax.set_xlabel('Actual')
    ax.set_ylabel('Predicted')
    ax.set_title(f'{title}\nR² = {r2:.2f}, Spearman ρ = {rho:.2f}')
    ax.legend()
    ax.grid(alpha=0.3)
    return ax
```

Real-only (LOO CV, n = 10)

In [ ]:

```
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.inspection import permutation_importance

mask_real = ~synth_mask_all
X_real = X_all[mask_real].reset_index(drop=True)
y_imdb_real = y_imdb_all[mask_real].reset_index(drop=True)
y_wom_real = y_wom_all[mask_real].reset_index(drop=True)

def make_gbm_pipeline(learning_rate=0.1, max_iter=100, max_depth=None,
                      min_samples_leaf=20, l2_regularization=0.0):

    return Pipeline([
        ('model', HistGradientBoostingRegressor(
            learning_rate=learning_rate, max_iter=max_iter,
            max_depth=max_depth, min_samples_leaf=min_samples_leaf,
            l2_regularization=l2_regularization, random_state=42)),
```

```
    ])

# Baseline
pipe = make_gbm_pipeline()
y_pred = loo_predict(pipe, X_real, y_imdb_real)
metrics_real_imdb_baseline = regression_metrics(y_imdb_real, y_pred)
report(metrics_real_imdb_baseline, 'Real / IMDb / baseline')

pipe = make_gbm_pipeline()
y_pred = loo_predict(pipe, X_real, y_wom_real)
metrics_real_wom_baseline = regression_metrics(y_wom_real, y_pred)
report(metrics_real_wom_baseline, 'Real / WOM-log / baseline')
```

```
──── Real / IMDb / baseline ────
         n: 10
        r2: -0.235
       mae: 0.678
      rmse: 0.774
  spearman: -1.000
   pearson: -1.000

──── Real / WOM-log / baseline ────
         n: 10
        r2: -0.235
       mae: 0.573
      rmse: 0.671
  spearman: -1.000
   pearson: -1.000
```

Tuned

In [ ]:

```
# REAL-ONLY: heavily constrained grid for n=10. Previous version
# overfit; we now restrict capacity strongly (max_depth less than 3,
# few iterations, higher l2_regularization).
param_grid = {
    'model__learning_rate':     [0.005, 0.01, 0.05],
    'model__max_iter':          [30, 50, 100],
    'model__max_depth':         [1, 2, 3],
    'model__min_samples_leaf':  [2, 4],
    'model__l2_regularization': [0.0, 1.0, 10.0],
}

search = GridSearchCV(make_gbm_pipeline(), param_grid,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
                      n_jobs=-1, refit=True)
search.fit(X_real, y_imdb_real)
best_params_imdb_real = search.best_params_
y_pred_real_imdb_tuned = loo_predict(search.best_estimator_, X_real, y_imdb_
metrics_real_imdb_tuned = regression_metrics(y_imdb_real, y_pred_real_imdb_t
print(f'\nBest IMDb params: {best_params_imdb_real}')
report(metrics_real_imdb_tuned, 'Real / IMDb / tuned')

search = GridSearchCV(make_gbm_pipeline(), param_grid,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
```

```
                      n_jobs=-1, refit=True)
search.fit(X_real, y_wom_real)
best_params_wom_real = search.best_params_
y_pred_real_wom_tuned = loo_predict(search.best_estimator_, X_real, y_wom_re
metrics_real_wom_tuned = regression_metrics(y_wom_real, y_pred_real_wom_tune
print(f'\nBest WOM params: {best_params_wom_real}')
report(metrics_real_wom_tuned, 'Real / WOM-log / tuned')
```

```
Best IMDb params: {'model__l2_regularization': 10.0, 'model__learning_rate':
0.005, 'model__max_depth': 2, 'model__max_iter': 30, 'model__min_samples_lea
f': 2}

——— Real / IMDb / tuned ———
           n: 10
          r2: -0.225
         mae: 0.679
        rmse: 0.771
    spearman: -0.896
     pearson: -0.948

Best WOM params: {'model__l2_regularization': 10.0, 'model__learning_rate':
0.05, 'model__max_depth': 1, 'model__max_iter': 100, 'model__min_samples_lea
f': 4}

——— Real / WOM-log / tuned ———
           n: 10
          r2: 0.245
         mae: 0.446
        rmse: 0.525
    spearman: 0.552
     pearson: 0.510
```

Augmented (5-fold CV, n = 50)

In [20]:

```
# More reasonable param ranges for n=50.
param_grid_aug = {
    'model__learning_rate':     [0.01, 0.05, 0.1],
    'model__max_iter':          [100, 200, 400],
    'model__max_depth':         [3, 5, 7, None],
    'model__min_samples_leaf':  [4, 10, 20],
    'model__l2_regularization': [0.0, 0.1, 1.0],
}

# Baseline
pipe = make_gbm_pipeline()
y_pred_aug_imdb = kfold_predict(pipe, X_all, y_imdb_all, n_splits=5)
metrics_aug_imdb_baseline = regression_metrics(y_imdb_all, y_pred_aug_imdb)
report(metrics_aug_imdb_baseline, 'Augmented / IMDb / baseline')

pipe = make_gbm_pipeline()
y_pred_aug_wom = kfold_predict(pipe, X_all, y_wom_all, n_splits=5)
metrics_aug_wom_baseline = regression_metrics(y_wom_all, y_pred_aug_wom)
report(metrics_aug_wom_baseline, 'Augmented / WOM-log / baseline')
```

```
──── Augmented / IMDb / baseline ────
           n: 50
          r2: 0.278
         mae: 0.362
        rmse: 0.488
    spearman: 0.542
     pearson: 0.540

──── Augmented / WOM-log / baseline ────
           n: 50
          r2: -0.032
         mae: 0.445
        rmse: 0.557
    spearman: 0.186
     pearson: 0.216
```

In [21]:

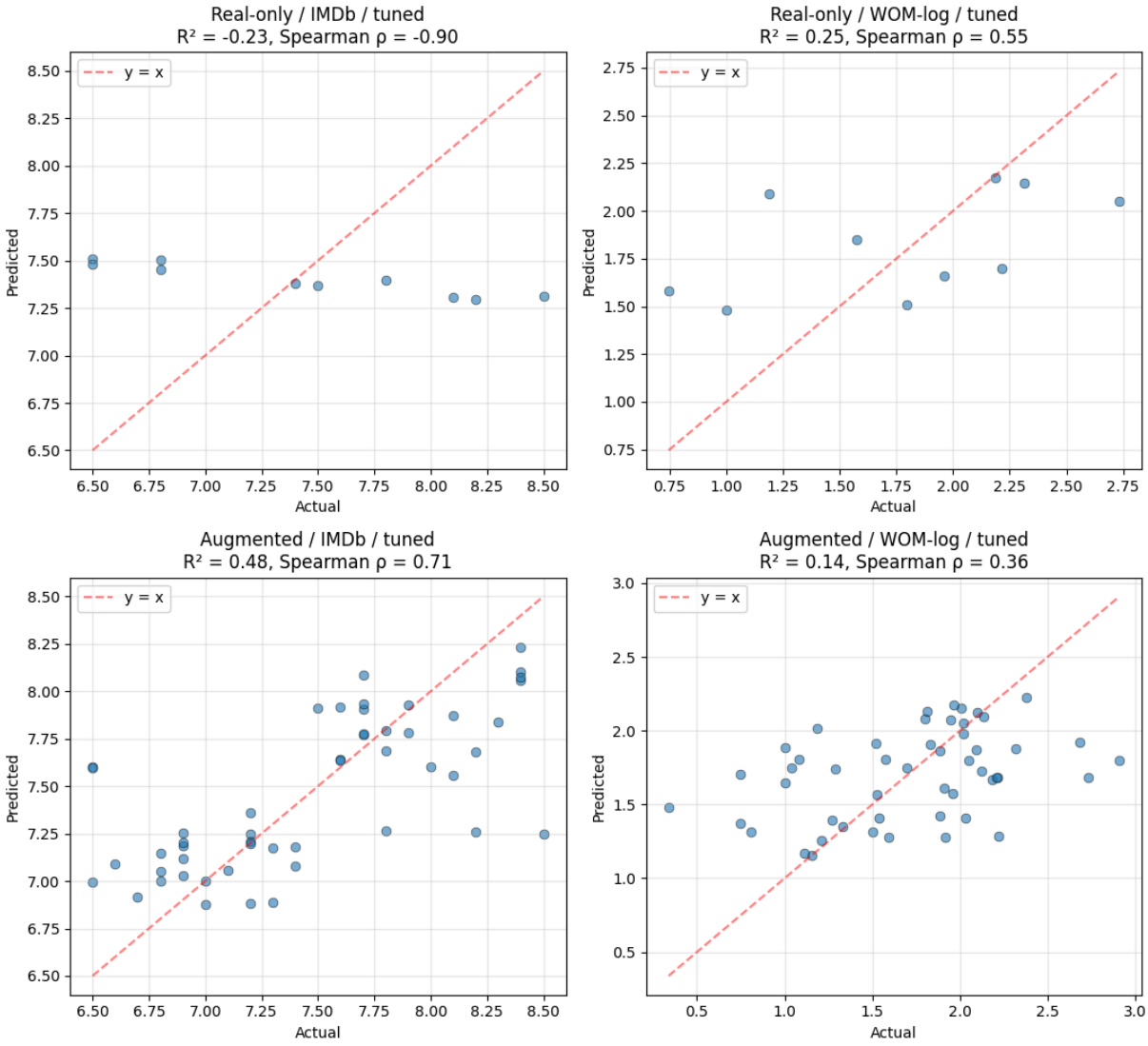
```
# AUGMENTED — wider grid, includes max_leaf_nodes
param_grid_aug = {
    'model__learning_rate':     [0.01, 0.05, 0.1],
    'model__max_iter':          [50, 100, 200, 400, 800],
    'model__max_depth':         [3, 5, 7, None],
    'model__min_samples_leaf':  [4, 10, 20],
    'model__l2_regularization': [0.0, 0.1, 1.0, 10.0],
    'model__max_leaf_nodes':    [15, 31, 63],
}

search = GridSearchCV(make_gbm_pipeline(), param_grid_aug,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_imdb_all)
best_params_imdb = search.best_params_
y_pred_aug_imdb_tuned = kfold_predict(search.best_estimator_, X_all, y_imdb_
metrics_aug_imdb_tuned = regression_metrics(y_imdb_all, y_pred_aug_imdb_tune
print(f'\nBest IMDb params: {best_params_imdb}')
report(metrics_aug_imdb_tuned, 'Augmented / IMDb / tuned')

search = GridSearchCV(make_gbm_pipeline(), param_grid_aug,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_wom_all)
best_params_wom = search.best_params_
y_pred_aug_wom_tuned = kfold_predict(search.best_estimator_, X_all, y_wom_al
metrics_aug_wom_tuned = regression_metrics(y_wom_all, y_pred_aug_wom_tuned)
print(f'\nBest WOM params: {best_params_wom}')
report(metrics_aug_wom_tuned, 'Augmented / WOM-log / tuned')
```

```
Best IMDb params: {'model__l2_regularization': 10.0, 'model__learning_rate':
0.05, 'model__max_depth': 3, 'model__max_iter': 100, 'model__max_leaf_node
s': 15, 'model__min_samples_leaf': 4}

──── Augmented / IMDb / tuned ────
           n: 50
          r2: 0.476
         mae: 0.307
        rmse: 0.416
    spearman: 0.712
     pearson: 0.690

Best WOM params: {'model__l2_regularization': 1.0, 'model__learning_rate':
0.1, 'model__max_depth': 3, 'model__max_iter': 800, 'model__max_leaf_nodes':
15, 'model__min_samples_leaf': 10}

──── Augmented / WOM-log / tuned ────
           n: 50
          r2: 0.136
         mae: 0.394
        rmse: 0.510
    spearman: 0.364
     pearson: 0.396
```

Visuals

In [22]:

```python
fig, axes = plt.subplots(2, 2, figsize=(11, 10))
plot_pred_vs_actual(y_imdb_real, y_pred_real_imdb_tuned,
                    'Real-only / IMDb / tuned', axes[0, 0])
plot_pred_vs_actual(y_wom_real, y_pred_real_wom_tuned,
                    'Real-only / WOM-log / tuned', axes[0, 1])
plot_pred_vs_actual(y_imdb_all, y_pred_aug_imdb_tuned,
                    'Augmented / IMDb / tuned', axes[1, 0])
plot_pred_vs_actual(y_wom_all, y_pred_aug_wom_tuned,
                    'Augmented / WOM-log / tuned', axes[1, 1])
plt.tight_layout()
plt.show()
```



Permutation importance

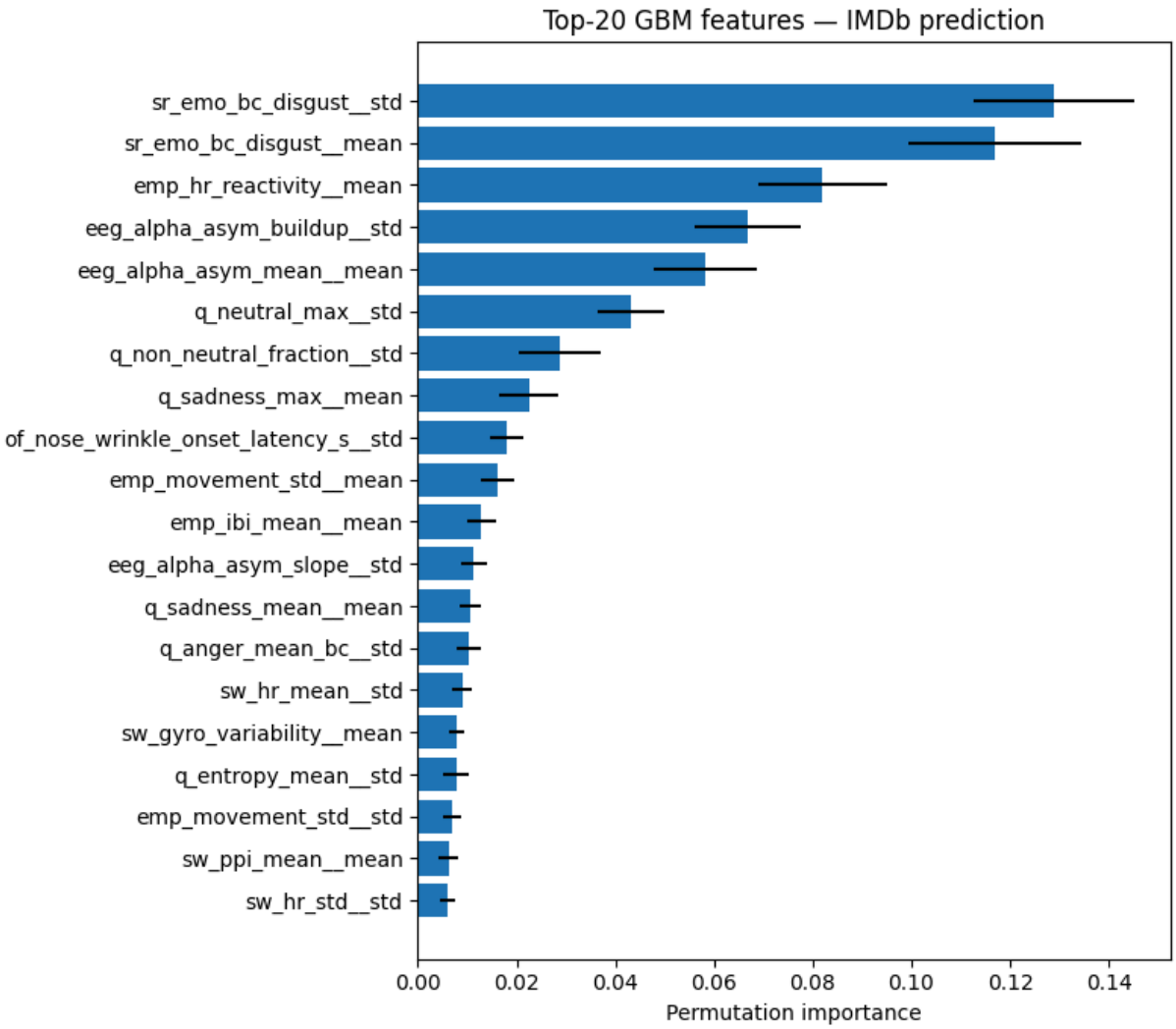
In [24]:

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.inspection import permutation_importance

def make_gbm_pipeline(learning_rate=0.1, max_iter=100, max_depth=None,
                      min_samples_leaf=20, l2_regularization=0.0, max_leaf_r
    # HistGradientBoosting handles NaN natively, so no imputer needed.
    return Pipeline([
        ('model', HistGradientBoostingRegressor(
            learning_rate=learning_rate, max_iter=max_iter,
            max_depth=max_depth, min_samples_leaf=min_samples_leaf,
            l2_regularization=l2_regularization, max_leaf_nodes=max_leaf_noc
            random_state=42)),
    ])

final_pipe = make_gbm_pipeline(**{k.replace('model__', ''): v for k, v in be
final_pipe.fit(X_all, y_imdb_all)
```

```
perm = permutation_importance(
    final_pipe, X_all, y_imdb_all,
    n_repeats=20, random_state=42, n_jobs=-1, scoring='neg_mean_absolute_err
)
imp_df = (pd.DataFrame({
    'feature': feature_cols,
    'imp_mean': perm.importances_mean,
    'imp_std':  perm.importances_std,
}).sort_values('imp_mean', ascending=False))
print('Top 30 features by permutation importance:')
display(imp_df.head(30).round(4))

top = imp_df.head(20).iloc[::-1]
fig, ax = plt.subplots(figsize=(8, 7))
ax.barh(top['feature'], top['imp_mean'], xerr=top['imp_std'])
ax.set_xlabel('Permutation importance')
ax.set_title('Top-20 GBM features — IMDb prediction')
plt.tight_layout()
plt.show()
```

```
Top 30 features by permutation importance:
```

	feature	imp_mean	imp_std
257	sr_emo_bc_disgust__std	0.1290	0.0163
256	sr_emo_bc_disgust__mean	0.1170	0.0175
60	emp_hr_reactivity__mean	0.0820	0.0130
9	eeg_alpha_asym_buildup__std	0.0669	0.0107
12	eeg_alpha_asym_mean__mean	0.0583	0.0104
221	q_neutral_max__std	0.0432	0.0067
227	q_non_neutral_fraction__std	0.0286	0.0083
230	q_sadness_max__mean	0.0226	0.0060
161	of_nose_wrinkle_onset_latency_s__std	0.0179	0.0034
70	emp_movement_std__mean	0.0162	0.0034
62	emp_ibi_mean__mean	0.0128	0.0030
15	eeg_alpha_asym_slope__std	0.0113	0.0026
232	q_sadness_mean__mean	0.0106	0.0022
191	q_anger_mean_bc__std	0.0103	0.0025
289	sw_hr_mean__std	0.0090	0.0020
286	sw_gyro_variability__mean	0.0078	0.0015
199	q_entropy_mean__std	0.0077	0.0026
71	emp_movement_std__std	0.0070	0.0019
308	sw_ppi_mean__mean	0.0062	0.0019
299	sw_hr_std__std	0.0061	0.0015
29	eeg_beta_asym_change__std	0.0059	0.0013
55	emp_hr_mean__std	0.0057	0.0021
285	sr_sam_valence__std	0.0056	0.0017
147	of_lip_depress_max__std	0.0055	0.0014
209	q_happiness_onset_s__std	0.0054	0.0015
162	of_smile_buildup__mean	0.0043	0.0008
251	sr_emo_bc_amusement__std	0.0042	0.0011
207	q_happiness_mean_bc__std	0.0040	0.0009
137	of_duchenne_fraction__std	0.0034	0.0011
130	of_brow_furrow_mean_bc__mean	0.0031	0.0008



In [ ]:
```python
# Save results
out_path = RUN_DIR / 'results_gradient_boosting.json'
results_to_save = {
    'model': 'gradient_boosting',
    'feature_count': len(feature_cols),
    'best_params_imdb': best_params_imdb,
    'best_params_wom':  best_params_wom,
    'metrics_real_imdb_baseline': metrics_real_imdb_baseline,
    'metrics_real_wom_baseline':  metrics_real_wom_baseline,
    'metrics_real_imdb_tuned':    metrics_real_imdb_tuned,
    'metrics_real_wom_tuned':     metrics_real_wom_tuned,
    'metrics_aug_imdb_baseline':  metrics_aug_imdb_baseline,
    'metrics_aug_wom_baseline':   metrics_aug_wom_baseline,
    'metrics_aug_imdb_tuned':     metrics_aug_imdb_tuned,
    'metrics_aug_wom_tuned':      metrics_aug_wom_tuned,
}
with open(out_path, 'w') as f:
    json.dump(results_to_save, f, indent=2, default=str)

print(f'\nSaved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v2/results_gradient_boosting.json
```

05

NOTEBOOK

Partial Least Squares regression

05_pls.pdf

Partial Least Squares Regression (PLS)

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)

from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v2'
RUN_DIR.mkdir(parents=True, exist_ok=True)

for f in ['movie_features_v6.csv', 'movie_features_v6_synthetic.csv',
          'scene_movie_metadata_v6.csv', 'scene_movie_metadata_v6_synthetic.
    assert (DATA_DIR / f).exists(), f'Missing: {f}'
print('Data dir:', DATA_DIR)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, cal
l drive.mount("/content/drive", force_remount=True).
Data dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6
```

In [15]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, GridSearchCV, cross_val_predict,
)
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
)
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
pd.set_option('display.width', 180)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

Data loading

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
```

```python
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')


LEAKAGE_AND_OLD_TARGETS = [
    'budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_mov = real_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')

# Columns from metadata to use as features (excluding IDs, text, leakage)
META_KEEP = [
    'movie_id',
    # Scene properties (will be encoded below)
    'targeted_emotion', 'clip_duration_s',
    'cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
    'silence_ratio', 'music_presence', 'dialogue_density',
    'face_screen_time_ratio', 'lead_screen_time_ratio',
    # Movie metadata
    'release_year', 'genre_primary', 'genre_secondary',
    'country_of_origin', 'budget_categorical',
    # Targets (kept to extract y; not used as features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]

real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left')
real_mov['is_synthetic'] = 0
syn_mov = syn_mov.merge(syn_meta_sub, on='movie_id', how='left')
syn_mov['is_synthetic'] = 1

df_all = pd.concat([real_mov, syn_mov], ignore_index=True)
print(f'Real:      {len(real_mov)} movies')
print(f'Synthetic: {len(syn_mov)} movies')
print(f'Combined:  {len(df_all)} movies × {len(df_all.columns)} columns (pre
print()
print('Target distributions (pre-encoding):')
display(df_all[['is_synthetic', 'imdb_rating', 'wom_multiplier_log']]
        .groupby('is_synthetic').describe().round(2))
```

```
Real:      10 movies
Synthetic: 40 movies
Combined:  50 movies × 343 columns (pre-encoding)

Target distributions (pre-encoding):
```

	imdb_rating											
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min
is_synthetic												
0	10.0	7.41	0.73	6.5	6.8	7.45	8.02	8.5	10.0	1.77	0.64	0.75
1	40.0	7.48	0.55	6.5	7.0	7.50	7.82	8.4	40.0	1.70	0.54	0.34

3 · Feature preparation

In [ ]:

```
DROP = {
    'movie_id', 'condition', 'n_participants',
    # Targets (cannot be features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
    # Control flag (not a feature)
    'is_synthetic',
}

df_feat = df_all.copy()

# Ordinal encoding: low / moderate / high → 1 / 2 / 3
ORD_MAP = {'low': 1, 'moderate': 2, 'high': 3}
ORD_COLS = ['cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
            'silence_ratio', 'music_presence', 'dialogue_density',
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'budget_categorical']
for c in ORD_COLS:
    if c in df_feat.columns:
        df_feat[c] = df_feat[c].map(ORD_MAP)

# One-hot encode multi-category text columns
OH_COLS = ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country_
OH_COLS = [c for c in OH_COLS if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH_COLS, prefix_sep='_',
                         dummy_na=False, dtype=int)

# Build feature matrix
feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
groups_all = df_feat['movie_id'].values if 'movie_id' in df_feat.columns els
synth_mask_all = df_feat['is_synthetic'].values.astype(bool)

# Drop all-NaN columns
all_nan_cols = X_all.columns[X_all.isna().all()].tolist()
if all_nan_cols:
    print(f'Dropping {len(all_nan_cols)} all-NaN columns')
    X_all = X_all.drop(columns=all_nan_cols)

# Drop zero-variance columns
zero_var_cols = X_all.columns[X_all.std() == 0].tolist()
```

```
if zero_var_cols:
    print(f'Dropping {len(zero_var_cols)} zero-variance columns')
    X_all = X_all.drop(columns=zero_var_cols)

feature_cols = list(X_all.columns)

# Categorise final features for transparency
phys_cols = [c for c in feature_cols if c.endswith('__mean') or c.endswith('
ord_in_X  = [c for c in feature_cols if c in ORD_COLS]
oh_in_X   = [c for c in feature_cols
             if any(c.startswith(p + '_') for p in
                    ['targeted_emotion', 'genre_primary', 'genre_secondary',
                     'country_of_origin'])]
other_cols = [c for c in feature_cols
              if c not in phys_cols + ord_in_X + oh_in_X]

print(f'\nFinal feature matrix: {X_all.shape}')
print(f'  Physiological (mean/std):  {len(phys_cols):>4d}')
print(f'  Scene/movie ordinal:       {len(ord_in_X):>4d}  ({ord_in_X})')
print(f'  Scene/movie one-hot:       {len(oh_in_X):>4d}')
print(f'  Other numeric (year, dur): {len(other_cols):>4d}  ({other_cols})')
print(f'  TOTAL features:            {len(feature_cols):>4d}')
```

```
Final feature matrix: (50, 360)
  Physiological (mean/std):   320
  Scene/movie ordinal:         10  (['cut_count', 'brightness', 'motion_inte
nsity', 'audio_loudness', 'silence_ratio', 'music_presence', 'dialogue_densi
ty', 'face_screen_time_ratio', 'lead_screen_time_ratio', 'budget_categorica
l'])
  Scene/movie one-hot:         28
  Other numeric (year, dur):    2  (['clip_duration_s', 'release_year'])
  TOTAL features:             360
```

Evaluation helpers

In [18]:

```
def regression_metrics(y_true, y_pred):
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    if len(yt) < 3:
        return {'n': len(yt), 'r2': np.nan, 'mae': np.nan, 'rmse': np.nan,
                'spearman': np.nan, 'pearson': np.nan}
    return {
        'n': len(yt),
        'r2': r2_score(yt, yp),
        'mae': mean_absolute_error(yt, yp),
        'rmse': np.sqrt(mean_squared_error(yt, yp)),
        'spearman': spearmanr(yt, yp).correlation,
        'pearson': pearsonr(yt, yp)[0],
    }

def loo_predict(estimator, X, y):
    return cross_val_predict(estimator, X, y, cv=LeaveOneOut(), n_jobs=-1)

def kfold_predict(estimator, X, y, n_splits=5, random_state=42):
```

```
    return cross_val_predict(estimator, X, y,
                             cv=KFold(n_splits=n_splits, shuffle=True,
                                      random_state=random_state),
                             n_jobs=-1)

def report(metrics, title=''):
    print(f'\n——— {title} ———')
    for k, v in metrics.items():
        if k == 'n': print(f'  {k:>10s}: {v}')
        else: print(f'  {k:>10s}: {v:.3f}' if not np.isnan(v) else f'  {k:>1

def plot_pred_vs_actual(y_true, y_pred, title, ax=None, y_std=None):
    if ax is None: fig, ax = plt.subplots(figsize=(5, 5))
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    if y_std is not None:
        ys = np.asarray(y_std)[mask]
        ax.errorbar(yt, yp, yerr=ys, fmt='o', alpha=0.6,
                    capsize=2, ecolor='gray', markersize=6)
    else:
        ax.scatter(yt, yp, alpha=0.6, s=40, edgecolor='k', linewidth=0.5)
    lo, hi = min(yt.min(), yp.min()), max(yt.max(), yp.max())
    ax.plot([lo, hi], [lo, hi], 'r--', alpha=0.5, label='y = x')
    r2 = r2_score(yt, yp)
    rho = spearmanr(yt, yp).correlation
    ax.set_xlabel('Actual'); ax.set_ylabel('Predicted')
    ax.set_title(f'{title}\nR²={r2:.2f}, Spearman ρ={rho:.2f}')
    ax.legend(); ax.grid(alpha=0.3)
    return ax
```

Real-only (LOO CV, n = 10)

In [ ]:

```
from sklearn.cross_decomposition import PLSRegression

mask_real = ~synth_mask_all
X_real = X_all[mask_real].reset_index(drop=True)
y_imdb_real = y_imdb_all[mask_real].reset_index(drop=True)
y_wom_real = y_wom_all[mask_real].reset_index(drop=True)

def make_pls_pipeline(n_components=2):
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('model', PLSRegression(n_components=n_components, scale=False)),
    ])

# Baseline: 2 components
pipe = make_pls_pipeline(n_components=2)
y_pred = loo_predict(pipe, X_real, y_imdb_real)
metrics_real_imdb_baseline = regression_metrics(y_imdb_real, y_pred)
report(metrics_real_imdb_baseline, 'Real / IMDb / baseline (n_comp=2)')

pipe = make_pls_pipeline(n_components=2)
y_pred = loo_predict(pipe, X_real, y_wom_real)
```

```
metrics_real_wom_baseline = regression_metrics(y_wom_real, y_pred)
report(metrics_real_wom_baseline, 'Real / WOM / baseline (n_comp=2)')
```

```
──── Real / IMDb / baseline (n_comp=2) ────
           n: 10
          r2: -1.045
         mae: 0.914
        rmse: 0.996
    spearman: -0.701
     pearson: -0.683

──── Real / WOM / baseline (n_comp=2) ────
           n: 10
          r2: -0.846
         mae: 0.740
        rmse: 0.821
    spearman: -0.770
     pearson: -0.710
```

5b · Tuning n_components (LOO CV)

In [ ]:

```
# With n=10 training, n_components must be less than n_train − 1 = 9.
# Constrain to small values to avoid overfitting.
param_grid_real = {'model__n_components': [1, 2, 3, 4, 5]}

search = GridSearchCV(make_pls_pipeline(), param_grid_real,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
                      n_jobs=-1, refit=True)
search.fit(X_real, y_imdb_real)
best_params_imdb_real = search.best_params_
y_pred_real_imdb_tuned = loo_predict(search.best_estimator_, X_real, y_imdb_
metrics_real_imdb_tuned = regression_metrics(y_imdb_real, y_pred_real_imdb_t
print(f'\nBest IMDb params: {best_params_imdb_real}')
report(metrics_real_imdb_tuned, 'Real / IMDb / tuned')

search = GridSearchCV(make_pls_pipeline(), param_grid_real,
                      cv=LeaveOneOut(), scoring='neg_mean_absolute_error',
                      n_jobs=-1, refit=True)
search.fit(X_real, y_wom_real)
best_params_wom_real = search.best_params_
y_pred_real_wom_tuned = loo_predict(search.best_estimator_, X_real, y_wom_re
metrics_real_wom_tuned = regression_metrics(y_wom_real, y_pred_real_wom_tune
print(f'\nBest WOM params: {best_params_wom_real}')
report(metrics_real_wom_tuned, 'Real / WOM / tuned')
```

```
Best IMDb params: {'model__n_components': 5}

——— Real / IMDb / tuned ———
           n: 10
          r2: -0.975
         mae: 0.898
        rmse: 0.979
    spearman: -0.713
     pearson: -0.677

Best WOM params: {'model__n_components': 1}

——— Real / WOM / tuned ———
           n: 10
          r2: -0.866
         mae: 0.711
        rmse: 0.825
    spearman: -0.782
     pearson: -0.760
```

Augmented (5-fold CV, n = 50)

In [21]:

```python
# Augmented — wider n_components search (1..20)
param_grid_aug = {'model__n_components': list(range(1, 21))}

search = GridSearchCV(make_pls_pipeline(), param_grid_aug,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_imdb_all)
best_params_imdb = search.best_params_
y_pred_aug_imdb_tuned = kfold_predict(search.best_estimator_, X_all, y_imdb_
metrics_aug_imdb_tuned = regression_metrics(y_imdb_all, y_pred_aug_imdb_tune
print(f'\nBest IMDb params: {best_params_imdb}')
report(metrics_aug_imdb_tuned, 'Augmented / IMDb / tuned')

search = GridSearchCV(make_pls_pipeline(), param_grid_aug,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_wom_all)
best_params_wom = search.best_params_
y_pred_aug_wom_tuned = kfold_predict(search.best_estimator_, X_all, y_wom_al
metrics_aug_wom_tuned = regression_metrics(y_wom_all, y_pred_aug_wom_tuned)
print(f'\nBest WOM params: {best_params_wom}')
report(metrics_aug_wom_tuned, 'Augmented / WOM / tuned')
```

```
Best IMDb params: {'model__n_components': 2}

──── Augmented / IMDb / tuned ────
           n: 50
          r2: 0.472
         mae: 0.309
        rmse: 0.418
    spearman: 0.680
     pearson: 0.689

Best WOM params: {'model__n_components': 1}

──── Augmented / WOM / tuned ────
           n: 50
          r2: 0.190
         mae: 0.402
        rmse: 0.494
    spearman: 0.385
     pearson: 0.461
```

In [22]:

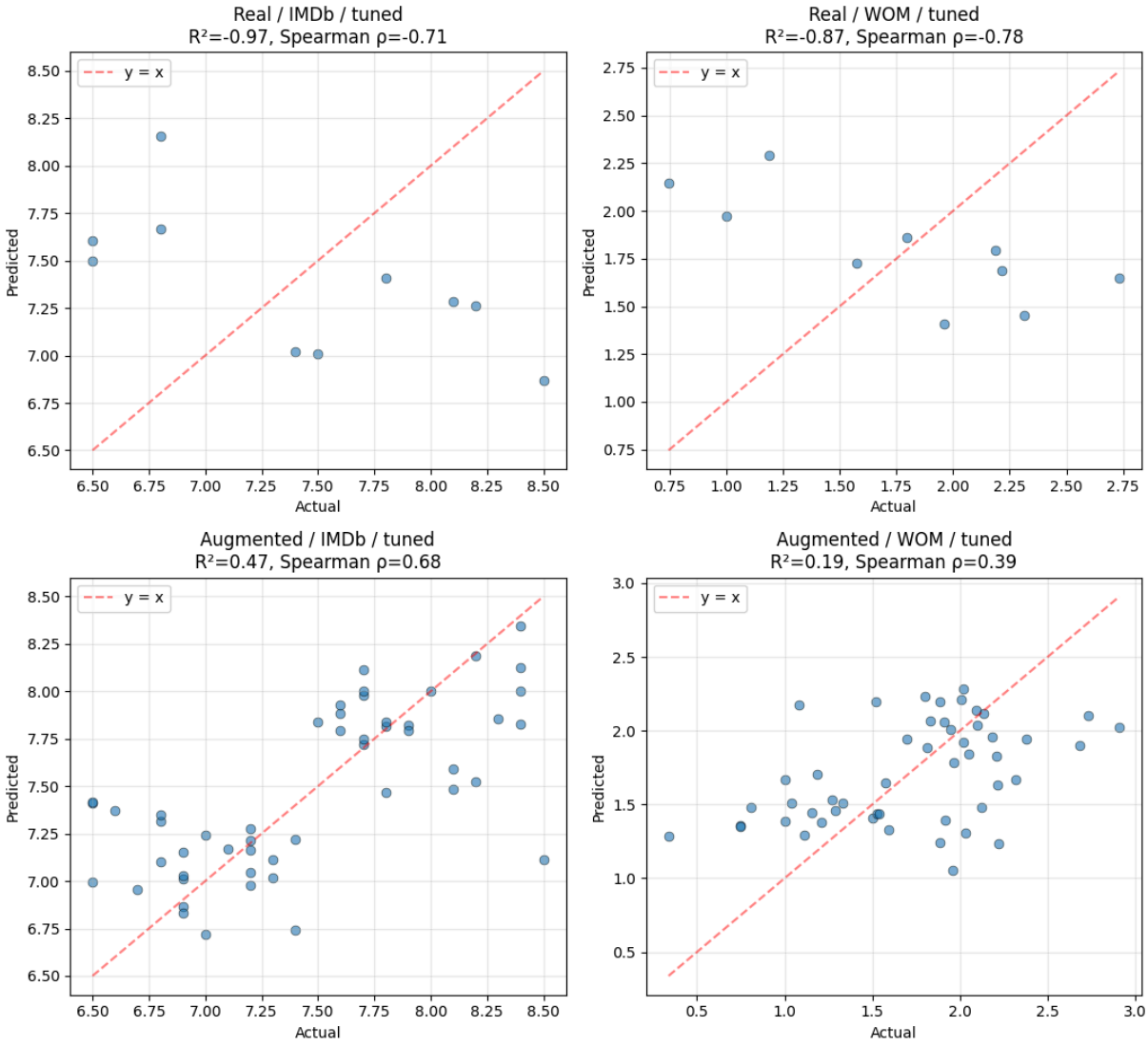
```python
# Tuned
search = GridSearchCV(make_pls_pipeline(), param_grid_aug,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_imdb_all)
best_params_imdb = search.best_params_
y_pred_aug_imdb_tuned = kfold_predict(search.best_estimator_, X_all, y_imdb_
metrics_aug_imdb_tuned = regression_metrics(y_imdb_all, y_pred_aug_imdb_tune
print(f'\nBest IMDb params: {best_params_imdb}')
report(metrics_aug_imdb_tuned, 'Augmented / IMDb / tuned')

search = GridSearchCV(make_pls_pipeline(), param_grid_aug,
                      cv=KFold(n_splits=5, shuffle=True, random_state=42),
                      scoring='neg_mean_absolute_error', n_jobs=-1, refit=Tru
search.fit(X_all, y_wom_all)
best_params_wom = search.best_params_
y_pred_aug_wom_tuned = kfold_predict(search.best_estimator_, X_all, y_wom_al
metrics_aug_wom_tuned = regression_metrics(y_wom_all, y_pred_aug_wom_tuned)
print(f'\nBest WOM params: {best_params_wom}')
report(metrics_aug_wom_tuned, 'Augmented / WOM / tuned')
```

```
Best IMDb params: {'model__n_components': 2}

──── Augmented / IMDb / tuned ────
           n: 50
          r2: 0.472
         mae: 0.309
        rmse: 0.418
    spearman: 0.680
     pearson: 0.689

Best WOM params: {'model__n_components': 1}

──── Augmented / WOM / tuned ────
           n: 50
          r2: 0.190
         mae: 0.402
        rmse: 0.494
    spearman: 0.385
     pearson: 0.461
```

Visuals

In [23]:

```python
fig, axes = plt.subplots(2, 2, figsize=(11, 10))
plot_pred_vs_actual(y_imdb_real, y_pred_real_imdb_tuned,
                    'Real / IMDb / tuned', axes[0, 0])
plot_pred_vs_actual(y_wom_real, y_pred_real_wom_tuned,
                    'Real / WOM / tuned', axes[0, 1])
plot_pred_vs_actual(y_imdb_all, y_pred_aug_imdb_tuned,
                    'Augmented / IMDb / tuned', axes[1, 0])
plot_pred_vs_actual(y_wom_all, y_pred_aug_wom_tuned,
                    'Augmented / WOM / tuned', axes[1, 1])
plt.tight_layout(); plt.show()
```



PLS components: variance

In [24]:
```
# Refit on full augmented data with best n_components for IMDb
n_comp = best_params_imdb['model__n_components']
final_pipe = make_pls_pipeline(n_components=n_comp)
final_pipe.fit(X_all, y_imdb_all)
pls = final_pipe.named_steps['model']

# Variance explained per component
# x_scores has shape (n_samples, n_comp). Total variance = sum of squares.
X_imp = final_pipe.named_steps['imputer'].transform(X_all)
X_sc  = final_pipe.named_steps['scaler'].transform(X_imp)
total_var_X = (X_sc ** 2).sum()
var_per_comp_X = ((pls.x_scores_) ** 2).sum(axis=0) / total_var_X * 100

total_var_Y = ((y_imdb_all - y_imdb_all.mean()) ** 2).sum()
var_per_comp_Y = ((pls.y_scores_) ** 2).sum(axis=0) / total_var_Y * 100

print(f'PLS components: {n_comp}')
print(f'\n  Component   Var(X) %   Var(Y) %   Cum Var(Y) %')
```

```
print('–' * 50)
cum = 0
for i in range(n_comp):
    cum += var_per_comp_Y[i]
    print(f'  PC{i+1:>3d}      {var_per_comp_X[i]:>7.2f}    '
          f'{var_per_comp_Y[i]:>7.2f}    {cum:>7.2f}')

# Top features per component (loadings)
loadings = pls.x_loadings_      # (n_features, n_comp)
print(f'\nTop 15 features for each component:')
for c in range(min(n_comp, 3)):  # Show first 3 components
    idx = np.argsort(np.abs(loadings[:, c]))[::-1][:15]
    print(f'\n  Component {c+1}:')
    for i in idx:
        print(f'    {feature_cols[i]:<40s}  loading = {loadings[i, c]:>+.4f}
```

```
PLS components: 2

  Component   Var(X) %   Var(Y) %   Cum Var(Y) %
──────────────────────────────────────────────────
  PC  1        10.80    16893.29    16893.29
  PC  2         3.41     5453.91    22347.20

Top 15 features for each component:

  Component 1:
    sr_emo_bc_disgust__std                    loading = +0.1398
    sr_emo_disgust__std                       loading = +0.1394
    emp_hr_reactivity__mean                   loading = -0.1375
    movie_familiarity__std                    loading = +0.1288
    sr_emo_disgust__mean                      loading = +0.1257
    sr_emo_bc_disgust__mean                   loading = +0.1245
    sr_emo_anger__std                         loading = +0.1211
    movie_familiarity__mean                   loading = +0.1205
    sr_emo_amusement__std                     loading = +0.1204
    eeg_alpha_asym_buildup__std               loading = +0.1171
    q_disgust_max__mean                       loading = +0.1156
    q_head_yaw_std__mean                      loading = +0.1076
    sr_emo_amusement__mean                    loading = +0.1060
    q_happiness_max__mean                     loading = +0.1048
    eeg_alpha_af7_mean_bc__mean               loading = +0.1038

  Component 2:
    emp_movement_std__mean                    loading = +0.1693
    emp_rmssd__mean                           loading = +0.1550
    q_happiness_mean__mean                    loading = +0.1472
    emp_ibi_mean__mean                        loading = +0.1361
    emp_hr_mean__mean                         loading = -0.1346
    emp_movement_std__std                     loading = +0.1344
    sw_hr_mean__mean                          loading = -0.1331
    q_non_neutral_fraction__mean              loading = +0.1305
    sw_ppi_mean__mean                         loading = +0.1303
    q_neutral_mean_bc__mean                   loading = -0.1279
    emp_pnn50__mean                           loading = +0.1278
    sw_hr_mean__std                           loading = -0.1242
    of_smile_peak_timing__mean                loading = +0.1161
    q_entropy_mean__mean                      loading = +0.1160
    of_nose_wrinkle_max__mean                 loading = +0.1141
```

In [ ]:

```python
# Visualise loadings on the first 2 components
fig, ax = plt.subplots(figsize=(9, 8))
top_n = 25
top_idx = np.argsort(np.linalg.norm(loadings[:, :2], axis=1))[::-1][:top_n]
ax.scatter(loadings[top_idx, 0], loadings[top_idx, 1], alpha=0.7)
for i in top_idx:
    ax.annotate(feature_cols[i][:30], (loadings[i, 0], loadings[i, 1]),
                fontsize=7, alpha=0.7)
ax.axhline(0, color='gray', linewidth=0.5)
ax.axvline(0, color='gray', linewidth=0.5)
ax.set_xlabel(f'Loading on Component 1 ({var_per_comp_Y[0]:.1f}% Y variance)')
ax.set_ylabel(f'Loading on Component 2 ({var_per_comp_Y[1]:.1f}% Y variance)')
ax.set_title('PLS loadings — top 25 features (IMDb prediction)')
```

```
ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```

In [26]:

```
out_path = RUN_DIR / 'results_pls.json'
results_to_save = {
    'model': 'pls',
    'feature_count': len(feature_cols),
    'best_params_imdb': best_params_imdb,
    'best_params_wom':  best_params_wom,
    'metrics_real_imdb_baseline': metrics_real_imdb_baseline,
    'metrics_real_wom_baseline':  metrics_real_wom_baseline,
    'metrics_real_imdb_tuned':    metrics_real_imdb_tuned,
    'metrics_real_wom_tuned':     metrics_real_wom_tuned,
    'metrics_aug_imdb_baseline':  metrics_aug_imdb_baseline,
    'metrics_aug_wom_baseline':   metrics_aug_wom_baseline,
    'metrics_aug_imdb_tuned':     metrics_aug_imdb_tuned,
    'metrics_aug_wom_tuned':      metrics_aug_wom_tuned,
}
with open(out_path, 'w') as f:
    json.dump(results_to_save, f, indent=2, default=str)
print(f'Saved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v2/results_pls.json
```

06

NOTEBOOK

Gaussian Process regression

06_gaussian_process.pdf

Gaussian Process Regression (GPR)

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)

from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v2'
RUN_DIR.mkdir(parents=True, exist_ok=True)

for f in ['movie_features_v6.csv', 'movie_features_v6_synthetic.csv',
          'scene_movie_metadata_v6.csv', 'scene_movie_metadata_v6_synthetic.
    assert (DATA_DIR / f).exists(), f'Missing: {f}'
print('Data dir:', DATA_DIR)
```

```
Mounted at /content/drive
Data dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6
```

In [2]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, GridSearchCV, cross_val_predict,
)
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
)
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
pd.set_option('display.width', 180)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

Data loading

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
```

```
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')

# Drop target / leakage columns from movie_features so we can re-merge clean
LEAKAGE_AND_OLD_TARGETS = [
    'budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_mov = real_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE_AND_OLD_TARGETS, errors='ignore')

# Columns from metadata to use as features (excluding IDs, text, leakage)
META_KEEP = [
    'movie_id',
    # Scene properties (will be encoded below)
    'targeted_emotion', 'clip_duration_s',
    'cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
    'silence_ratio', 'music_presence', 'dialogue_density',
    'face_screen_time_ratio', 'lead_screen_time_ratio',
    # Movie metadata
    'release_year', 'genre_primary', 'genre_secondary',
    'country_of_origin', 'budget_categorical',
    # Targets (kept to extract y; not used as features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]

real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left')
real_mov['is_synthetic'] = 0
syn_mov = syn_mov.merge(syn_meta_sub, on='movie_id', how='left')
syn_mov['is_synthetic'] = 1

df_all = pd.concat([real_mov, syn_mov], ignore_index=True)
print(f'Real:      {len(real_mov)} movies')
print(f'Synthetic: {len(syn_mov)} movies')
print(f'Combined:  {len(df_all)} movies × {len(df_all.columns)} columns (pre
print()
print('Target distributions (pre-encoding):')
display(df_all[['is_synthetic', 'imdb_rating', 'wom_multiplier_log']]
        .groupby('is_synthetic').describe().round(2))
```

```
Real:      10 movies
Synthetic: 40 movies
Combined:  50 movies × 343 columns (pre-encoding)

Target distributions (pre-encoding):
```

	imdb_rating								wom_multiplier_log			
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min
is_synthetic												
0	10.0	7.41	0.73	6.5	6.8	7.45	8.02	8.5	10.0	1.77	0.64	0.75
1	40.0	7.48	0.55	6.5	7.0	7.50	7.82	8.4	40.0	1.70	0.54	0.34

Feature preparation

In [ ]:

```
DROP = {
    'movie_id', 'condition', 'n_participants',
    # Targets (cannot be features)
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
    # Control flag (not a feature)
    'is_synthetic',
}

df_feat = df_all.copy()

# ── Ordinal encoding: low / moderate / high → 1 / 2 / 3 ──
ORD_MAP = {'low': 1, 'moderate': 2, 'high': 3}
ORD_COLS = ['cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
            'silence_ratio', 'music_presence', 'dialogue_density',
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'budget_categorical']
for c in ORD_COLS:
    if c in df_feat.columns:
        df_feat[c] = df_feat[c].map(ORD_MAP)

# ── One-hot encode multi-category text columns ──
OH_COLS = ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country_
OH_COLS = [c for c in OH_COLS if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH_COLS, prefix_sep='_',
                         dummy_na=False, dtype=int)

# ── Build feature matrix ──
feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
groups_all = df_feat['movie_id'].values if 'movie_id' in df_feat.columns els
synth_mask_all = df_feat['is_synthetic'].values.astype(bool)

# Drop all-NaN columns
all_nan_cols = X_all.columns[X_all.isna().all()].tolist()
if all_nan_cols:
    print(f'Dropping {len(all_nan_cols)} all-NaN columns')
    X_all = X_all.drop(columns=all_nan_cols)

# Drop zero-variance columns
zero_var_cols = X_all.columns[X_all.std() == 0].tolist()
if zero_var_cols:
    print(f'Dropping {len(zero_var_cols)} zero-variance columns')
    X_all = X_all.drop(columns=zero_var_cols)

feature_cols = list(X_all.columns)

#Categorise final features for transparency
phys_cols = [c for c in feature_cols if c.endswith('__mean') or c.endswith('
ord_in_X  = [c for c in feature_cols if c in ORD_COLS]
```

```
oh_in_X   = [c for c in feature_cols
             if any(c.startswith(p + '_') for p in
                    ['targeted_emotion', 'genre_primary', 'genre_secondary',
                     'country_of_origin'])]
other_cols = [c for c in feature_cols
              if c not in phys_cols + ord_in_X + oh_in_X]

print(f'\nFinal feature matrix: {X_all.shape}')
print(f'  Physiological (mean/std):  {len(phys_cols):>4d}')
print(f'  Scene/movie ordinal:       {len(ord_in_X):>4d}  ({ord_in_X})')
print(f'  Scene/movie one-hot:       {len(oh_in_X):>4d}')
print(f'  Other numeric (year, dur): {len(other_cols):>4d}  ({other_cols})')
print(f'  TOTAL features:            {len(feature_cols):>4d}')
```

```
Final feature matrix: (50, 360)
  Physiological (mean/std):   320
  Scene/movie ordinal:         10  (['cut_count', 'brightness', 'motion_inte
nsity', 'audio_loudness', 'silence_ratio', 'music_presence', 'dialogue_densi
ty', 'face_screen_time_ratio', 'lead_screen_time_ratio', 'budget_categorica
l'])
  Scene/movie one-hot:         28
  Other numeric (year, dur):    2  (['clip_duration_s', 'release_year'])
  TOTAL features:             360
```

Evaluation helpers

In [5]:

```
def regression_metrics(y_true, y_pred):
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    if len(yt) < 3:
        return {'n': len(yt), 'r2': np.nan, 'mae': np.nan, 'rmse': np.nan,
                'spearman': np.nan, 'pearson': np.nan}
    return {
        'n': len(yt),
        'r2': r2_score(yt, yp),
        'mae': mean_absolute_error(yt, yp),
        'rmse': np.sqrt(mean_squared_error(yt, yp)),
        'spearman': spearmanr(yt, yp).correlation,
        'pearson': pearsonr(yt, yp)[0],
    }

def loo_predict(estimator, X, y):
    return cross_val_predict(estimator, X, y, cv=LeaveOneOut(), n_jobs=-1)

def kfold_predict(estimator, X, y, n_splits=5, random_state=42):
    return cross_val_predict(estimator, X, y,
                             cv=KFold(n_splits=n_splits, shuffle=True,
                                      random_state=random_state),
                             n_jobs=-1)

def report(metrics, title=''):
    print(f'\n——— {title} ———')
    for k, v in metrics.items():
        if k == 'n': print(f'  {k:>10s}: {v}')
```

```
        else: print(f'  {k:>10s}: {v:.3f}' if not np.isnan(v) else f'  {k:>1

def plot_pred_vs_actual(y_true, y_pred, title, ax=None, y_std=None):
    if ax is None: fig, ax = plt.subplots(figsize=(5, 5))
    mask = ~(np.isnan(y_pred) | np.isnan(y_true))
    yt, yp = np.asarray(y_true)[mask], np.asarray(y_pred)[mask]
    if y_std is not None:
        ys = np.asarray(y_std)[mask]
        ax.errorbar(yt, yp, yerr=ys, fmt='o', alpha=0.6,
                    capsize=2, ecolor='gray', markersize=6)
    else:
        ax.scatter(yt, yp, alpha=0.6, s=40, edgecolor='k', linewidth=0.5)
    lo, hi = min(yt.min(), yp.min()), max(yt.max(), yp.max())
    ax.plot([lo, hi], [lo, hi], 'r--', alpha=0.5, label='y = x')
    r2 = r2_score(yt, yp)
    rho = spearmanr(yt, yp).correlation
    ax.set_xlabel('Actual'); ax.set_ylabel('Predicted')
    ax.set_title(f'{title}\nR²={r2:.2f}, Spearman ρ={rho:.2f}')
    ax.legend(); ax.grid(alpha=0.3)
    return ax
```

Real-only (LOO CV, n = 10)

In [ ]:

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import (
    ConstantKernel, RBF, Matern, WhiteKernel
)

mask_real = ~synth_mask_all
X_real = X_all[mask_real].reset_index(drop=True)
y_imdb_real = y_imdb_all[mask_real].reset_index(drop=True)
y_wom_real = y_wom_all[mask_real].reset_index(drop=True)

def make_gpr_pipeline(kernel_type='ard_rbf'):
    n_features_inferred = X_all.shape[1]   # uses available feature count
    if kernel_type == 'ard_rbf':
        kernel = (ConstantKernel(1.0, (1e-3, 1e3))
                  * RBF(length_scale=np.ones(n_features_inferred),
                        length_scale_bounds=(1e-2, 1e3))
                  + WhiteKernel(noise_level=0.1, noise_level_bounds=(1e-5, 1
    elif kernel_type == 'ard_matern25':
        kernel = (ConstantKernel(1.0, (1e-3, 1e3))
                  * Matern(length_scale=np.ones(n_features_inferred), nu=2.5
                           length_scale_bounds=(1e-2, 1e3))
                  + WhiteKernel(noise_level=0.1, noise_level_bounds=(1e-5, 1
    elif kernel_type == 'ard_matern15':
        kernel = (ConstantKernel(1.0, (1e-3, 1e3))
                  * Matern(length_scale=np.ones(n_features_inferred), nu=1.5
                           length_scale_bounds=(1e-2, 1e3))
                  + WhiteKernel(noise_level=0.1, noise_level_bounds=(1e-5, 1
    elif kernel_type == 'iso_rbf':           # isotropic RBF (single length-
        kernel = (ConstantKernel(1.0, (1e-3, 1e3))
                  * RBF(length_scale=1.0, length_scale_bounds=(1e-2, 1e3))
                  + WhiteKernel(noise_level=0.1, noise_level_bounds=(1e-5, 1
```

```
    else:
        raise ValueError(kernel_type)
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('model', GaussianProcessRegressor(
            kernel=kernel,
            n_restarts_optimizer=20,
            normalize_y=True,
            alpha=1e-8,
            random_state=42)),
    ])

# Baseline: ARD-RBF
pipe = make_gpr_pipeline('ard_rbf')
y_pred = loo_predict(pipe, X_real, y_imdb_real)
metrics_real_imdb_baseline = regression_metrics(y_imdb_real, y_pred)
report(metrics_real_imdb_baseline, 'Real / IMDb / baseline (ARD-RBF)')

pipe = make_gpr_pipeline('ard_rbf')
y_pred = loo_predict(pipe, X_real, y_wom_real)
metrics_real_wom_baseline = regression_metrics(y_wom_real, y_pred)
report(metrics_real_wom_baseline, 'Real / WOM / baseline (ARD-RBF)')
```

```
──── Real / IMDb / baseline (ARD-RBF) ────
         n: 10
        r2: -0.235
       mae: 0.678
      rmse: 0.774
  spearman: -1.000
   pearson: -1.000

──── Real / WOM / baseline (ARD-RBF) ────
         n: 10
        r2: -0.235
       mae: 0.573
      rmse: 0.671
  spearman: -1.000
   pearson: -1.000
```

Kernel selection (LOO grid)

In [ ]:
```
# Tune over kernel TYPE (ARD-RBF, ARD-Matern-1.5, ARD-Matern-2.5, isotropic-
# Kernel hyperparameters within each are auto-optimised
def cv_score_kernel(kernel_type, X, y, cv):
    pipe = make_gpr_pipeline(kernel_type)
    y_pred = cross_val_predict(pipe, X, y, cv=cv, n_jobs=-1)
    return regression_metrics(y, y_pred), y_pred

KERNEL_TYPES = ['ard_rbf', 'ard_matern25', 'ard_matern15', 'iso_rbf']

results_imdb = {}
for kt in KERNEL_TYPES:
    m, _ = cv_score_kernel(kt, X_real, y_imdb_real, LeaveOneOut())
```

```
    results_imdb[kt] = m
    print(f'  Real / IMDb / {kt:14s}: MAE={m["mae"]:.3f}, '
          f'R²={m["r2"]:.3f}, Spearman={m["spearman"]:.2f}')

best_kernel_imdb = min(results_imdb, key=lambda k: results_imdb[k]['mae'])
best_params_imdb_real = {'model__kernel': best_kernel_imdb}
metrics_real_imdb_tuned, y_pred_real_imdb_tuned = cv_score_kernel(
    best_kernel_imdb, X_real, y_imdb_real, LeaveOneOut())
print(f'\nBest IMDb kernel: {best_kernel_imdb}')
report(metrics_real_imdb_tuned, 'Real / IMDb / tuned')

results_wom = {}
for kt in KERNEL_TYPES:
    m, _ = cv_score_kernel(kt, X_real, y_wom_real, LeaveOneOut())
    results_wom[kt] = m
    print(f'  Real / WOM  / {kt:14s}: MAE={m["mae"]:.3f}, '
          f'R²={m["r2"]:.3f}, Spearman={m["spearman"]:.2f}')

best_kernel_wom = min(results_wom, key=lambda k: results_wom[k]['mae'])
best_params_wom_real = {'model__kernel': best_kernel_wom}
metrics_real_wom_tuned, y_pred_real_wom_tuned = cv_score_kernel(
    best_kernel_wom, X_real, y_wom_real, LeaveOneOut())
print(f'\nBest WOM kernel: {best_kernel_wom}')
report(metrics_real_wom_tuned, 'Real / WOM / tuned')
```

```
  Real / IMDb / ard_rbf        : MAE=0.678, R²=-0.235, Spearman=-1.00
  Real / IMDb / ard_matern25   : MAE=0.678, R²=-0.235, Spearman=-1.00
  Real / IMDb / ard_matern15   : MAE=0.678, R²=-0.235, Spearman=-1.00
  Real / IMDb / iso_rbf        : MAE=0.678, R²=-0.235, Spearman=-1.00

Best IMDb kernel: ard_rbf

──── Real / IMDb / tuned ────
           n: 10
          r2: -0.235
         mae: 0.678
        rmse: 0.774
    spearman: -1.000
     pearson: -1.000
  Real / WOM  / ard_rbf        : MAE=0.573, R²=-0.235, Spearman=-1.00
  Real / WOM  / ard_matern25   : MAE=0.573, R²=-0.235, Spearman=-1.00
  Real / WOM  / ard_matern15   : MAE=0.573, R²=-0.235, Spearman=-1.00
  Real / WOM  / iso_rbf        : MAE=0.573, R²=-0.235, Spearman=-1.00

Best WOM kernel: ard_rbf

──── Real / WOM / tuned ────
           n: 10
          r2: -0.235
         mae: 0.573
        rmse: 0.671
    spearman: -1.000
     pearson: -1.000
```

Posterior uncertainty (the key advantage of this model)

In [8]:
```
# For each held-out movie, get the prediction's posterior std.
# This is the unique selling point of GPR: principled CIs at small n.

def loo_predict_with_std(pipe_factory, kernel_type, X, y):
    """Returns (y_pred, y_std) using LOO; refits per fold."""
    n = len(y)
    y_pred = np.empty(n)
    y_std  = np.empty(n)
    for i in range(n):
        idx = np.arange(n) != i
        pipe = pipe_factory(kernel_type)
        pipe.fit(X.iloc[idx], y.iloc[idx])
        # Need to access the GPR step to get std
        X_test_imp = pipe.named_steps['imputer'].transform(X.iloc[[i]])
        X_test_sc  = pipe.named_steps['scaler'].transform(X_test_imp)
        mu, sigma = pipe.named_steps['model'].predict(X_test_sc, return_std=
        y_pred[i] = mu[0]; y_std[i] = sigma[0]
    return y_pred, y_std

print('Computing LOO predictions with posterior std (IMDb, real-only)...')
y_pred_real_imdb_with_std, y_std_real_imdb = loo_predict_with_std(
    make_gpr_pipeline, best_kernel_imdb, X_real, y_imdb_real)

print(f'\nMovie-by-movie predictions (real, IMDb):')
print(f'  {"movie_idx":>10s}  {"actual":>8s}  {"predicted":>10s}  {"±std":>8
for i in range(len(y_imdb_real)):
    print(f'  {i:>10d}  {y_imdb_real.iloc[i]:>8.2f}  '
          f'{y_pred_real_imdb_with_std[i]:>10.2f}  ±{y_std_real_imdb[i]:.2f}

# Recompute metrics from these predictions
metrics_real_imdb_tuned = regression_metrics(y_imdb_real, y_pred_real_imdb_w
print(f'\nFinal real / IMDb / tuned metrics with posterior:')
report(metrics_real_imdb_tuned, '')
```

```
Computing LOO predictions with posterior std (IMDb, real-only)...

Movie-by-movie predictions (real, IMDb):
   movie_idx    actual   predicted      ±std
           0      7.80        7.37  ±0.72
           1      8.20        7.32  ±0.68
           2      8.10        7.33  ±0.69
           3      6.80        7.48  ±0.70
           4      8.50        7.29  ±0.63
           5      7.40        7.41  ±0.73
           6      6.50        7.51  ±0.66
           7      6.50        7.51  ±0.66
           8      7.50        7.40  ±0.73
           9      6.80        7.48  ±0.70

Final real / IMDb / tuned metrics with posterior:

————  ————
           n: 10
          r2: -0.235
         mae: 0.678
        rmse: 0.774
    spearman: -1.000
     pearson: -1.000
```

In [9]:
```python
# Same for WOM
print('Computing LOO predictions with posterior std (WOM, real-only)...')
y_pred_real_wom_with_std, y_std_real_wom = loo_predict_with_std(
    make_gpr_pipeline, best_kernel_wom, X_real, y_wom_real)
metrics_real_wom_tuned = regression_metrics(y_wom_real, y_pred_real_wom_with
report(metrics_real_wom_tuned, 'Real / WOM / tuned (with posterior std)')
```

```
Computing LOO predictions with posterior std (WOM, real-only)...

——— Real / WOM / tuned (with posterior std) ———
           n: 10
          r2: -0.235
         mae: 0.573
        rmse: 0.671
    spearman: -1.000
     pearson: -1.000
```

Augmented (5-fold CV, n = 50)

In [10]:
```python
# Baseline ARD-RBF
pipe = make_gpr_pipeline('ard_rbf')
y_pred_aug_imdb = kfold_predict(pipe, X_all, y_imdb_all, n_splits=5)
metrics_aug_imdb_baseline = regression_metrics(y_imdb_all, y_pred_aug_imdb)
report(metrics_aug_imdb_baseline, 'Augmented / IMDb / baseline (ARD-RBF)')

pipe = make_gpr_pipeline('ard_rbf')
y_pred_aug_wom = kfold_predict(pipe, X_all, y_wom_all, n_splits=5)
metrics_aug_wom_baseline = regression_metrics(y_wom_all, y_pred_aug_wom)
report(metrics_aug_wom_baseline, 'Augmented / WOM / baseline (ARD-RBF)')
```

```
──── Augmented / IMDb / baseline (ARD-RBF) ────
           n: 50
          r2: -0.013
         mae: 0.500
        rmse: 0.579
    spearman: -0.171
     pearson: -0.155

──── Augmented / WOM / baseline (ARD-RBF) ────
           n: 50
          r2: -0.028
         mae: 0.460
        rmse: 0.556
    spearman: -0.219
     pearson: -0.224
```

In [11]:

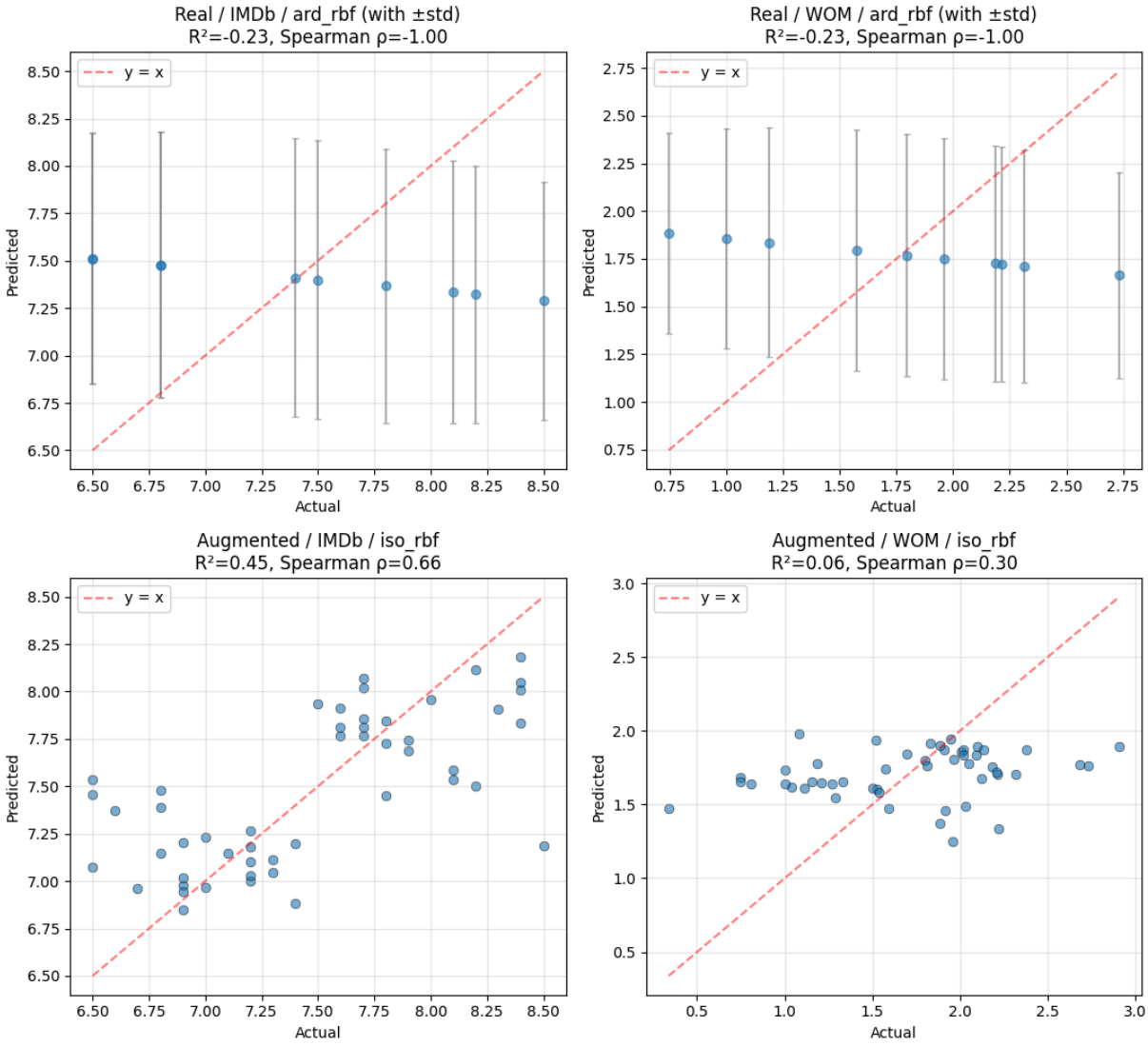
```python
# Tuned: select best kernel via 5-fold CV
results_aug_imdb = {}
for kt in KERNEL_TYPES:
    m, _ = cv_score_kernel(kt, X_all, y_imdb_all,
                           KFold(5, shuffle=True, random_state=42))
    results_aug_imdb[kt] = m
    print(f'  Aug / IMDb / {kt:14s}: MAE={m["mae"]:.3f}, '
          f'R²={m["r2"]:.3f}, Spearman={m["spearman"]:.2f}')

best_kernel_imdb_aug = min(results_aug_imdb, key=lambda k: results_aug_imdb[
best_params_imdb = {'model__kernel': best_kernel_imdb_aug}
metrics_aug_imdb_tuned, y_pred_aug_imdb_tuned = cv_score_kernel(
    best_kernel_imdb_aug, X_all, y_imdb_all,
    KFold(5, shuffle=True, random_state=42))
print(f'\nBest IMDb kernel: {best_kernel_imdb_aug}')
report(metrics_aug_imdb_tuned, 'Augmented / IMDb / tuned')

results_aug_wom = {}
for kt in KERNEL_TYPES:
    m, _ = cv_score_kernel(kt, X_all, y_wom_all,
                           KFold(5, shuffle=True, random_state=42))
    results_aug_wom[kt] = m
    print(f'  Aug / WOM  / {kt:14s}: MAE={m["mae"]:.3f}, '
          f'R²={m["r2"]:.3f}, Spearman={m["spearman"]:.2f}')

best_kernel_wom_aug = min(results_aug_wom, key=lambda k: results_aug_wom[k][
best_params_wom = {'model__kernel': best_kernel_wom_aug}
metrics_aug_wom_tuned, y_pred_aug_wom_tuned = cv_score_kernel(
    best_kernel_wom_aug, X_all, y_wom_all,
    KFold(5, shuffle=True, random_state=42))
print(f'\nBest WOM kernel: {best_kernel_wom_aug}')
report(metrics_aug_wom_tuned, 'Augmented / WOM / tuned')
```

```
  Aug / IMDb / ard_rbf       : MAE=0.500, R²=-0.013, Spearman=-0.17
  Aug / IMDb / ard_matern25  : MAE=0.500, R²=-0.013, Spearman=-0.17
  Aug / IMDb / ard_matern15  : MAE=0.500, R²=-0.013, Spearman=-0.09
  Aug / IMDb / iso_rbf       : MAE=0.319, R²=0.454, Spearman=0.66

Best IMDb kernel: iso_rbf

——— Augmented / IMDb / tuned ———
           n: 50
          r2: 0.454
         mae: 0.319
        rmse: 0.425
    spearman: 0.661
     pearson: 0.674
  Aug / WOM  / ard_rbf       : MAE=0.460, R²=-0.028, Spearman=-0.22
  Aug / WOM  / ard_matern25  : MAE=0.460, R²=-0.028, Spearman=-0.22
  Aug / WOM  / ard_matern15  : MAE=0.460, R²=-0.028, Spearman=-0.15
  Aug / WOM  / iso_rbf       : MAE=0.431, R²=0.063, Spearman=0.30

Best WOM kernel: iso_rbf

——— Augmented / WOM / tuned ———
           n: 50
          r2: 0.063
         mae: 0.431
        rmse: 0.531
    spearman: 0.300
     pearson: 0.254
```

Visuals(with posterior uncertainty for real-only)

In [12]:

```python
fig, axes = plt.subplots(2, 2, figsize=(11, 10))
# Real-only: show error bars from posterior std
plot_pred_vs_actual(y_imdb_real, y_pred_real_imdb_with_std,
                    f'Real / IMDb / {best_kernel_imdb} (with ±std)',
                    axes[0, 0], y_std=y_std_real_imdb)
plot_pred_vs_actual(y_wom_real, y_pred_real_wom_with_std,
                    f'Real / WOM / {best_kernel_wom} (with ±std)',
                    axes[0, 1], y_std=y_std_real_wom)
plot_pred_vs_actual(y_imdb_all, y_pred_aug_imdb_tuned,
                    f'Augmented / IMDb / {best_kernel_imdb_aug}', axes[1, 0]
plot_pred_vs_actual(y_wom_all, y_pred_aug_wom_tuned,
                    f'Augmented / WOM / {best_kernel_wom_aug}', axes[1, 1])
plt.tight_layout(); plt.show()
```



8 · Posterior calibration check

In [ ]:

```
true_y = np.asarray(y_imdb_real)
pred_y = y_pred_real_imdb_with_std
std_y  = y_std_real_imdb
within_1 = np.abs(true_y - pred_y) < std_y
within_2 = np.abs(true_y - pred_y) < 2 * std_y
print(f'IMDb posterior calibration (real-only, n={len(true_y)}):')
print(f'  {within_1.sum()}/{len(true_y)} ({within_1.mean()*100:.0f}%) within
      f'(ideal: 68%)')
print(f'  {within_2.sum()}/{len(true_y)} ({within_2.mean()*100:.0f}%) within
      f'(ideal: 95%)')

true_y = np.asarray(y_wom_real)
pred_y = y_pred_real_wom_with_std
std_y  = y_std_real_wom
within_1 = np.abs(true_y - pred_y) < std_y
within_2 = np.abs(true_y - pred_y) < 2 * std_y
print(f'\nWOM posterior calibration (real-only, n={len(true_y)}):')
```

```
print(f'  {within_1.sum()}/{len(true_y)} ({within_1.mean()*100:.0f}%) within
print(f'  {within_2.sum()}/{len(true_y)} ({within_2.mean()*100:.0f}%) within
```

```
IMDb posterior calibration (real-only, n=10):
  5/10 (50%) within ±1 std  (ideal: 68%)
  10/10 (100%) within ±2 std  (ideal: 95%)

WOM posterior calibration (real-only, n=10):
  6/10 (60%) within ±1 std
  9/10 (90%) within ±2 std
```

In [14]:

```
out_path = RUN_DIR / 'results_gaussian_process.json'
results_to_save = {
    'model': 'gaussian_process',
    'feature_count': len(feature_cols),
    'best_params_imdb': best_params_imdb,
    'best_params_wom':  best_params_wom,
    'metrics_real_imdb_baseline': metrics_real_imdb_baseline,
    'metrics_real_wom_baseline':  metrics_real_wom_baseline,
    'metrics_real_imdb_tuned':    metrics_real_imdb_tuned,
    'metrics_real_wom_tuned':     metrics_real_wom_tuned,
    'metrics_aug_imdb_baseline':  metrics_aug_imdb_baseline,
    'metrics_aug_wom_baseline':   metrics_aug_wom_baseline,
    'metrics_aug_imdb_tuned':     metrics_aug_imdb_tuned,
    'metrics_aug_wom_tuned':      metrics_aug_wom_tuned,
}
with open(out_path, 'w') as f:
    json.dump(results_to_save, f, indent=2, default=str)
print(f'Saved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v2/results_gaussian_process.json
```

07

NOTEBOOK

Baseline classification (six classifiers)

07_classification.pdf

Classification — Combined Success (IMDb + WOM)

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)

from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v3'
RUN_DIR.mkdir(parents=True, exist_ok=True)
print('Data dir:', DATA_DIR)
print('Run dir: ', RUN_DIR)
```

```
Mounted at /content/drive
Data dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6
Run dir:  /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6/colab_runs_v3
```

In [ ]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, StratifiedKFold, GridSearchCV,
    cross_val_predict, train_test_split, cross_val_score,
)
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
    accuracy_score, f1_score, roc_auc_score,
    confusion_matrix, classification_report, ConfusionMatrixDisplay,
)
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

2 · Load data

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')

LEAKAGE = ['budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
           'imdb_rating', 'wom_multiplier', 'wom_multiplier_log']
real_mov = real_mov.drop(columns=LEAKAGE, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE, errors='ignore')

META_KEEP = [
    'movie_id', 'targeted_emotion', 'clip_duration_s',
    'cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
    'silence_ratio', 'music_presence', 'dialogue_density',
    'face_screen_time_ratio', 'lead_screen_time_ratio',
    'release_year', 'genre_primary', 'genre_secondary',
    'country_of_origin', 'budget_categorical',
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]
real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left'); real_mo
syn_mov  = syn_mov.merge(syn_meta_sub, on='movie_id', how='left'); syn_mov['
df_all = pd.concat([real_mov, syn_mov], ignore_index=True)
print(f'Combined: {len(df_all)} movies × {len(df_all.columns)} cols')
```

```
Combined: 50 movies × 343 cols
```

Encode features

In [ ]:
```
DROP = {'movie_id', 'condition', 'n_participants',
        'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
        'is_synthetic'}
df_feat = df_all.copy()

ORD_MAP = {'low': 1, 'moderate': 2, 'high': 3}
ORD_COLS = ['cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
            'silence_ratio', 'music_presence', 'dialogue_density',
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'budget_categorical']
for c in ORD_COLS:
    if c in df_feat.columns: df_feat[c] = df_feat[c].map(ORD_MAP)

OH_COLS = ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country_
OH_COLS = [c for c in OH_COLS if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH_COLS, prefix_sep='_',
                         dummy_na=False, dtype=int)

feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
synth_mask_all = df_feat['is_synthetic'].values.astype(bool)
```

```
X_all = X_all.drop(columns=X_all.columns[X_all.isna().all()].tolist())
X_all = X_all.drop(columns=X_all.columns[X_all.std() == 0].tolist())
feature_cols = list(X_all.columns)
print(f'Final feature matrix: {X_all.shape}')
```

```
Final feature matrix: (50, 360)
```

Building the combined success class

In [ ]:
```
# Combine IMDb and WOM into a single composite score, then threshold at medi
z_imdb = (y_imdb_all - y_imdb_all.mean()) / y_imdb_all.std()
z_wom  = (y_wom_all  - y_wom_all.mean())  / y_wom_all.std()
composite = (z_imdb + z_wom) / 2
threshold = composite.median()
y_class_all = (composite >= threshold).astype(int)

print(f'Composite stats:  mean={composite.mean():.3f}, std={composite.std():
print(f'Threshold (median): {threshold:.3f}')
print(f'\nClass distribution:')
print(y_class_all.value_counts().sort_index().rename({0:'low',1:'high'}).to_
print(f'\nReal-only class distribution:')
print(y_class_all[~synth_mask_all].value_counts().sort_index()
      .rename({0:'low',1:'high'}).to_string())
```

```
Composite stats:  mean=0.000, std=0.880
Threshold (median): 0.113

Class distribution:
low     25
high    25

Real-only class distribution:
low     4
high    6
```

80/20 stratified split

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X_all, y_class_all, test_size=0.2, random_state=42,
    stratify=y_class_all,
)
print(f'Train: {len(X_train)} (class balance: {y_train.value_counts(normaliz
print(f'Test:  {len(X_test)}  (class balance: {y_test.value_counts(normalize
print(f'\nTrain features: {X_train.shape}')
print(f'Test features:  {X_test.shape}')
```

```
Train: 40 (class balance: {0: 0.5, 1: 0.5})
Test:  10  (class balance: {0: 0.5, 1: 0.5})

Train features: (40, 360)
Test features:  (10, 360)
```

Helper functions

In [ ]:
```
def evaluate_classifier(estimator, X_train, X_test, y_train, y_test, name=''
    """Fit on train, predict on test, return metrics + predictions."""
    estimator.fit(X_train, y_train)
    y_pred  = estimator.predict(X_test)
    try:
        y_proba = estimator.predict_proba(X_test)[:, 1]
    except (AttributeError, NotImplementedError):
        try:
            y_proba = estimator.decision_function(X_test)
        except (AttributeError, NotImplementedError):
            y_proba = y_pred.astype(float)
    metrics = {
        'accuracy':  accuracy_score(y_test, y_pred),
        'f1':        f1_score(y_test, y_pred, zero_division=0),
        'roc_auc':   roc_auc_score(y_test, y_proba) if len(np.unique(y_test)
        'n_test':    len(y_test),
    }
    return metrics, y_pred, y_proba

def report_clf(metrics, name):
    print(f'\n——— {name} ———')
    for k, v in metrics.items():
        print(f'  {k:>10s}: {v:.3f}' if isinstance(v, float) else f'  {k:>10

def tune_and_evaluate(pipeline_factory, param_grid, X_train, y_train, X_test
                      scoring='roc_auc', cv_splits=5, name='Model'):
    """GridSearchCV on training set, then evaluate on test."""
    pipe = pipeline_factory()
    cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=42)
    search = GridSearchCV(pipe, param_grid, cv=cv, scoring=scoring,
                          n_jobs=-1, refit=True)
    search.fit(X_train, y_train)
    best = search.best_estimator_
    metrics, y_pred, y_proba = evaluate_classifier(best, X_train, X_test, y_
    report_clf(metrics, f'{name} (test set)')
    print(f'  Best params: {search.best_params_}')
    return metrics, search.best_params_, y_pred, y_proba
```

Train and evaluate all 6 classifiers

Logistic Regression with Elastic Net (L1+L2)

In [ ]:
```
from sklearn.linear_model import LogisticRegression

def make_logreg_enet():
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
```

```
        ('scaler', StandardScaler()),
        ('model', LogisticRegression(penalty='elasticnet', solver='saga',
                                     max_iter=10000, random_state=42)),
    ])

param_grid = {
    'model__C':        np.logspace(-3, 2, 12).tolist(),
    'model__l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9],
}
m_enet, p_enet, _, _ = tune_and_evaluate(make_logreg_enet, param_grid,
                                         X_train, y_train, X_test, y_test,
                                         name='LogReg (ElasticNet)')
```

```
─── LogReg (ElasticNet) (test set) ───
   accuracy: 0.900
         f1: 0.889
    roc_auc: 0.880
     n_test: 10
 Best params: {'model__C': 0.02310129700083159, 'model__l1_ratio': 0.1}
```

Logistic Regression with L2 (Ridge)

In [ ]:
```
def make_logreg_l2():
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('model', LogisticRegression(penalty='l2', solver='lbfgs',
                                     max_iter=10000, random_state=42)),
    ])

param_grid = {'model__C': np.logspace(-3, 3, 15).tolist()}
m_l2, p_l2, _, _ = tune_and_evaluate(make_logreg_l2, param_grid,
                                     X_train, y_train, X_test, y_test,
                                     name='LogReg (L2)')
```

```
─── LogReg (L2) (test set) ───
   accuracy: 0.900
         f1: 0.889
    roc_auc: 0.880
     n_test: 10
 Best params: {'model__C': 0.001}
```

Random Forest Classifier

In [ ]:
```
from sklearn.ensemble import RandomForestClassifier

def make_rf_clf():
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('model', RandomForestClassifier(random_state=42, n_jobs=-1)),
    ])

param_grid = {
```

```
    'model__n_estimators':    [200, 500],
    'model__max_depth':       [3, 5, 7, None],
    'model__min_samples_leaf':[1, 2, 4],
    'model__max_features':   ['sqrt', 0.3, 0.5],
}
m_rf, p_rf, _, _ = tune_and_evaluate(make_rf_clf, param_grid,
                                     X_train, y_train, X_test, y_test,
                                     name='Random Forest')
```

```
──── Random Forest (test set) ────
    accuracy: 0.900
          f1: 0.889
     roc_auc: 0.800
      n_test: 10
  Best params: {'model__max_depth': 3, 'model__max_features': 0.3, 'model__m
in_samples_leaf': 1, 'model__n_estimators': 500}
```

Gradient Boosting Classifier

In [ ]:
```
from sklearn.ensemble import HistGradientBoostingClassifier

def make_gbm_clf():
    return Pipeline([
        ('model', HistGradientBoostingClassifier(random_state=42)),
    ])

param_grid = {
    'model__learning_rate':     [0.01, 0.05, 0.1],
    'model__max_iter':          [100, 200, 400],
    'model__max_depth':         [3, 5, None],
    'model__min_samples_leaf':  [4, 10],
    'model__l2_regularization': [0.0, 1.0, 10.0],
}
m_gbm, p_gbm, _, _ = tune_and_evaluate(make_gbm_clf, param_grid,
                                       X_train, y_train, X_test, y_test,
                                       name='Gradient Boosting')
```

```
──── Gradient Boosting (test set) ────
    accuracy: 0.800
          f1: 0.800
     roc_auc: 0.760
      n_test: 10
  Best params: {'model__l2_regularization': 0.0, 'model__learning_rate': 0.0
5, 'model__max_depth': 3, 'model__max_iter': 400, 'model__min_samples_leaf':
4}
```

PLS-DA (PLS Discriminant Analysis)

In [ ]:
```
# Use PLSRegression with binarytarget, then threshold at 0.5
from sklearn.cross_decomposition import PLSRegression
from sklearn.base import BaseEstimator, ClassifierMixin

class PLSDA(BaseEstimator, ClassifierMixin):
```

```
    def __init__(self, n_components=2):
        self.n_components = n_components
    def fit(self, X, y):
        self.classes_ = np.unique(y)
        self.pls = PLSRegression(n_components=self.n_components, scale=False
        self.pls.fit(X, y.astype(float))
        return self
    def predict_proba(self, X):
        score = self.pls.predict(X).flatten()
        score = np.clip(score, 0, 1)
        return np.column_stack([1 - score, score])
    def predict(self, X):
        return (self.predict_proba(X)[:, 1] >= 0.5).astype(int)

def make_plsda():
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('model', PLSDA(n_components=2)),
    ])

param_grid = {'model__n_components': list(range(1, 16))}
m_pls, p_pls, _, _ = tune_and_evaluate(make_plsda, param_grid,
                                       X_train, y_train, X_test, y_test,
                                       scoring='roc_auc',
                                       name='PLS-DA')
```

```
─── PLS-DA (test set) ───
   accuracy: 0.900
         f1: 0.889
    roc_auc: 0.880
     n_test: 10
Best params: {'model__n_components': 1}
```

Gaussian Process Classifier

In [ ]:

```
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.gaussian_process.kernels import ConstantKernel, RBF, Matern

def make_gpc(kernel_type='rbf'):
    if kernel_type == 'rbf':
        kernel = ConstantKernel(1.0) * RBF(length_scale=1.0,
                                           length_scale_bounds=(1e-2, 1e3))
    elif kernel_type == 'matern25':
        kernel = ConstantKernel(1.0) * Matern(length_scale=1.0, nu=2.5,
                                              length_scale_bounds=(1e-2, 1
    else:
        raise ValueError(kernel_type)
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('model', GaussianProcessClassifier(kernel=kernel,
                                            n_restarts_optimizer=10,
                                            random_state=42)),
```

```
    ])

results_gpc = {}
for kt in ['rbf', 'matern25']:
    pipe = make_gpc(kt)
    pipe.fit(X_train, y_train)
    metrics, _, _ = evaluate_classifier(pipe, X_train, X_test, y_train, y_te
                                        f'GPC ({kt})')
    results_gpc[kt] = metrics
    report_clf(metrics, f'GPC ({kt}) — test')

# Pick best kernel by ROC-AUC
best_kernel = max(results_gpc, key=lambda k: results_gpc[k].get('roc_auc', 0
                                             if not np.isnan(results_gpc[k]
                                             else -1)
m_gpc = results_gpc[best_kernel]
p_gpc = {'model__kernel': best_kernel}
print(f'\nBest GPC kernel: {best_kernel}')
```

```
─── GPC (rbf) — test ───
    accuracy: 0.900
          f1: 0.889
     roc_auc: 0.840
      n_test: 10

─── GPC (matern25) — test ───
    accuracy: 0.900
          f1: 0.889
     roc_auc: 0.840
      n_test: 10

Best GPC kernel: rbf
```

Comparison of all classification models

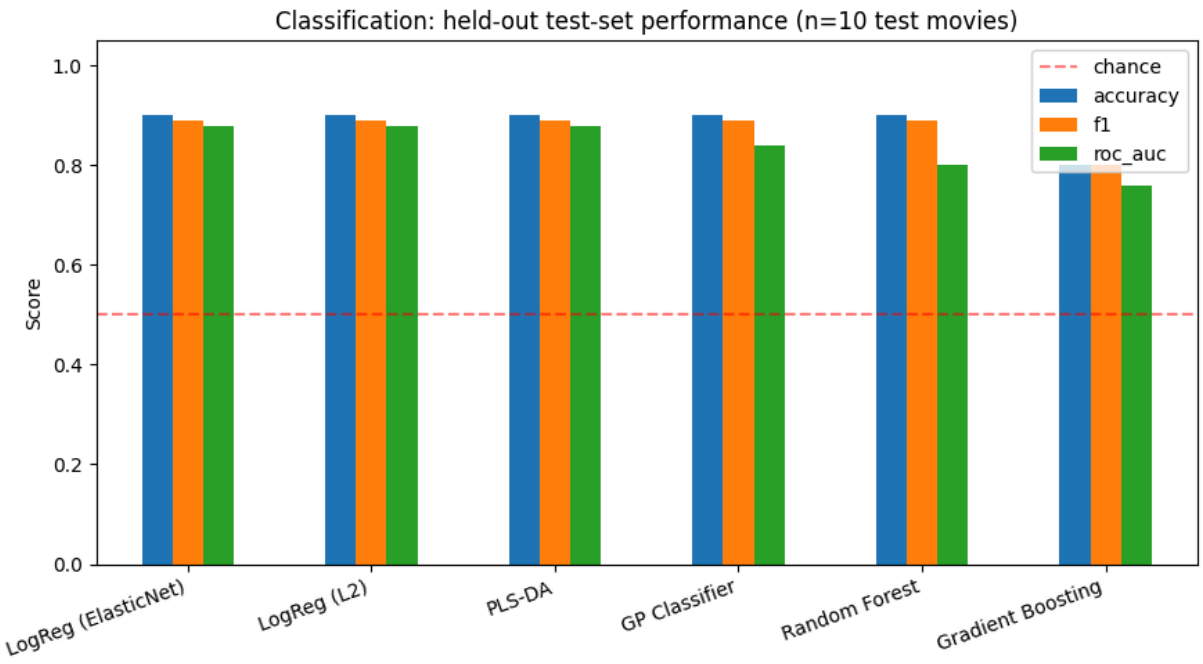
In [ ]:

```
all_results = {
    'LogReg (ElasticNet)': (m_enet, p_enet),
    'LogReg (L2)':          (m_l2, p_l2),
    'Random Forest':        (m_rf, p_rf),
    'Gradient Boosting':    (m_gbm, p_gbm),
    'PLS-DA':               (m_pls, p_pls),
    'GP Classifier':        (m_gpc, p_gpc),
}

comparison = pd.DataFrame({name: m for name, (m, _) in all_results.items()}).
comparison = comparison.sort_values('roc_auc', ascending=False)
print('Test-set performance ranking (by ROC-AUC):')
display(comparison.round(3))


fig, ax = plt.subplots(figsize=(9, 5))
comparison[['accuracy', 'f1', 'roc_auc']].plot(kind='bar', ax=ax)
ax.set_ylim(0, 1.05)
ax.axhline(0.5, color='red', linestyle='--', alpha=0.5, label='chance')
```

```
ax.set_ylabel('Score')
ax.set_title('Classification: held-out test-set performance (n=10 test movie
ax.legend()
plt.xticks(rotation=20, ha='right')
plt.tight_layout()
plt.show()
```

```
Test-set performance ranking (by ROC-AUC):
```

	accuracy	f1	roc_auc	n_test
LogReg (ElasticNet)	0.9	0.889	0.88	10.0
LogReg (L2)	0.9	0.889	0.88	10.0
PLS-DA	0.9	0.889	0.88	10.0
GP Classifier	0.9	0.889	0.84	10.0
Random Forest	0.9	0.889	0.80	10.0
Gradient Boosting	0.8	0.800	0.76	10.0

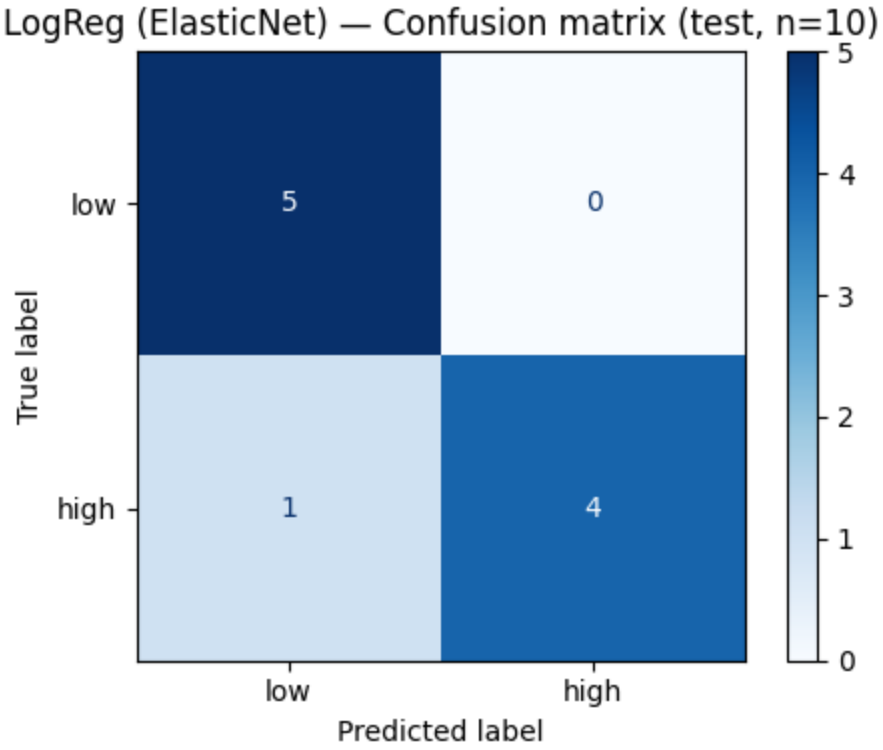


Confusion matrix for the best classifier

In [ ]:
```
best_name = comparison.index[0]
print(f'Best classifier on test set: {best_name}')

factories = {
    'LogReg (ElasticNet)': make_logreg_enet,
    'LogReg (L2)':          make_logreg_l2,
    'Random Forest':        make_rf_clf,
    'Gradient Boosting':    make_gbm_clf,
    'PLS-DA':               make_plsda,
    'GP Classifier':        lambda: make_gpc(p_gpc['model__kernel']),
```

```
}
factory = factories[best_name]
best_params = all_results[best_name][1]
pipe = factory()
if best_params:
    set_params = {k: v for k, v in best_params.items() if k.startswith('mode
    if set_params: pipe.set_params(**set_params)
pipe.fit(X_train, y_train)
y_pred_best = pipe.predict(X_test)

cm = confusion_matrix(y_test, y_pred_best)
fig, ax = plt.subplots(figsize=(5, 4))
ConfusionMatrixDisplay(cm, display_labels=['low', 'high']).plot(
    ax=ax, cmap='Blues', values_format='d')
ax.set_title(f'{best_name} — Confusion matrix (test, n=10)')
plt.tight_layout(); plt.show()
print('\nClassification report:')
print(classification_report(y_test, y_pred_best, target_names=['low','high']
```

```
Best classifier on test set: LogReg (ElasticNet)
```




```
Classification report:
              precision    recall  f1-score   support

         low       0.83      1.00      0.91         5
        high       1.00      0.80      0.89         5

    accuracy                           0.90        10
   macro avg       0.92      0.90      0.90        10
weighted avg       0.92      0.90      0.90        10
```

In [ ]:
```
out = {
    'task': 'binary_classification_combined_success',
```

```
    'composite_threshold': float(threshold),
    'class_distribution': y_class_all.value_counts().to_dict(),
    'train_size': len(X_train),
    'test_size': len(X_test),
    'feature_count': len(feature_cols),
    'results_per_model': {name: {'metrics': m, 'best_params': p}
                          for name, (m, p) in all_results.items()},
    'best_model_test': best_name,
}
out_path = RUN_DIR / 'results_classification.json'
with open(out_path, 'w') as f: json.dump(out, f, indent=2, default=str)
print(f'Saved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v3/results_classification.json
```

08

NOTEBOOK

Leakage-free nested cross-validation

08_nested_cv_finals.pdf

Nested CV — Top 3 Regressors

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)

from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v3'
RUN_DIR.mkdir(parents=True, exist_ok=True)
print('Data dir:', DATA_DIR)
print('Run dir: ', RUN_DIR)
```

```
Mounted at /content/drive
Data dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6
Run dir:  /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6/colab_runs_v3
```

In [ ]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, StratifiedKFold, GridSearchCV,
    cross_val_predict, train_test_split, cross_val_score,
)
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
    accuracy_score, f1_score, roc_auc_score,
    confusion_matrix, classification_report, ConfusionMatrixDisplay,
)
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')

LEAKAGE = ['budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
```

```
            'imdb_rating', 'wom_multiplier', 'wom_multiplier_log']
real_mov = real_mov.drop(columns=LEAKAGE, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE, errors='ignore')

META_KEEP = [
    'movie_id', 'targeted_emotion', 'clip_duration_s',
    'cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
    'silence_ratio', 'music_presence', 'dialogue_density',
    'face_screen_time_ratio', 'lead_screen_time_ratio',
    'release_year', 'genre_primary', 'genre_secondary',
    'country_of_origin', 'budget_categorical',
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]
real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left'); real_mo
syn_mov  = syn_mov.merge(syn_meta_sub, on='movie_id', how='left'); syn_mov['
df_all = pd.concat([real_mov, syn_mov], ignore_index=True)
print(f'Combined: {len(df_all)} movies × {len(df_all.columns)} cols')
```

```
Combined: 50 movies × 343 cols
```

In [ ]:

```
DROP = {'movie_id', 'condition', 'n_participants',
        'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
        'is_synthetic'}
df_feat = df_all.copy()

ORD_MAP = {'low': 1, 'moderate': 2, 'high': 3}
ORD_COLS = ['cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
            'silence_ratio', 'music_presence', 'dialogue_density',
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'budget_categorical']
for c in ORD_COLS:
    if c in df_feat.columns: df_feat[c] = df_feat[c].map(ORD_MAP)

OH_COLS = ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country_
OH_COLS = [c for c in OH_COLS if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH_COLS, prefix_sep='_',
                         dummy_na=False, dtype=int)

feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
synth_mask_all = df_feat['is_synthetic'].values.astype(bool)

X_all = X_all.drop(columns=X_all.columns[X_all.isna().all()].tolist())
X_all = X_all.drop(columns=X_all.columns[X_all.std() == 0].tolist())
feature_cols = list(X_all.columns)
print(f'Final feature matrix: {X_all.shape}')
```

```
Final feature matrix: (50, 360)
```

Nested CV helper

In [ ]:
```python
from sklearn.base import clone

def nested_cv(pipeline_factory, param_grid, X, y, n_outer=5, n_inner=5,
              scoring='neg_mean_absolute_error', name=''):
    """
    Proper nested cross-validation:
      - Outer K-fold splits into train/test.
      - Inside each outer fold, GridSearchCV does hyperparameter tuning
        using inner K-fold on the OUTER TRAIN set ONLY.
      - Best model per outer fold predicts on the outer TEST set
        (which the inner tuning has never seen).

    Returns:
      out_predictions: array same length as y, with per-sample predictions
                       from the outer fold in which they were the test set.
      best_params_per_fold: list of best param dicts (one per outer fold).
      metrics: dict of aggregated R², MAE, RMSE, Spearman, Pearson.
    """
    outer_cv = KFold(n_splits=n_outer, shuffle=True, random_state=42)
    inner_cv = KFold(n_splits=n_inner, shuffle=True, random_state=42)

    out_pred = np.empty(len(y))
    out_pred[:] = np.nan
    fold_params = []

    for fold_i, (train_idx, test_idx) in enumerate(outer_cv.split(X)):
        X_tr, X_te = X.iloc[train_idx], X.iloc[test_idx]
        y_tr, y_te = y.iloc[train_idx], y.iloc[test_idx]

        pipe = pipeline_factory()
        search = GridSearchCV(pipe, param_grid, cv=inner_cv,
                              scoring=scoring, n_jobs=-1, refit=True)
        search.fit(X_tr, y_tr)
        out_pred[test_idx] = search.best_estimator_.predict(X_te)
        fold_params.append(search.best_params_)
        print(f'  Outer fold {fold_i+1}/{n_outer}: best_params={search.best_

    metrics = {
        'r2':       r2_score(y, out_pred),
        'mae':      mean_absolute_error(y, out_pred),
        'rmse':     np.sqrt(mean_squared_error(y, out_pred)),
        'spearman': spearmanr(y, out_pred).correlation,
        'pearson':  pearsonr(y, out_pred)[0],
    }
    print(f'\n{name}: R²={metrics["r2"]:.3f}, MAE={metrics["mae"]:.3f}, '
          f'Spearman ρ={metrics["spearman"]:.3f}')
    return out_pred, fold_params, metrics
```

Random Forest (nested CV)

In [ ]:
```python
from sklearn.ensemble import RandomForestRegressor

def make_rf():
```

```
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('model', RandomForestRegressor(random_state=42, n_jobs=-1)),
    ])

rf_grid = {
    'model__n_estimators':    [200, 500],
    'model__max_depth':       [3, 5, 7, None],
    'model__min_samples_leaf':[1, 2, 4],
    'model__max_features':   ['sqrt', 0.3, 0.5],
}

print('=== RANDOM FOREST — IMDb (nested CV) ===')
rf_imdb_pred, rf_imdb_params, rf_imdb_m = nested_cv(
    make_rf, rf_grid, X_all, y_imdb_all, name='RF-IMDb')

print('\n=== RANDOM FOREST — WOM (nested CV) ===')
rf_wom_pred, rf_wom_params, rf_wom_m = nested_cv(
    make_rf, rf_grid, X_all, y_wom_all, name='RF-WOM')
```

```
=== RANDOM FOREST — IMDb (nested CV) ===
  Outer fold 1/5: best_params={'model__max_depth': 3, 'model__max_features':
'sqrt', 'model__min_samples_leaf': 4, 'model__n_estimators': 500}
  Outer fold 2/5: best_params={'model__max_depth': 3, 'model__max_features':
'sqrt', 'model__min_samples_leaf': 1, 'model__n_estimators': 200}
  Outer fold 3/5: best_params={'model__max_depth': 5, 'model__max_features':
'sqrt', 'model__min_samples_leaf': 1, 'model__n_estimators': 500}
  Outer fold 4/5: best_params={'model__max_depth': 3, 'model__max_features':
0.3, 'model__min_samples_leaf': 4, 'model__n_estimators': 500}
  Outer fold 5/5: best_params={'model__max_depth': 5, 'model__max_features':
0.3, 'model__min_samples_leaf': 1, 'model__n_estimators': 200}

RF-IMDb: R²=0.492, MAE=0.318, Spearman ρ=0.643

=== RANDOM FOREST — WOM (nested CV) ===
  Outer fold 1/5: best_params={'model__max_depth': 3, 'model__max_features':
0.3, 'model__min_samples_leaf': 1, 'model__n_estimators': 200}
  Outer fold 2/5: best_params={'model__max_depth': 5, 'model__max_features':
'sqrt', 'model__min_samples_leaf': 1, 'model__n_estimators': 200}
  Outer fold 3/5: best_params={'model__max_depth': 3, 'model__max_features':
0.3, 'model__min_samples_leaf': 1, 'model__n_estimators': 500}
  Outer fold 4/5: best_params={'model__max_depth': 5, 'model__max_features':
'sqrt', 'model__min_samples_leaf': 4, 'model__n_estimators': 200}
  Outer fold 5/5: best_params={'model__max_depth': 3, 'model__max_features':
0.5, 'model__min_samples_leaf': 1, 'model__n_estimators': 200}

RF-WOM: R²=0.174, MAE=0.397, Spearman ρ=0.377
```

Gradient Boosting (nested CV)

In [ ]:

```
from sklearn.ensemble import HistGradientBoostingRegressor

def make_gbm():
    return Pipeline([
```

```
        ('model', HistGradientBoostingRegressor(random_state=42)),
    ])

gbm_grid = {
    'model__learning_rate':     [0.01, 0.05, 0.1],
    'model__max_iter':          [100, 200, 400],
    'model__max_depth':         [3, 5, None],
    'model__min_samples_leaf':  [4, 10],
    'model__l2_regularization': [0.0, 1.0, 10.0],
}

print('=== GRADIENT BOOSTING — IMDb (nested CV) ===')
gbm_imdb_pred, gbm_imdb_params, gbm_imdb_m = nested_cv(
    make_gbm, gbm_grid, X_all, y_imdb_all, name='GBM-IMDb')

print('\n=== GRADIENT BOOSTING — WOM (nested CV) ===')
gbm_wom_pred, gbm_wom_params, gbm_wom_m = nested_cv(
    make_gbm, gbm_grid, X_all, y_wom_all, name='GBM-WOM')
```

```
=== GRADIENT BOOSTING — IMDb (nested CV) ===
  Outer fold 1/5: best_params={'model__l2_regularization': 10.0, 'model__lea
rning_rate': 0.1, 'model__max_depth': 3, 'model__max_iter': 100, 'model__min
_samples_leaf': 10}
  Outer fold 2/5: best_params={'model__l2_regularization': 10.0, 'model__lea
rning_rate': 0.1, 'model__max_depth': 3, 'model__max_iter': 400, 'model__min
_samples_leaf': 10}
  Outer fold 3/5: best_params={'model__l2_regularization': 10.0, 'model__lea
rning_rate': 0.05, 'model__max_depth': 3, 'model__max_iter': 200, 'model__mi
n_samples_leaf': 10}
  Outer fold 4/5: best_params={'model__l2_regularization': 1.0, 'model__lear
ning_rate': 0.05, 'model__max_depth': 3, 'model__max_iter': 400, 'model__min
_samples_leaf': 10}
  Outer fold 5/5: best_params={'model__l2_regularization': 1.0, 'model__lear
ning_rate': 0.1, 'model__max_depth': 3, 'model__max_iter': 400, 'model__min_
samples_leaf': 4}

GBM-IMDb: R²=0.369, MAE=0.341, Spearman ρ=0.609

=== GRADIENT BOOSTING — WOM (nested CV) ===
  Outer fold 1/5: best_params={'model__l2_regularization': 10.0, 'model__lea
rning_rate': 0.05, 'model__max_depth': 3, 'model__max_iter': 200, 'model__mi
n_samples_leaf': 10}
  Outer fold 2/5: best_params={'model__l2_regularization': 0.0, 'model__lear
ning_rate': 0.1, 'model__max_depth': None, 'model__max_iter': 100, 'model__m
in_samples_leaf': 4}
  Outer fold 3/5: best_params={'model__l2_regularization': 1.0, 'model__lear
ning_rate': 0.01, 'model__max_depth': 3, 'model__max_iter': 100, 'model__min
_samples_leaf': 4}
  Outer fold 4/5: best_params={'model__l2_regularization': 10.0, 'model__lea
rning_rate': 0.01, 'model__max_depth': 3, 'model__max_iter': 100, 'model__mi
n_samples_leaf': 10}
  Outer fold 5/5: best_params={'model__l2_regularization': 0.0, 'model__lear
ning_rate': 0.01, 'model__max_depth': 3, 'model__max_iter': 200, 'model__min
_samples_leaf': 10}

GBM-WOM: R²=0.079, MAE=0.417, Spearman ρ=0.261
```

PLS (nested CV)

In [ ]:
```
from sklearn.cross_decomposition import PLSRegression

def make_pls():
    return Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('model', PLSRegression(scale=False)),
    ])

pls_grid = {'model__n_components': list(range(1, 16))}

print('=== PLS — IMDb (nested CV) ===')
pls_imdb_pred, pls_imdb_params, pls_imdb_m = nested_cv(
    make_pls, pls_grid, X_all, y_imdb_all, name='PLS-IMDb')

print('\n=== PLS — WOM (nested CV) ===')
pls_wom_pred, pls_wom_params, pls_wom_m = nested_cv(
    make_pls, pls_grid, X_all, y_wom_all, name='PLS-WOM')
```

```
=== PLS — IMDb (nested CV) ===
  Outer fold 1/5: best_params={'model__n_components': 1}
  Outer fold 2/5: best_params={'model__n_components': 3}
  Outer fold 3/5: best_params={'model__n_components': 2}
  Outer fold 4/5: best_params={'model__n_components': 3}
  Outer fold 5/5: best_params={'model__n_components': 3}

PLS-IMDb: R²=0.427, MAE=0.329, Spearman ρ=0.649

=== PLS — WOM (nested CV) ===
  Outer fold 1/5: best_params={'model__n_components': 1}
  Outer fold 2/5: best_params={'model__n_components': 1}
  Outer fold 3/5: best_params={'model__n_components': 1}
  Outer fold 4/5: best_params={'model__n_components': 1}
  Outer fold 5/5: best_params={'model__n_components': 1}

PLS-WOM: R²=0.190, MAE=0.402, Spearman ρ=0.385
```

Comparison table normal CV vs nested CV

In [ ]:
```
v2 = {
    'RF-IMDb':  {'r2': 0.544, 'spearman': 0.683, 'mae': 0.302},
    'RF-WOM':   {'r2': 0.207, 'spearman': 0.415, 'mae': 0.380},
    'GBM-IMDb': {'r2': 0.476, 'spearman': 0.712, 'mae': 0.326},
    'GBM-WOM':  {'r2': 0.136, 'spearman': 0.364, 'mae': 0.395},
    'PLS-IMDb': {'r2': 0.472, 'spearman': 0.680, 'mae': 0.309},
    'PLS-WOM':  {'r2': 0.190, 'spearman': 0.385, 'mae': 0.402},
}

nested_metrics = {
    'RF-IMDb':  rf_imdb_m,  'RF-WOM':  rf_wom_m,
```

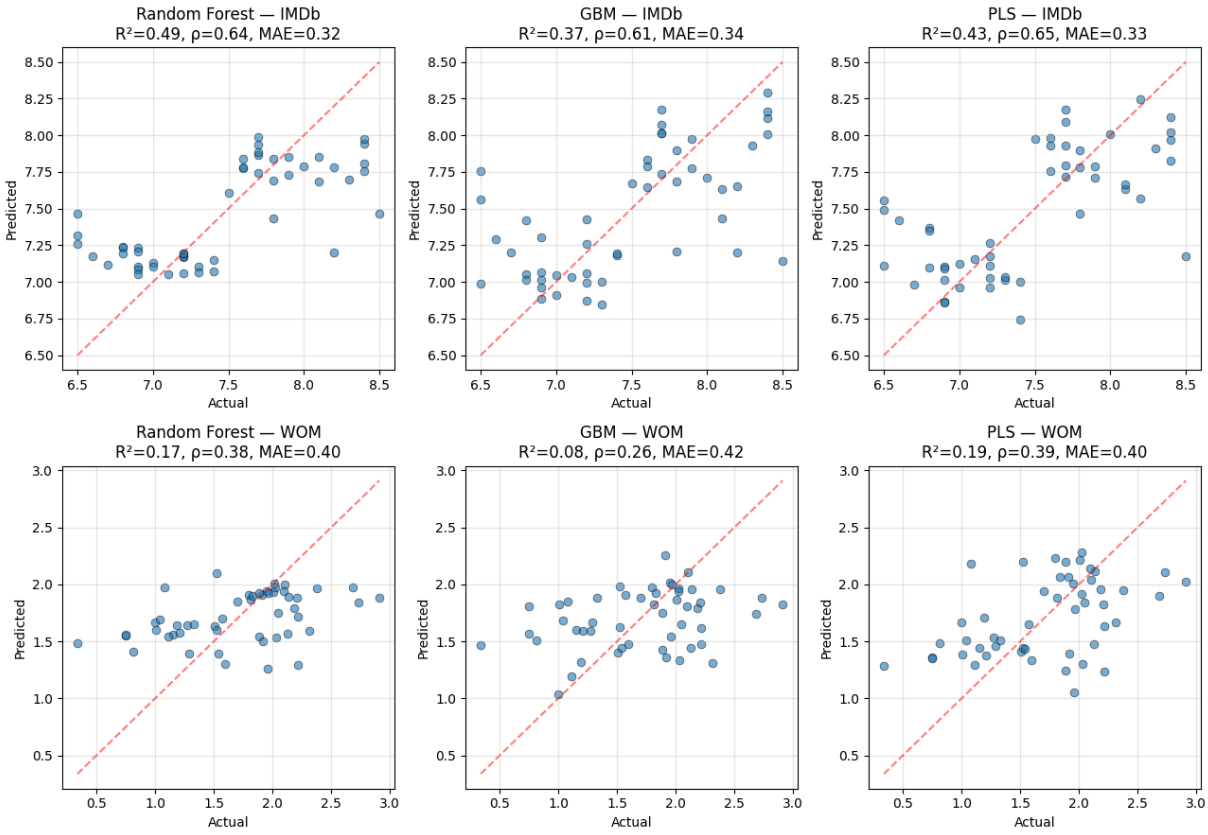
```
    'GBM-IMDb': gbm_imdb_m, 'GBM-WOM': gbm_wom_m,
    'PLS-IMDb': pls_imdb_m, 'PLS-WOM': pls_wom_m,
}

rows = []
for k in v2:
    rows.append({
        'Model_Target':  k,
        'v2_R²':         v2[k]['r2'],
        'nested_R²':     nested_metrics[k]['r2'],
        'Δ R²':          nested_metrics[k]['r2'] - v2[k]['r2'],
        'v2_ρ':          v2[k]['spearman'],
        'nested_ρ':      nested_metrics[k]['spearman'],
        'Δ ρ':           nested_metrics[k]['spearman'] - v2[k]['spearman'],
    })
comparison = pd.DataFrame(rows)
print('Nested CV vs v2 (leaky) — negative deltas confirm leakage was small:'
display(comparison.round(3))
```

Nested CV vs v2 (leaky) — negative deltas confirm leakage was small:

	Model_Target	v2_R²	nested_R²	Δ R²	v2_ρ	nested_ρ	Δ ρ
0	RF-IMDb	0.544	0.492	-0.052	0.683	0.643	-0.040
1	RF-WOM	0.207	0.174	-0.033	0.415	0.377	-0.038
2	GBM-IMDb	0.476	0.369	-0.107	0.712	0.609	-0.103
3	GBM-WOM	0.136	0.079	-0.057	0.364	0.261	-0.103
4	PLS-IMDb	0.472	0.427	-0.045	0.680	0.649	-0.031
5	PLS-WOM	0.190	0.190	-0.000	0.385	0.385	0.000

Visualisation

In [ ]:
```
fig, axes = plt.subplots(2, 3, figsize=(13, 9))
configs = [
    ('Random Forest — IMDb', y_imdb_all, rf_imdb_pred,  rf_imdb_m),
    ('GBM — IMDb',           y_imdb_all, gbm_imdb_pred, gbm_imdb_m),
    ('PLS — IMDb',           y_imdb_all, pls_imdb_pred, pls_imdb_m),
    ('Random Forest — WOM',  y_wom_all,  rf_wom_pred,   rf_wom_m),
    ('GBM — WOM',            y_wom_all,  gbm_wom_pred,  gbm_wom_m),
    ('PLS — WOM',            y_wom_all,  pls_wom_pred,  pls_wom_m),
]
for ax, (name, y_t, y_p, m) in zip(axes.flat, configs):
    ax.scatter(y_t, y_p, alpha=0.6, s=40, edgecolor='k', linewidth=0.5)
    lo, hi = min(y_t.min(), y_p.min()), max(y_t.max(), y_p.max())
    ax.plot([lo, hi], [lo, hi], 'r--', alpha=0.5)
    ax.set_xlabel('Actual'); ax.set_ylabel('Predicted')
    ax.set_title(f'{name}\nR²={m["r2"]:.2f}, ρ={m["spearman"]:.2f}, MAE={m["
    ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



In [ ]:

```
out = {
    'task': 'nested_cv_top3_regressors',
    'cv_strategy': 'outer 5-fold + inner 5-fold (no hyperparameter leakage)'
    'feature_count': len(feature_cols),
    'metrics': {
        'rf_imdb':  rf_imdb_m,  'rf_wom':  rf_wom_m,
        'gbm_imdb': gbm_imdb_m, 'gbm_wom': gbm_wom_m,
        'pls_imdb': pls_imdb_m, 'pls_wom': pls_wom_m,
    },
    'best_params_per_outer_fold': {
        'rf_imdb':  rf_imdb_params,  'rf_wom':  rf_wom_params,
        'gbm_imdb': gbm_imdb_params, 'gbm_wom': gbm_wom_params,
        'pls_imdb': pls_imdb_params, 'pls_wom': pls_wom_params,
    },
}
out_path = RUN_DIR / 'results_nested_cv_finals.json'
with open(out_path, 'w') as f: json.dump(out, f, indent=2, default=str)
print(f'Saved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v3/results_nested_cv_finals.json
```

09

NOTEBOOK

Permutation test and stacking ensemble

09_stacking.pdf

Stacking Ensemble

Purpose: combine predictions from multiple base learners into a meta-learner.

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)

from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v3'
RUN_DIR.mkdir(parents=True, exist_ok=True)
print('Data dir:', DATA_DIR)
print('Run dir: ', RUN_DIR)
```

```
Mounted at /content/drive
Data dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6
Run dir:  /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movi
e_success_v6/colab_runs_v3
```

In [ ]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, StratifiedKFold, GridSearchCV,
    cross_val_predict, train_test_split, cross_val_score,
)
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
    accuracy_score, f1_score, roc_auc_score,
    confusion_matrix, classification_report, ConfusionMatrixDisplay,
)
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

2 · Load + prepare data

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')

LEAKAGE = ['budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
           'imdb_rating', 'wom_multiplier', 'wom_multiplier_log']
real_mov = real_mov.drop(columns=LEAKAGE, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE, errors='ignore')

META_KEEP = [
    'movie_id', 'targeted_emotion', 'clip_duration_s',
    'cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
    'silence_ratio', 'music_presence', 'dialogue_density',
    'face_screen_time_ratio', 'lead_screen_time_ratio',
    'release_year', 'genre_primary', 'genre_secondary',
    'country_of_origin', 'budget_categorical',
    'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
]
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]
real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left'); real_mo
syn_mov  = syn_mov.merge(syn_meta_sub, on='movie_id', how='left'); syn_mov['
df_all = pd.concat([real_mov, syn_mov], ignore_index=True)
print(f'Combined: {len(df_all)} movies × {len(df_all.columns)} cols')
```

```
Combined: 50 movies × 343 cols
```

In [ ]:
```
DROP = {'movie_id', 'condition', 'n_participants',
        'imdb_rating', 'wom_multiplier', 'wom_multiplier_log',
        'is_synthetic'}
df_feat = df_all.copy()

ORD_MAP = {'low': 1, 'moderate': 2, 'high': 3}
ORD_COLS = ['cut_count', 'brightness', 'motion_intensity', 'audio_loudness',
            'silence_ratio', 'music_presence', 'dialogue_density',
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'budget_categorical']
for c in ORD_COLS:
    if c in df_feat.columns: df_feat[c] = df_feat[c].map(ORD_MAP)

OH_COLS = ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country_
OH_COLS = [c for c in OH_COLS if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH_COLS, prefix_sep='_',
                         dummy_na=False, dtype=int)

feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
synth_mask_all = df_feat['is_synthetic'].values.astype(bool)

X_all = X_all.drop(columns=X_all.columns[X_all.isna().all()].tolist())
X_all = X_all.drop(columns=X_all.columns[X_all.std() == 0].tolist())
```

```
feature_cols = list(X_all.columns)
print(f'Final feature matrix: {X_all.shape}')
```

```
Final feature matrix: (50, 360)
```

Building the stacking ensemble

In [ ]:

```
from sklearn.ensemble import (
    StackingRegressor, RandomForestRegressor, HistGradientBoostingRegressor
)
from sklearn.linear_model import Ridge, ElasticNet
from sklearn.cross_decomposition import PLSRegression

def make_stacking():
    base = [
        ('ridge',  Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler()),
            ('model', Ridge(alpha=100.0, random_state=42)),
        ])),
        ('enet',   Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler()),
            ('model', ElasticNet(alpha=1.0, l1_ratio=0.05,
                                 max_iter=10000, random_state=42)),
        ])),
        ('rf',     Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('model', RandomForestRegressor(n_estimators=500, max_depth=7,
                                            max_features=0.3,
                                            random_state=42, n_jobs=-1)),
        ])),
        ('gbm',    Pipeline([
            ('model', HistGradientBoostingRegressor(
                learning_rate=0.05, max_iter=200, max_depth=3,
                l2_regularization=1.0, random_state=42)),
        ])),
        ('pls',    Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler()),
            ('model', PLSRegression(n_components=2, scale=False)),
        ])),
    ]
    return StackingRegressor(
        estimators=base,
        final_estimator=Ridge(alpha=1.0, random_state=42),
        cv=5,        # internal CV for out-of-fold base predictions
        n_jobs=-1,
        passthrough=False,
    )
```

Train/test split (80/20)

In [ ]:
```
# 80/20 random split (regression target — can't stratify on continuous,
# but with seed=42 it's reproducible)
X_train, X_test, yi_train, yi_test = train_test_split(
    X_all, y_imdb_all, test_size=0.2, random_state=42)
_,       _,       yw_train, yw_test = train_test_split(
    X_all, y_wom_all,  test_size=0.2, random_state=42)
print(f'Train: {len(X_train)}, Test: {len(X_test)}')
```

```
Train: 40, Test: 10
```

Fit and evaluate stacking on IMDb

In [ ]:
```
def regression_metrics(y_true, y_pred):
    return {
        'r2':       r2_score(y_true, y_pred),
        'mae':      mean_absolute_error(y_true, y_pred),
        'rmse':     np.sqrt(mean_squared_error(y_true, y_pred)),
        'spearman': spearmanr(y_true, y_pred).correlation,
        'pearson':  pearsonr(y_true, y_pred)[0],
    }

stack_imdb = make_stacking()
stack_imdb.fit(X_train, yi_train)
y_pred_train_imdb = stack_imdb.predict(X_train)
y_pred_test_imdb  = stack_imdb.predict(X_test)

m_train_imdb = regression_metrics(yi_train, y_pred_train_imdb)
m_test_imdb  = regression_metrics(yi_test,  y_pred_test_imdb)
print('IMDb prediction:')
print(f'  Train: R²={m_train_imdb["r2"]:.3f}, ρ={m_train_imdb["spearman"]:.3
print(f'  Test : R²={m_test_imdb["r2"]:.3f},  ρ={m_test_imdb["spearman"]:.3f
```

```
IMDb prediction:
  Train: R²=0.873, ρ=0.983
  Test : R²=0.631,  ρ=0.742, MAE=0.262
```

Fit and evaluate stacking on WOM

In [ ]:
```
stack_wom = make_stacking()
stack_wom.fit(X_train, yw_train)
y_pred_train_wom = stack_wom.predict(X_train)
y_pred_test_wom  = stack_wom.predict(X_test)

m_train_wom = regression_metrics(yw_train, y_pred_train_wom)
m_test_wom  = regression_metrics(yw_test,  y_pred_test_wom)
print('WOM-log prediction:')
print(f'  Train: R²={m_train_wom["r2"]:.3f}, ρ={m_train_wom["spearman"]:.3f}
print(f'  Test : R²={m_test_wom["r2"]:.3f},  ρ={m_test_wom["spearman"]:.3f},
```

```
WOM-log prediction:
  Train: R²=0.719, ρ=0.981
  Test : R²=0.018,  ρ=0.079, MAE=0.438
```

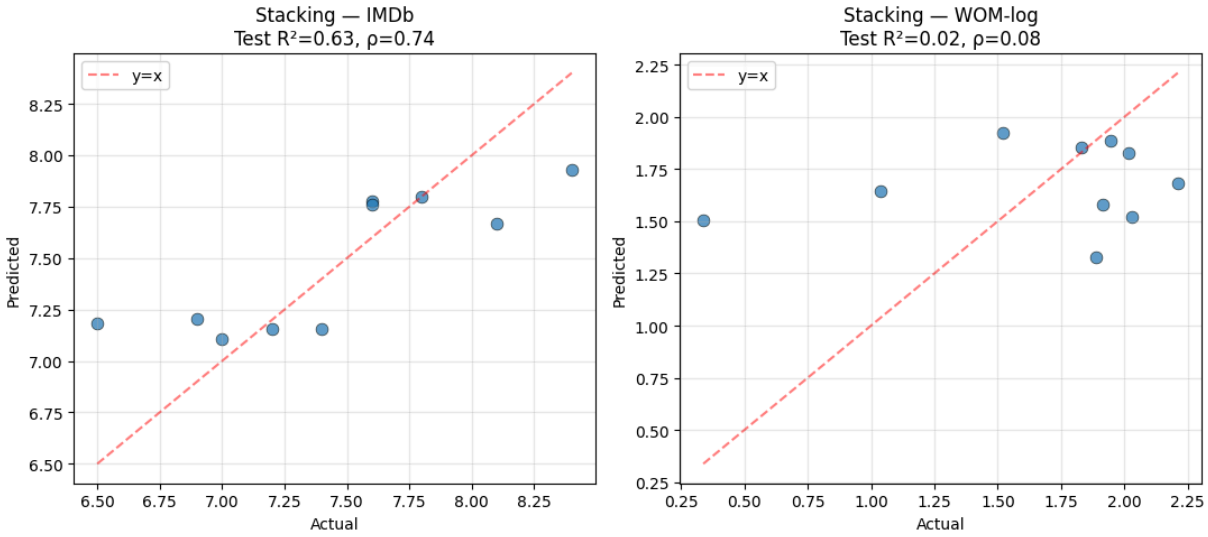
Final-estimator coefficients (which base learners contributed)

In [ ]:
```
# The Ridge meta-learner coefficients tells which base learner contributed
# most to the stacked prediction.
base_names = ['ridge', 'enet', 'rf', 'gbm', 'pls']

print('Meta-learner coefficients (relative weight of each base learner):\n')
print(f'{"base":>6s}  {"IMDb":>10s}  {"WOM":>10s}')
print('-' * 35)
for i, name in enumerate(base_names):
    print(f'  {name:>4s}  {stack_imdb.final_estimator_.coef_[i]:>10.3f}  '
          f'{stack_wom.final_estimator_.coef_[i]:>10.3f}')

print(f'\nMeta intercept: IMDb={stack_imdb.final_estimator_.intercept_:.3f},
      f'WOM={stack_wom.final_estimator_.intercept_:.3f}')
```

```
Meta-learner coefficients (relative weight of each base learner):

  base        IMDb         WOM
-----------------------------------
  ridge       0.027       0.547
  enet        0.055      -0.103
    rf        0.777       0.365
   gbm       -0.088      -0.185
   pls        0.079      -0.030

Meta intercept: IMDb=1.142, WOM=0.690
```

Visualisation

In [ ]:
```
fig, axes = plt.subplots(1, 2, figsize=(11, 5))
for ax, (y_t, y_p, m, label) in zip(
        axes,
        [(yi_test, y_pred_test_imdb, m_test_imdb, 'IMDb'),
         (yw_test, y_pred_test_wom,  m_test_wom,  'WOM-log')]):
    ax.scatter(y_t, y_p, alpha=0.7, s=60, edgecolor='k', linewidth=0.5)
    lo, hi = min(y_t.min(), y_p.min()), max(y_t.max(), y_p.max())
    ax.plot([lo, hi], [lo, hi], 'r--', alpha=0.5, label='y=x')
    ax.set_xlabel('Actual'); ax.set_ylabel('Predicted')
    ax.set_title(f'Stacking — {label}\nTest R²={m["r2"]:.2f}, ρ={m["spearmar
    ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



In [ ]:
```
out = {
    'task': 'stacking_regression_train_test',
    'train_size': len(X_train),
    'test_size': len(X_test),
    'feature_count': len(feature_cols),
    'imdb': {
        'train': m_train_imdb,
        'test':  m_test_imdb,
        'meta_coefs': dict(zip(base_names,
                               stack_imdb.final_estimator_.coef_.tolist())),
        'meta_intercept': float(stack_imdb.final_estimator_.intercept_),
    },
    'wom': {
        'train': m_train_wom,
        'test':  m_test_wom,
        'meta_coefs': dict(zip(base_names,
                               stack_wom.final_estimator_.coef_.tolist())),
        'meta_intercept': float(stack_wom.final_estimator_.intercept_),
    },
}
out_path = RUN_DIR / 'results_stacking.json'
with open(out_path, 'w') as f: json.dump(out, f, indent=2, default=str)
print(f'Saved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v3/results_stacking.json
```

10

NOTEBOOK

Robust classification (top-25%, top-30%)

10_robust_classification.pdf

Robust Classification

Aims to solve the simplicity issue of initial median split.. And provide more business context more applicable results that better fit the target, problem and industry

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)

from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v3'
RUN_DIR.mkdir(parents=True, exist_ok=True)
print('Run dir:', RUN_DIR)
```

```
Mounted at /content/drive
Run dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie
_success_v6/colab_runs_v3
```

In [ ]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, StratifiedKFold, GridSearchCV,
    cross_val_predict, train_test_split,
)
from sklearn.metrics import (
    accuracy_score, f1_score, roc_auc_score,
    confusion_matrix, classification_report,
    precision_score, recall_score,
)
from sklearn.utils import shuffle as sk_shuffle

pd.set_option('display.max_columns', 80)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

In [ ]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')

LEAKAGE = ['budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
           'imdb_rating', 'wom_multiplier', 'wom_multiplier_log']
real_mov = real_mov.drop(columns=LEAKAGE, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE, errors='ignore')

META_KEEP = ['movie_id', 'targeted_emotion', 'clip_duration_s',
             'cut_count', 'brightness', 'motion_intensity', 'audio_loudness'
             'silence_ratio', 'music_presence', 'dialogue_density',
             'face_screen_time_ratio', 'lead_screen_time_ratio',
             'release_year', 'genre_primary', 'genre_secondary',
             'country_of_origin', 'budget_categorical',
             'imdb_rating', 'wom_multiplier', 'wom_multiplier_log']
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]
real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left'); real_mo
syn_mov  = syn_mov.merge(syn_meta_sub, on='movie_id', how='left');  syn_mov[
df_all = pd.concat([real_mov, syn_mov], ignore_index=True)

DROP = {'movie_id','condition','n_participants',
        'imdb_rating','wom_multiplier','wom_multiplier_log','is_synthetic'}
df_feat = df_all.copy()
ORD = {'low':1,'moderate':2,'high':3}
for c in ['cut_count','brightness','motion_intensity','audio_loudness',
          'silence_ratio','music_presence','dialogue_density',
          'face_screen_time_ratio','lead_screen_time_ratio','budget_categori
    if c in df_feat.columns: df_feat[c] = df_feat[c].map(ORD)
OH = [c for c in ['targeted_emotion','genre_primary','genre_secondary','cour
      if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH, prefix_sep='_', dummy_na=False

feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
y_imdb_all = df_feat['imdb_rating'].astype(float)
y_wom_all  = df_feat['wom_multiplier_log'].astype(float)
synth_mask = df_feat['is_synthetic'].values.astype(bool)
X_all = X_all.drop(columns=X_all.columns[X_all.isna().all()].tolist())
X_all = X_all.drop(columns=X_all.columns[X_all.std()==0].tolist())
feature_cols = list(X_all.columns)

# Composite score
z_imdb = (y_imdb_all - y_imdb_all.mean()) / y_imdb_all.std()
z_wom  = (y_wom_all  - y_wom_all.mean())  / y_wom_all.std()
composite = (z_imdb + z_wom) / 2
print(f'X_all: {X_all.shape}, composite range: [{composite.min():.2f}, {comp
```

```
X_all: (50, 360), composite range: [-1.70, 1.89]
```

Classifier helpers

In [ ]:

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingCla
from sklearn.cross_decomposition import PLSRegression
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.gaussian_process.kernels import ConstantKernel, RBF
from sklearn.base import BaseEstimator, ClassifierMixin

class PLSDA(BaseEstimator, ClassifierMixin):
    def __init__(self, n_components=2):
        self.n_components = n_components
    def fit(self, X, y):
        self.classes_ = np.unique(y)
        self.pls = PLSRegression(n_components=self.n_components, scale=False
        self.pls.fit(X, y.astype(float))
        return self
    def predict_proba(self, X):
        s = np.clip(self.pls.predict(X).flatten(), 0, 1)
        return np.column_stack([1-s, s])
    def predict(self, X):
        return (self.predict_proba(X)[:,1] >= 0.5).astype(int)

def make_enet(C=0.023, l1_ratio=0.1):
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
        ('m',   LogisticRegression(penalty='elasticnet', solver='saga',
                                   C=C, l1_ratio=l1_ratio, class_weight='bal
                                   max_iter=10000, random_state=42))])

def make_l2(C=0.001):
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
        ('m',   LogisticRegression(penalty='l2', C=C, class_weight='balanced
                                   max_iter=10000, random_state=42))])

def make_rf(max_depth=3, max_features=0.3, n_estimators=500):
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('m',   RandomForestClassifier(n_estimators=n_estimators, max_depth=
                                       max_features=max_features, class_weig
                                       random_state=42, n_jobs=-1))])

def make_gbm(learning_rate=0.05, max_iter=200, max_depth=3, l2_regularizatic
    return Pipeline([
        ('m', HistGradientBoostingClassifier(learning_rate=learning_rate,
                                             max_iter=max_iter, max_depth=m
                                             l2_regularization=l2_regulariz
                                             class_weight='balanced',
                                             random_state=42))])

def make_plsda(n_components=1):
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
```

```
        ('m',   PLSDA(n_components=n_components))])

def make_gpc():
    kernel = ConstantKernel(1.0) * RBF(length_scale=1.0, length_scale_bounds
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
        ('m',   GaussianProcessClassifier(kernel=kernel, n_restarts_optimize
                                          random_state=42))])

CLASSIFIERS = {
    'LogReg-ElasticNet': make_enet,
    'LogReg-L2':         make_l2,
    'Random Forest':     make_rf,
    'Gradient Boosting': make_gbm,
    'PLS-DA':            make_plsda,
    'GP Classifier':     make_gpc,
}
print(f'Defined {len(CLASSIFIERS)} classifier factories')
```

```
Defined 6 classifier factories
```

Permutation test

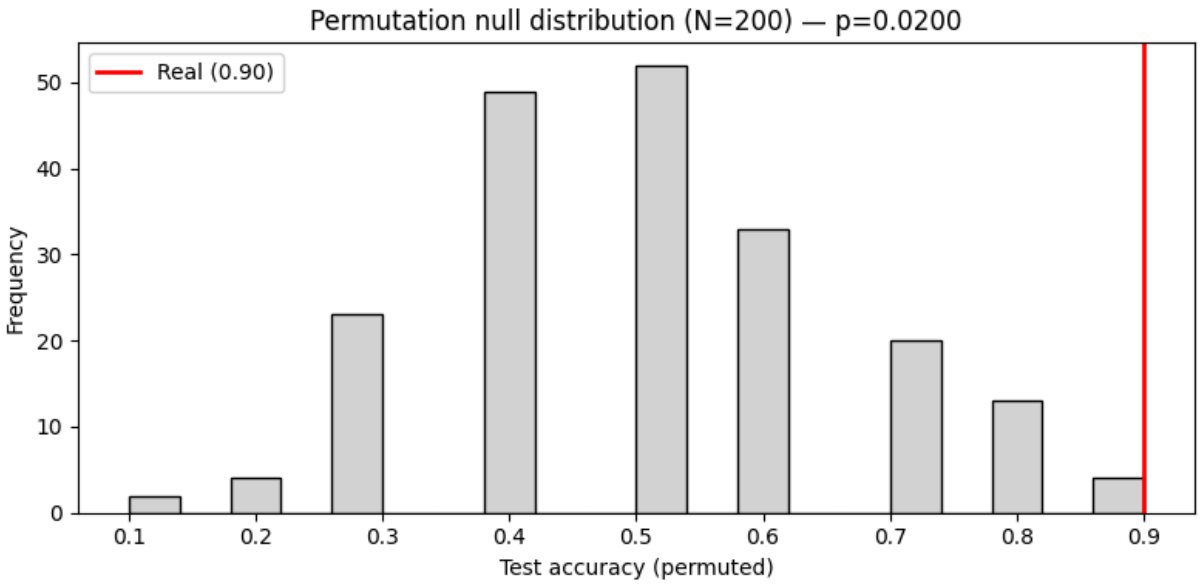
Question is whether 90% accuracy is reliable or not. Most likely it isn't.

Shuffle labels 200 times, redo the train/test split, retrain LogReg-ElasticNet (the best of initial classifiers), record accuracy. Compute p-value as the fraction of permutations with accuracy ≥ real.

In [ ]:
```
y_class_50 = (composite >= composite.median()).astype(int)
print(f'Class balance (50/50 split): {y_class_50.value_counts().to_dict()}')

# Real performance
X_train, X_test, y_train, y_test = train_test_split(
    X_all, y_class_50, test_size=0.2, random_state=42, stratify=y_class_50)
pipe = make_enet(C=0.023, l1_ratio=0.1)
pipe.fit(X_train, y_train)
real_acc = accuracy_score(y_test, pipe.predict(X_test))
print(f'\nReal LogReg-ElasticNet test accuracy: {real_acc:.3f}')

# Permutation test
N_PERMS = 200
perm_accs = []
print(f'\nRunning {N_PERMS} permutations...')
for i in range(N_PERMS):
    y_perm = sk_shuffle(y_class_50.values, random_state=i)
    Xtr, Xte, ytr, yte = train_test_split(
        X_all, y_perm, test_size=0.2, random_state=42, stratify=y_perm)
    p = make_enet(C=0.023, l1_ratio=0.1)
    p.fit(Xtr, ytr)
    perm_accs.append(accuracy_score(yte, p.predict(Xte)))
    if (i+1) % 50 == 0: print(f'  {i+1}/{N_PERMS} done')
```

```python

perm_accs = np.array(perm_accs)
p_value = np.mean(perm_accs >= real_acc)
print(f'\nPermutation accuracies: mean={perm_accs.mean():.3f}, '
      f'std={perm_accs.std():.3f}, max={perm_accs.max():.3f}')
print(f'Real accuracy: {real_acc:.3f}')
print(f'p-value (P[perm_acc ≥ real_acc]): {p_value:.4f}')
print('  → significant at p<0.05' if p_value < 0.05 else '  → NOT significar

# Visualise
fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(perm_accs, bins=20, color='lightgray', edgecolor='black')
ax.axvline(real_acc, color='red', linewidth=2, label=f'Real ({real_acc:.2f})
ax.set_xlabel('Test accuracy (permuted)')
ax.set_ylabel('Frequency')
ax.set_title(f'Permutation null distribution (N={N_PERMS}) — p={p_value:.4f}
ax.legend()
plt.tight_layout(); plt.show()
test_a_results = {'real_acc': float(real_acc),
                  'perm_mean': float(perm_accs.mean()),
                  'perm_std': float(perm_accs.std()),
                  'perm_max': float(perm_accs.max()),
                  'p_value': float(p_value),
                  'n_permutations': int(N_PERMS)}
```

```
Class balance (50/50 split): {1: 25, 0: 25}

Real LogReg-ElasticNet test accuracy: 0.900

Running 200 permutations...
  50/200 done
  100/200 done
  150/200 done
  200/200 done

Permutation accuracies: mean=0.507, std=0.158, max=0.900
Real accuracy: 0.900
p-value (P[perm_acc ≥ real_acc]): 0.0200
  → significant at p<0.05
```



Top-25% "hit" classification

Aims to identify outlier successes which are important in terms of business considerations

In [ ]:
```
hit_threshold = composite.quantile(0.75)
y_hit = (composite >= hit_threshold).astype(int)
print(f'Hit threshold: {hit_threshold:.3f}')
print(f'Class balance: {y_hit.value_counts().to_dict()}  '
      f'(hit fraction: {y_hit.mean():.0%})')

# 80/20 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X_all, y_hit, test_size=0.2, random_state=42, stratify=y_hit)
print(f'Train: {len(X_train)} ({y_train.value_counts().to_dict()})')
print(f'Test:  {len(X_test)} ({y_test.value_counts().to_dict()})')

baseline_acc = max(y_test.mean(), 1 - y_test.mean())
print(f'\nMajority-class baseline accuracy: {baseline_acc:.3f}  '
      '(predict everything as the dominant class)')
```

```
Hit threshold: 0.601
Class balance: {0: 37, 1: 13}  (hit fraction: 26%)
Train: 40 ({0: 30, 1: 10})
Test:  10 ({0: 7, 1: 3})

Majority-class baseline accuracy: 0.700  (predict everything as the dominant
class)
```

In [ ]:
```
# Run all 6 classifiers, tune the regularisation C / params on training set
def evaluate(pipe, X_tr, y_tr, X_te, y_te):
    pipe.fit(X_tr, y_tr)
    y_pred = pipe.predict(X_te)
    try:
        y_proba = pipe.predict_proba(X_te)[:, 1]
```

```
    except Exception:
        try:    y_proba = pipe.decision_function(X_te)
        except: y_proba = y_pred.astype(float)
    return {
        'accuracy':  accuracy_score(y_te, y_pred),
        'f1':        f1_score(y_te, y_pred, zero_division=0),
        'precision': precision_score(y_te, y_pred, zero_division=0),
        'recall':    recall_score(y_te, y_pred, zero_division=0),
        'roc_auc':   (roc_auc_score(y_te, y_proba)
                      if len(np.unique(y_te)) > 1 else float('nan')),
    }

test_b_results = {}
for name, factory in CLASSIFIERS.items():
    pipe = factory()
    metrics = evaluate(pipe, X_train, y_train, X_test, y_test)
    test_b_results[name] = metrics
    print(f'  {name:22s}  Acc={metrics["accuracy"]:.2f}  '
          f'F1={metrics["f1"]:.2f}  AUC={metrics["roc_auc"]:.2f}  '
          f'Prec={metrics["precision"]:.2f}  Rec={metrics["recall"]:.2f}')

# Best by ROC-AUC
best_b = max(test_b_results, key=lambda k: test_b_results[k]['roc_auc']
             if not np.isnan(test_b_results[k]['roc_auc']) else -1)
print(f'\nBest classifier (Top-25% hit, by ROC-AUC): {best_b}')
print(f'  → AUC={test_b_results[best_b]["roc_auc"]:.3f}, '
      f'F1={test_b_results[best_b]["f1"]:.3f}, '
      f'Recall={test_b_results[best_b]["recall"]:.3f}')

comparison_b = pd.DataFrame(test_b_results).T.sort_values('roc_auc', ascendi
display(comparison_b.round(3))
```

```
  LogReg-ElasticNet       Acc=0.80  F1=0.75  AUC=1.00  Prec=0.60  Rec=1.00
  LogReg-L2               Acc=0.70  F1=0.67  AUC=0.95  Prec=0.50  Rec=1.00
  Random Forest           Acc=0.70  F1=0.00  AUC=0.86  Prec=0.00  Rec=0.00
  Gradient Boosting       Acc=0.60  F1=0.50  AUC=0.81  Prec=0.40  Rec=0.67
  PLS-DA                  Acc=0.90  F1=0.86  AUC=0.90  Prec=0.75  Rec=1.00
  GP Classifier           Acc=0.70  F1=0.00  AUC=0.95  Prec=0.00  Rec=0.00

Best classifier (Top-25% hit, by ROC-AUC): LogReg-ElasticNet
  → AUC=1.000, F1=0.750, Recall=1.000
```

	accuracy	f1	precision	recall	roc_auc
LogReg-ElasticNet	0.8	0.750	0.60	1.000	1.000
LogReg-L2	0.7	0.667	0.50	1.000	0.952
GP Classifier	0.7	0.000	0.00	0.000	0.952
PLS-DA	0.9	0.857	0.75	1.000	0.905
Random Forest	0.7	0.000	0.00	0.000	0.857
Gradient Boosting	0.6	0.500	0.40	0.667	0.810

Real-only LOO classification

10 real movies, threshold composite at the median of those 10 (5/5 split), leave-one-out cross-validation. Each fold trains on 9 movies, predicts the held-out one. Aggregate predictions across all 10 LOO folds and compute accuracy.

In [ ]:
```
real_mask = ~synth_mask
X_real = X_all[real_mask].reset_index(drop=True)
composite_real = composite[real_mask].reset_index(drop=True)
real_threshold = composite_real.median()
y_real = (composite_real >= real_threshold).astype(int)
print(f'Real-only threshold (median of 10): {real_threshold:.3f}')
print(f'Class balance: {y_real.value_counts().to_dict()}')

test_c_results = {}
for name, factory in CLASSIFIERS.items():
    pipe = factory()
    y_pred = cross_val_predict(pipe, X_real, y_real, cv=LeaveOneOut(), n_job
    try:
        y_proba = cross_val_predict(pipe, X_real, y_real,
                                    cv=LeaveOneOut(), method='predict_proba'
                                    n_jobs=-1)[:, 1]
    except Exception:
        y_proba = y_pred.astype(float)
    metrics = {
        'accuracy':  accuracy_score(y_real, y_pred),
        'f1':        f1_score(y_real, y_pred, zero_division=0),
        'precision': precision_score(y_real, y_pred, zero_division=0),
        'recall':    recall_score(y_real, y_pred, zero_division=0),
        'roc_auc':   (roc_auc_score(y_real, y_proba)
                      if len(np.unique(y_real)) > 1 else float('nan')),
    }
    test_c_results[name] = metrics
    print(f'  {name:22s}  Acc={metrics["accuracy"]:.2f}  '
          f'F1={metrics["f1"]:.2f}  AUC={metrics["roc_auc"]:.2f}')

print('\nNote: with n=10 LOO, accuracy is in 10% steps. 50% = chance, '
      '70%+ = meaningful signal.')

comparison_c = pd.DataFrame(test_c_results).T.sort_values('roc_auc', ascendi
display(comparison_c.round(3))
```

```
Real-only threshold (median of 10): 0.338
Class balance: {1: 5, 0: 5}
  LogReg-ElasticNet       Acc=0.60  F1=0.71  AUC=0.68
  LogReg-L2               Acc=0.30  F1=0.22  AUC=0.24
  Random Forest           Acc=0.30  F1=0.22  AUC=0.16
  Gradient Boosting       Acc=0.50  F1=0.00  AUC=0.10
  PLS-DA                  Acc=0.30  F1=0.22  AUC=0.24
  GP Classifier           Acc=0.40  F1=0.00  AUC=0.50

Note: with n=10 LOO, accuracy is in 10% steps. 50% = chance, 70%+ = meaningf
ul signal.
```

	accuracy	f1	precision	recall	roc_auc
LogReg-ElasticNet	0.6	0.714	0.556	1.0	0.68
GP Classifier	0.4	0.000	0.000	0.0	0.50
LogReg-L2	0.3	0.222	0.250	0.2	0.24
PLS-DA	0.3	0.222	0.250	0.2	0.24
Random Forest	0.3	0.222	0.250	0.2	0.16
Gradient Boosting	0.5	0.000	0.000	0.0	0.10

Train on synthetic, test on real

In [ ]:
```
X_syn = X_all[synth_mask].reset_index(drop=True)
y_syn = y_class_50[synth_mask].reset_index(drop=True)
X_real_test = X_all[~synth_mask].reset_index(drop=True)
y_real_test = y_class_50[~synth_mask].reset_index(drop=True)
print(f'Train (synthetic): {len(X_syn)}, class balance: {y_syn.value_counts(
print(f'Test  (real):      {len(X_real_test)}, class balance: {y_real_test.v

test_d_results = {}
for name, factory in CLASSIFIERS.items():
    pipe = factory()
    pipe.fit(X_syn, y_syn)
    y_pred = pipe.predict(X_real_test)
    try:    y_proba = pipe.predict_proba(X_real_test)[:, 1]
    except Exception:
        try:    y_proba = pipe.decision_function(X_real_test)
        except: y_proba = y_pred.astype(float)
    metrics = {
        'accuracy':  accuracy_score(y_real_test, y_pred),
        'f1':        f1_score(y_real_test, y_pred, zero_division=0),
        'precision': precision_score(y_real_test, y_pred, zero_division=0),
        'recall':    recall_score(y_real_test, y_pred, zero_division=0),
        'roc_auc':   (roc_auc_score(y_real_test, y_proba)
                      if len(np.unique(y_real_test)) > 1 else float('nan')),
    }
    test_d_results[name] = metrics
    print(f'  {name:22s}  Acc={metrics["accuracy"]:.2f}  '
          f'F1={metrics["f1"]:.2f}  AUC={metrics["roc_auc"]:.2f}')

comparison_d = pd.DataFrame(test_d_results).T.sort_values('roc_auc', ascendi
display(comparison_d.round(3))
```

```
Train (synthetic): 40, class balance: {0: 21, 1: 19}
Test  (real):      10, class balance: {1: 6, 0: 4}
  LogReg-ElasticNet       Acc=0.60  F1=0.60  AUC=0.62
  LogReg-L2               Acc=0.70  F1=0.73  AUC=0.71
  Random Forest           Acc=0.60  F1=0.50  AUC=0.54
  Gradient Boosting       Acc=0.60  F1=0.50  AUC=0.88
  PLS-DA                  Acc=0.80  F1=0.80  AUC=0.73
  GP Classifier           Acc=0.60  F1=0.67  AUC=0.71
```

	accuracy	f1	precision	recall	roc_auc
Gradient Boosting	0.6	0.500	1.000	0.333	0.875
PLS-DA	0.8	0.800	1.000	0.667	0.729
GP Classifier	0.6	0.667	0.667	0.667	0.708
LogReg-L2	0.7	0.727	0.800	0.667	0.708
LogReg-ElasticNet	0.6	0.600	0.750	0.500	0.625
Random Forest	0.6	0.500	1.000	0.333	0.542

In [ ]:
```
out = {
    'task': 'robust_classification_sanity_tests',
    'feature_count': len(feature_cols),
    'test_a_permutation': test_a_results,
    'test_b_top25_hit':   test_b_results,
    'test_c_real_only_loo': test_c_results,
    'test_d_synthetic_train_real_test': test_d_results,
    'best_per_test': {
        'A_permutation_p_value': test_a_results['p_value'],
        'B_top25_best': max(test_b_results,
                            key=lambda k: test_b_results[k].get('roc_auc', 0
                                                               if not np.isnan(test_b_results[k].
        'C_real_only_best': max(test_c_results,
                                key=lambda k: test_c_results[k]['accuracy'])
        'D_synth_to_real_best': max(test_d_results,
                                    key=lambda k: test_d_results[k]['accurac
    },
}
out_path = RUN_DIR / 'results_robust_classification.json'
with open(out_path, 'w') as f: json.dump(out, f, indent=2, default=str)
print(f'Saved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v3/results_robust_classification.json
```

11

NOTEBOOK

Business classification — synth-to-real transfer

11_business_classification.pdf

Classification framing that is closer to business

In [1]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)
from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v3'
RUN_DIR.mkdir(parents=True, exist_ok=True)
```

```
Mounted at /content/drive
```

In [2]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import (
    LeaveOneOut, KFold, StratifiedKFold, GridSearchCV,
    cross_val_predict, train_test_split,
)
from sklearn.metrics import (
    accuracy_score, f1_score, roc_auc_score, precision_score,
    recall_score, average_precision_score, precision_recall_curve,
    confusion_matrix, classification_report,
)
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingCla
from sklearn.cross_decomposition import PLSRegression
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.gaussian_process.kernels import ConstantKernel, RBF
from sklearn.base import BaseEstimator, ClassifierMixin

pd.set_option('display.max_columns', 80)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

In [3]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')

LEAKAGE = ['budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
           'imdb_rating', 'wom_multiplier', 'wom_multiplier_log']
```

```
real_mov = real_mov.drop(columns=LEAKAGE, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE, errors='ignore')

META_KEEP = ['movie_id', 'targeted_emotion', 'clip_duration_s',
             'cut_count', 'brightness', 'motion_intensity', 'audio_loudness'
             'silence_ratio', 'music_presence', 'dialogue_density',
             'face_screen_time_ratio', 'lead_screen_time_ratio',
             'release_year', 'genre_primary', 'genre_secondary',
             'country_of_origin', 'budget_categorical',
             'imdb_rating', 'wom_multiplier', 'wom_multiplier_log']
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]
real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left'); real_mo
syn_mov  = syn_mov.merge(syn_meta_sub, on='movie_id', how='left');  syn_mov[
df_all = pd.concat([real_mov, syn_mov], ignore_index=True)

DROP = {'movie_id','condition','n_participants',
        'imdb_rating','wom_multiplier','wom_multiplier_log','is_synthetic'}
df_feat = df_all.copy()
ORD = {'low':1,'moderate':2,'high':3}
for c in ['cut_count','brightness','motion_intensity','audio_loudness',
          'silence_ratio','music_presence','dialogue_density',
          'face_screen_time_ratio','lead_screen_time_ratio','budget_categori
    if c in df_feat.columns: df_feat[c] = df_feat[c].map(ORD)
OH = [c for c in ['targeted_emotion','genre_primary','genre_secondary','cour
      if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH, prefix_sep='_', dummy_na=False

feature_cols = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[feature_cols].apply(pd.to_numeric, errors='coerce')
X_all = X_all.drop(columns=X_all.columns[X_all.isna().all()].tolist())
X_all = X_all.drop(columns=X_all.columns[X_all.std()==0].tolist())
feature_cols = list(X_all.columns)

y_imdb = df_feat['imdb_rating'].astype(float)
y_wom  = df_feat['wom_multiplier_log'].astype(float)
synth_mask = df_feat['is_synthetic'].values.astype(bool)
print(f'Feature matrix: {X_all.shape}')
```

```
Feature matrix: (50, 360)
```

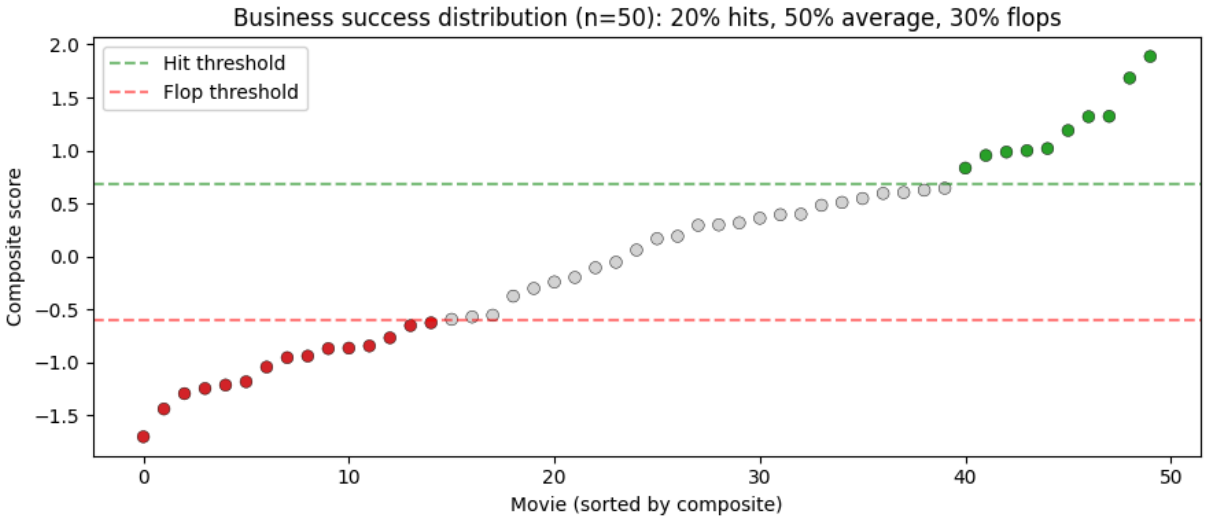
Definition of business success tiers

In [4]:

```
# Composite score
z_imdb = (y_imdb - y_imdb.mean()) / y_imdb.std()
z_wom  = (y_wom  - y_wom.mean())  / y_wom.std()
composite = (z_imdb + z_wom) / 2

# Three-tier categorisation
hit_threshold  = composite.quantile(0.80)   # top 20%
flop_threshold = composite.quantile(0.30)   # bottom 30%

is_hit  = (composite >= hit_threshold).astype(int)
is_flop = (composite <= flop_threshold).astype(int)
```

```
tier = pd.Series(['average'] * len(composite), index=composite.index)
tier[composite >= hit_threshold]  = 'hit'
tier[composite <= flop_threshold] = 'flop'

print('Three-tier business classification:')
print(tier.value_counts())
print(f'\nThresholds:')
print(f'  Hit  (top 20%):  composite ≥ {hit_threshold:.3f}')
print(f'  Flop (bot 30%):  composite ≤ {flop_threshold:.3f}')
print(f'\nClass distributions per binary task:')
print(f'  is_hit:  {is_hit.value_counts().to_dict()}  '
      f'(positive = {is_hit.mean():.0%})')
print(f'  is_flop: {is_flop.value_counts().to_dict()}  '
      f'(positive = {is_flop.mean():.0%})')

# Visualise
fig, ax = plt.subplots(figsize=(9, 4))
colors = ['#d62728' if t == 'flop' else '#2ca02c' if t == 'hit' else 'lightg
          for t in tier]
ax.scatter(range(len(composite)), composite.sort_values().values,
           c=[colors[i] for i in composite.sort_values().index],
           s=40, edgecolor='k', linewidth=0.3)
ax.axhline(hit_threshold, color='green', linestyle='--', alpha=0.5, label='H
ax.axhline(flop_threshold, color='red', linestyle='--', alpha=0.5, label='Fl
ax.set_xlabel('Movie (sorted by composite)')
ax.set_ylabel('Composite score')
ax.set_title('Business success distribution (n=50): 20% hits, 50% average, 3
ax.legend()
plt.tight_layout(); plt.show()
```

```
Three-tier business classification:
average    25
flop       15
hit        10
Name: count, dtype: int64

Thresholds:
  Hit  (top 20%):  composite ≥ 0.681
  Flop (bot 30%):  composite ≤ -0.604

Class distributions per binary task:
  is_hit:  {0: 40, 1: 10}  (positive = 20%)
  is_flop: {0: 35, 1: 15}  (positive = 30%)
```



Classifier helper

In [5]:

```
class PLSDA(BaseEstimator, ClassifierMixin):
    def __init__(self, n_components=2):
        self.n_components = n_components
    def fit(self, X, y):
        self.classes_ = np.unique(y)
        self.pls = PLSRegression(n_components=self.n_components, scale=False
        self.pls.fit(X, y.astype(float))
        return self
    def predict_proba(self, X):
        s = np.clip(self.pls.predict(X).flatten(), 0, 1)
        return np.column_stack([1-s, s])
    def predict(self, X):
        return (self.predict_proba(X)[:,1] >= 0.5).astype(int)

def make_enet(C=0.05, l1_ratio=0.3):
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
        ('m',   LogisticRegression(penalty='elasticnet', solver='saga',
                                   C=C, l1_ratio=l1_ratio, class_weight='bal
                                   max_iter=10000, random_state=42))])
def make_l2(C=0.1):
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
        ('m',   LogisticRegression(penalty='l2', C=C, class_weight='balanced
                                   max_iter=10000, random_state=42))])
def make_rf():
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('m',   RandomForestClassifier(n_estimators=500, max_depth=5,
                                       max_features=0.3, class_weight='bala
                                       random_state=42, n_jobs=-1))])
def make_gbm():
    return Pipeline([
```

```
        ('m', HistGradientBoostingClassifier(learning_rate=0.05, max_iter=30
                                             max_depth=3, l2_regularizatior
                                             class_weight='balanced',
                                             random_state=42))])
def make_plsda(n_components=2):
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
        ('m',   PLSDA(n_components=n_components))])
def make_gpc():
    kernel = ConstantKernel(1.0) * RBF(length_scale=1.0, length_scale_bounds
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
        ('m',   GaussianProcessClassifier(kernel=kernel, n_restarts_optimize
                                          random_state=42))])

CLASSIFIERS = {
    'LogReg-ElasticNet': make_enet,
    'LogReg-L2':         make_l2,
    'Random Forest':     make_rf,
    'Gradient Boosting': make_gbm,
    'PLS-DA':            make_plsda,
    'GP Classifier':     make_gpc,
}
```

Helper for binary classification

In [6]:

```
def get_proba(pipe, X):
    try:    return pipe.predict_proba(X)[:, 1]
    except Exception:
        try: return pipe.decision_function(X)
        except: return pipe.predict(X).astype(float)

def evaluate_binary(pipe, X_tr, y_tr, X_te, y_te):
    pipe.fit(X_tr, y_tr)
    y_pred  = pipe.predict(X_te)
    y_proba = get_proba(pipe, X_te)

    # Operating point: max precision while keeping recall >= 0.8
    precs, recs, thrs = precision_recall_curve(y_te, y_proba)
    valid = recs[:-1] >= 0.8
    if valid.any():
        idx = np.argmax(precs[:-1][valid])
        prec_at_r80 = precs[:-1][valid][idx]
        rec_at_r80  = recs[:-1][valid][idx]
        thr_at_r80  = thrs[valid][idx]
    else:
        prec_at_r80 = float('nan')
        rec_at_r80  = float('nan')
        thr_at_r80  = float('nan')

    return {
        'accuracy':   accuracy_score(y_te, y_pred),
```

```
        'f1':         f1_score(y_te, y_pred, zero_division=0),
        'precision':  precision_score(y_te, y_pred, zero_division=0),
        'recall':     recall_score(y_te, y_pred, zero_division=0),
        'roc_auc':    (roc_auc_score(y_te, y_proba)
                       if len(np.unique(y_te)) > 1 else float('nan')),
        'pr_auc':     (average_precision_score(y_te, y_proba)
                       if len(np.unique(y_te)) > 1 else float('nan')),
        'precision_at_recall80': prec_at_r80,
        'threshold_at_recall80': thr_at_r80,
    }, y_proba
```

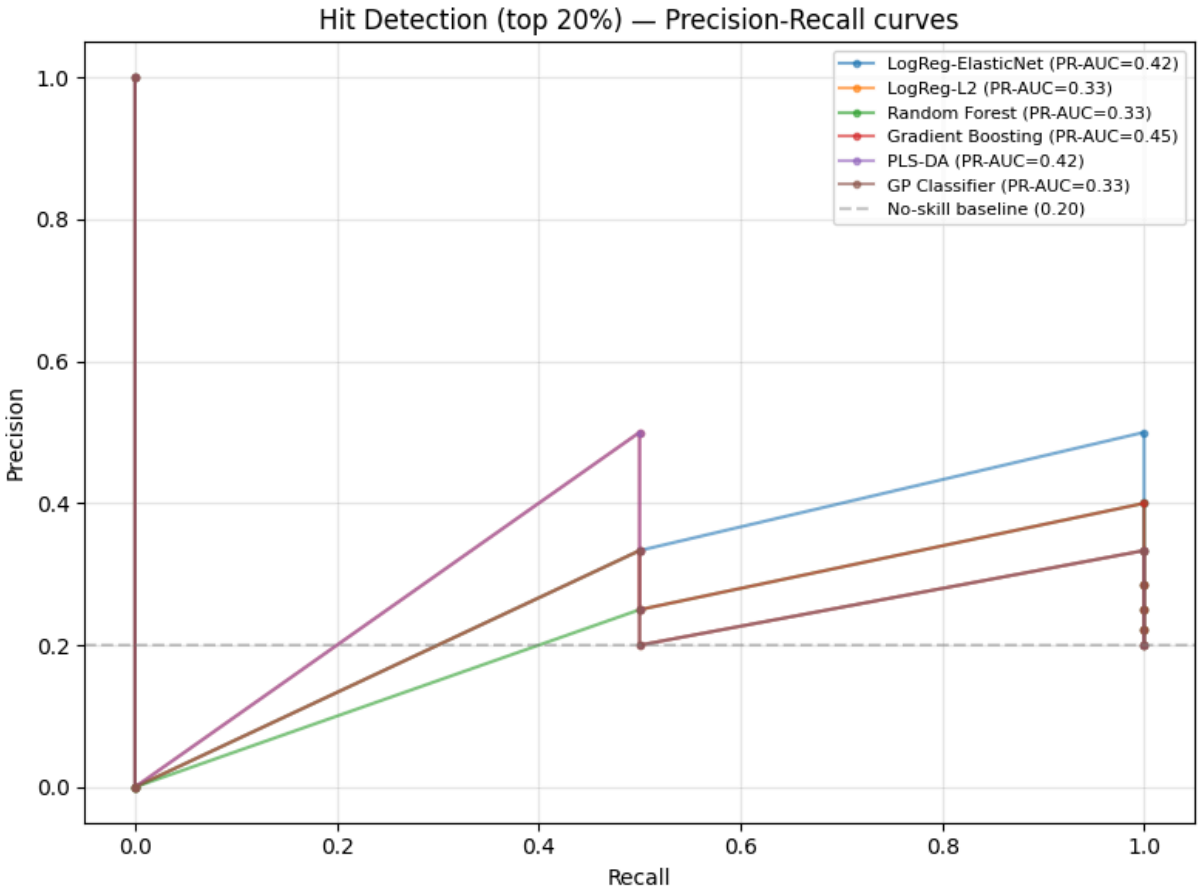
Hit detection (top 20%)

In [7]:
```
# 80/20 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X_all, is_hit, test_size=0.2, random_state=42, stratify=is_hit)
print(f'Hit detection: train {len(X_train)} ({y_train.value_counts().to_dict
      f'test {len(X_test)} ({y_test.value_counts().to_dict()})')

baseline = 1 - y_test.mean()
print(f'Majority-class baseline accuracy: {baseline:.2%} (predicting all=not
print()

hit_results = {}
hit_probas  = {}
for name, factory in CLASSIFIERS.items():
    pipe = factory()
    metrics, proba = evaluate_binary(pipe, X_train, y_train, X_test, y_test)
    hit_results[name] = metrics
    hit_probas[name]  = proba
    print(f'  {name:22s}  AUC={metrics["roc_auc"]:.2f}  PR-AUC={metrics["pr_
          f'F1={metrics["f1"]:.2f}  P@R80={metrics["precision_at_recall80"]:

best_hit = max(hit_results, key=lambda k: hit_results[k]['pr_auc']
               if not np.isnan(hit_results[k]['pr_auc']) else -1)
print(f'\nBest hit detector (by PR-AUC): {best_hit}')
display(pd.DataFrame(hit_results).T.sort_values('pr_auc', ascending=False).r
```

```
Hit detection: train 40 ({0: 32, 1: 8}), test 10 ({0: 8, 1: 2})
Majority-class baseline accuracy: 80.00% (predicting all=not_hit)

  LogReg-ElasticNet       AUC=0.75  PR-AUC=0.42  F1=0.50  P@R80=0.50
  LogReg-L2               AUC=0.62  PR-AUC=0.33  F1=0.00  P@R80=0.33
  Random Forest           AUC=0.62  PR-AUC=0.33  F1=0.00  P@R80=0.40
  Gradient Boosting       AUC=0.75  PR-AUC=0.45  F1=0.33  P@R80=0.40
  PLS-DA                  AUC=0.69  PR-AUC=0.42  F1=0.00  P@R80=0.33
  GP Classifier           AUC=0.62  PR-AUC=0.33  F1=0.00  P@R80=0.33

Best hit detector (by PR-AUC): Gradient Boosting
```

	accuracy	f1	precision	recall	roc_auc	pr_auc	precision_at_recall80	t
Gradient Boosting	0.6	0.333	0.250	0.5	0.750	0.450	0.400	
LogReg-ElasticNet	0.6	0.500	0.333	1.0	0.750	0.417	0.500	
PLS-DA	0.7	0.000	0.000	0.0	0.688	0.417	0.333	
LogReg-L2	0.6	0.000	0.000	0.0	0.625	0.333	0.333	
GP Classifier	0.8	0.000	0.000	0.0	0.625	0.333	0.333	
Random Forest	0.8	0.000	0.000	0.0	0.625	0.325	0.400	

In [8]:
```
# Plot PR curves
fig, ax = plt.subplots(figsize=(8, 6))
for name, proba in hit_probas.items():
    p, r, _ = precision_recall_curve(y_test, proba)
    ap = average_precision_score(y_test, proba)
    ax.plot(r, p, marker='.', alpha=0.6, label=f'{name} (PR-AUC={ap:.2f})')
ax.axhline(y_test.mean(), color='gray', linestyle='--', alpha=0.4,
           label=f'No-skill baseline ({y_test.mean():.2f})')
ax.set_xlabel('Recall')
ax.set_ylabel('Precision')
ax.set_title('Hit Detection (top 20%) — Precision-Recall curves')
ax.legend(fontsize=8)
ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



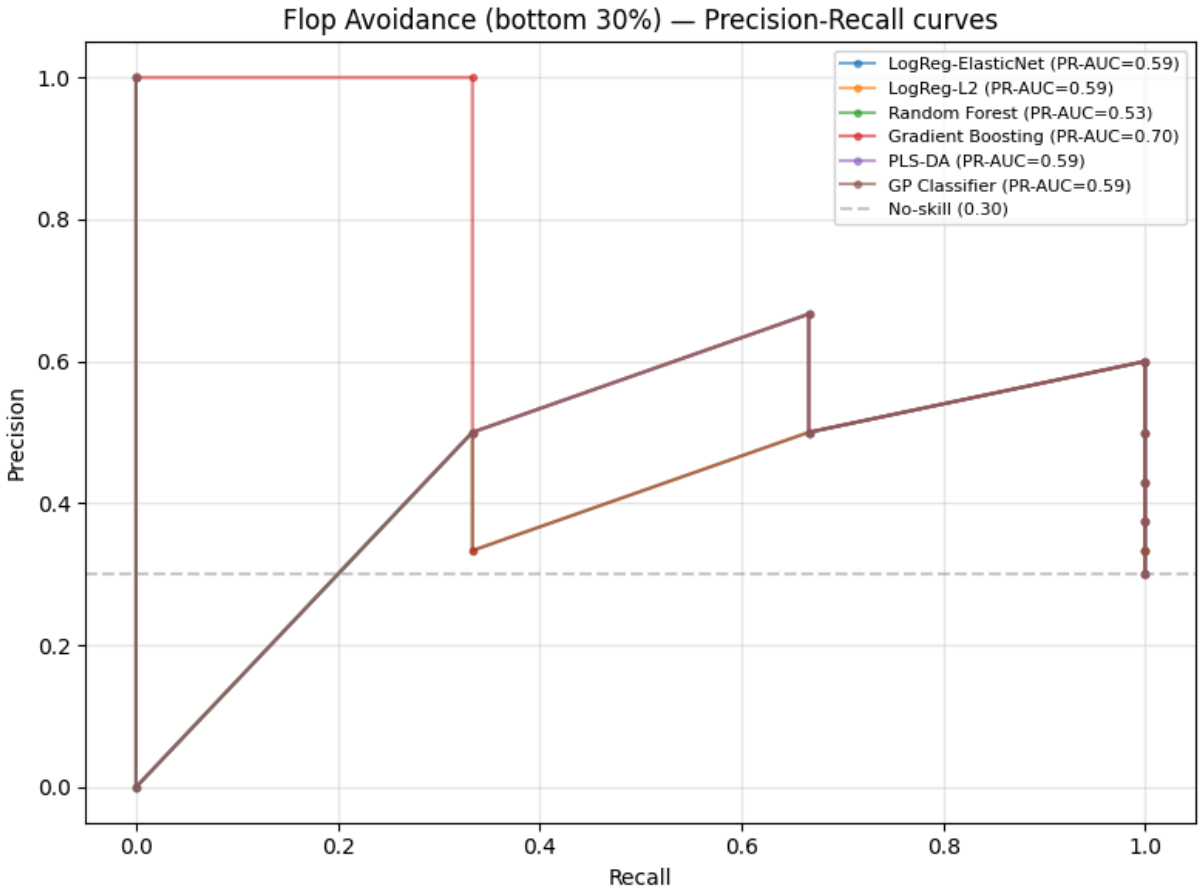
"Flop" avoidance

In [9]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X_all, is_flop, test_size=0.2, random_state=42, stratify=is_flop)
print(f'Flop avoidance: train {len(X_train)} ({y_train.value_counts().to_dic
      f'test {len(X_test)} ({y_test.value_counts().to_dict()})')

baseline = 1 - y_test.mean()
print(f'Majority-class baseline accuracy: {baseline:.2%}')
print()

flop_results = {}
flop_probas  = {}
for name, factory in CLASSIFIERS.items():
    pipe = factory()
    metrics, proba = evaluate_binary(pipe, X_train, y_train, X_test, y_test)
    flop_results[name] = metrics
    flop_probas[name]  = proba
    print(f'  {name:22s}  AUC={metrics["roc_auc"]:.2f}  PR-AUC={metrics["pr_
          f'F1={metrics["f1"]:.2f}  P@R80={metrics["precision_at_recall80"]:

best_flop = max(flop_results, key=lambda k: flop_results[k]['pr_auc']
                if not np.isnan(flop_results[k]['pr_auc']) else -1)
print(f'\nBest flop screener (by PR-AUC): {best_flop}')
display(pd.DataFrame(flop_results).T.sort_values('pr_auc', ascending=False).
```

```
Flop avoidance: train 40 ({0: 28, 1: 12}), test 10 ({0: 7, 1: 3})
Majority-class baseline accuracy: 70.00%

  LogReg-ElasticNet       AUC=0.81  PR-AUC=0.59  F1=0.67  P@R80=0.60
  LogReg-L2               AUC=0.81  PR-AUC=0.59  F1=0.67  P@R80=0.60
  Random Forest           AUC=0.76  PR-AUC=0.53  F1=0.40  P@R80=0.60
  Gradient Boosting       AUC=0.81  PR-AUC=0.70  F1=0.57  P@R80=0.60
  PLS-DA                  AUC=0.81  PR-AUC=0.59  F1=0.40  P@R80=0.60
  GP Classifier           AUC=0.81  PR-AUC=0.59  F1=0.40  P@R80=0.60

Best flop screener (by PR-AUC): Gradient Boosting
```

	accuracy	f1	precision	recall	roc_auc	pr_auc	precision_at_recall80	t
Gradient Boosting	0.7	0.571	0.500	0.667	0.810	0.700	0.6	
LogReg-ElasticNet	0.8	0.667	0.667	0.667	0.810	0.589	0.6	
LogReg-L2	0.8	0.667	0.667	0.667	0.810	0.589	0.6	
GP Classifier	0.7	0.400	0.500	0.333	0.810	0.589	0.6	
PLS-DA	0.7	0.400	0.500	0.333	0.810	0.589	0.6	
Random Forest	0.7	0.400	0.500	0.333	0.762	0.533	0.6	

In [10]:
```python
fig, ax = plt.subplots(figsize=(8, 6))
for name, proba in flop_probas.items():
    p, r, _ = precision_recall_curve(y_test, proba)
    ap = average_precision_score(y_test, proba)
    ax.plot(r, p, marker='.', alpha=0.6, label=f'{name} (PR-AUC={ap:.2f})')
ax.axhline(y_test.mean(), color='gray', linestyle='--', alpha=0.4,
           label=f'No-skill ({y_test.mean():.2f})')
ax.set_xlabel('Recall')
ax.set_ylabel('Precision')
ax.set_title('Flop Avoidance (bottom 30%) — Precision-Recall curves')
ax.legend(fontsize=8)
ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



Simulated to real transfer per classification framing

In [11]:
```
X_syn = X_all[synth_mask].reset_index(drop=True)
X_real_test = X_all[~synth_mask].reset_index(drop=True)
y_syn_hit = is_hit[synth_mask].reset_index(drop=True)
y_real_hit = is_hit[~synth_mask].reset_index(drop=True)
y_syn_flop = is_flop[synth_mask].reset_index(drop=True)
y_real_flop = is_flop[~synth_mask].reset_index(drop=True)

print(f'Synthetic train: {len(X_syn)} (hit fraction {y_syn_hit.mean():.0%},
      f'flop fraction {y_syn_flop.mean():.0%})')
print(f'Real test:       {len(X_real_test)} (hit fraction {y_real_hit.mean()
      f'flop fraction {y_real_flop.mean():.0%})')

transfer_results = {}
for name, factory in CLASSIFIERS.items():
    transfer_results[name] = {}
    for task_name, y_tr, y_te in [
        ('hit',  y_syn_hit,  y_real_hit),
        ('flop', y_syn_flop, y_real_flop),
    ]:
        pipe = factory()
        metrics, _ = evaluate_binary(pipe, X_syn, y_tr, X_real_test, y_te)
        transfer_results[name][task_name] = metrics
```

```
print('\nSynth→real transfer (test n=10):')
print(f'\n  {"Model":<20s}  {"Hit AUC":>10s}  {"Hit P@R80":>10s}  '
      f'{"Flop AUC":>10s}  {"Flop P@R80":>10s}')
for name, tr in transfer_results.items():
    h = tr['hit']; f = tr['flop']
    p_h_80 = '   nan' if np.isnan(h["precision_at_recall80"]) else f'{h["pre
    p_f_80 = '   nan' if np.isnan(f["precision_at_recall80"]) else f'{f["pre
    print(f'  {name:<20s}  {h["roc_auc"]:>10.2f}  {p_h_80}  '
          f'{f["roc_auc"]:>10.2f}  {p_f_80}')
```

```
Synthetic train: 40 (hit fraction 18%, flop fraction 30%)
Real test:       10 (hit fraction 30%, flop fraction 30%)

Synth→real transfer (test n=10):

  Model                    Hit AUC   Hit P@R80    Flop AUC  Flop P@R80
  LogReg-ElasticNet           0.67        0.38        0.71        0.50
  LogReg-L2                   0.76        0.50        0.67        0.50
  Random Forest               0.81        0.60        0.71        0.50
  Gradient Boosting           1.00        1.00        0.57        0.50
  PLS-DA                      0.62        0.43        0.67        0.50
  GP Classifier               0.76        0.50        0.76        0.60
```

In [13]:

```
out = {
    'task': 'business_classification_hit_flop',
    'thresholds': {'hit_quantile': 0.80, 'flop_quantile': 0.30,
                   'hit_threshold': float(hit_threshold),
                   'flop_threshold': float(flop_threshold)},
    'class_distributions': {
        'is_hit':  {str(k): int(v) for k, v in is_hit.value_counts().items()
        'is_flop': {str(k): int(v) for k, v in is_flop.value_counts().items(
        'tier':    tier.value_counts().to_dict(),
    },
    'feature_count': len(feature_cols),
    'hit_detection':   hit_results,
    'flop_avoidance':  flop_results,
    'synth_to_real_transfer': transfer_results,
}
out_path = RUN_DIR / 'results_business_classification.json'
with open(out_path, 'w') as f: json.dump(out, f, indent=2, default=str)
print(f'Saved → {out_path}')
```

```
Saved → /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie_
success_v6/colab_runs_v3/results_business_classification.json
```

12

NOTEBOOK

Feature-group ablation study

12_ablation_study.pdf

Feature Group Contributions test

In [1]:
```
from google.colab import drive
drive.mount('/content/drive', force_remount=False)
from pathlib import Path
PROJECT = Path('/content/drive/MyDrive/MSC THESIS')
DATA_DIR = PROJECT / 'ml_dataset' / 'data' / 'model_ready' / 'movie_success_
RUN_DIR = DATA_DIR / 'colab_runs_v3'
RUN_DIR.mkdir(parents=True, exist_ok=True)
print('Run dir:', RUN_DIR)
```

```
Mounted at /content/drive
Run dir: /content/drive/MyDrive/MSC THESIS/ml_dataset/data/model_ready/movie
_success_v6/colab_runs_v3
```

In [2]:
```
import warnings; warnings.filterwarnings('ignore')
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold, cross_val_predict, GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_scor
from sklearn.ensemble import RandomForestRegressor
from sklearn.cross_decomposition import PLSRegression
from scipy.stats import spearmanr, pearsonr

pd.set_option('display.max_columns', 80)
plt.rcParams['figure.dpi'] = 100
np.random.seed(42)
print('Imports OK')
```

```
Imports OK
```

2 · Load and merge data

In [3]:
```
real_mov  = pd.read_csv(DATA_DIR / 'movie_features_v6.csv')
syn_mov   = pd.read_csv(DATA_DIR / 'movie_features_v6_synthetic.csv')
real_meta = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6.csv')
syn_meta  = pd.read_csv(DATA_DIR / 'scene_movie_metadata_v6_synthetic.csv')

LEAKAGE = ['budget_usd', 'revenue_usd', 'roi_percent', 'success_class',
           'imdb_rating', 'wom_multiplier', 'wom_multiplier_log']
real_mov = real_mov.drop(columns=LEAKAGE, errors='ignore')
syn_mov  = syn_mov.drop(columns=LEAKAGE, errors='ignore')

META_KEEP = ['movie_id', 'targeted_emotion', 'clip_duration_s',
             'cut_count', 'brightness', 'motion_intensity', 'audio_loudness'
             'silence_ratio', 'music_presence', 'dialogue_density',
```

```
            'face_screen_time_ratio', 'lead_screen_time_ratio',
            'release_year', 'genre_primary', 'genre_secondary',
            'country_of_origin', 'budget_categorical',
            'imdb_rating', 'wom_multiplier', 'wom_multiplier_log']
real_meta_sub = real_meta[[c for c in META_KEEP if c in real_meta.columns]]
syn_meta_sub  = syn_meta[[c for c in META_KEEP if c in syn_meta.columns]]
real_mov = real_mov.merge(real_meta_sub, on='movie_id', how='left'); real_mo
syn_mov  = syn_mov.merge(syn_meta_sub, on='movie_id', how='left');  syn_mov[
df_all = pd.concat([real_mov, syn_mov], ignore_index=True)

# Encode metadata
DROP = {'movie_id','condition','n_participants',
        'imdb_rating','wom_multiplier','wom_multiplier_log','is_synthetic'}
df_feat = df_all.copy()
ORD = {'low':1,'moderate':2,'high':3}
SCENE_ORDINAL = ['cut_count','brightness','motion_intensity','audio_loudness
                 'silence_ratio','music_presence','dialogue_density',
                 'face_screen_time_ratio','lead_screen_time_ratio']
for c in SCENE_ORDINAL + ['budget_categorical']:
    if c in df_feat.columns: df_feat[c] = df_feat[c].map(ORD)
OH = [c for c in ['targeted_emotion','genre_primary','genre_secondary','cour
      if c in df_feat.columns]
df_feat = pd.get_dummies(df_feat, columns=OH, prefix_sep='_', dummy_na=False

ALL_FEATS = [c for c in df_feat.columns if c not in DROP]
X_all = df_feat[ALL_FEATS].apply(pd.to_numeric, errors='coerce')
X_all = X_all.drop(columns=X_all.columns[X_all.isna().all()].tolist())
X_all = X_all.drop(columns=X_all.columns[X_all.std()==0].tolist())
y_imdb = df_feat['imdb_rating'].astype(float)
y_wom  = df_feat['wom_multiplier_log'].astype(float)
print(f'Combined: {len(df_all)} movies × {X_all.shape[1]} features')
```

```
Combined: 50 movies × 360 features
```

Define feature groups

In [4]:

```
# Categorise every feature column
def classify(c):
    # Self-report
    if c.startswith('sr_'): return 'self_report'
    # Physiological (note: has_* are also physiology-related quality flags)
    if c.startswith('of_') or c.startswith('q_') or c.startswith('emp_') \
        or c.startswith('eeg_') or c.startswith('sw_') or c.startswith('has_
        return 'physiology'
    # Scene properties (ordinal-encoded)
    if c in SCENE_ORDINAL:
        return 'scene'
    # Movie metadata: ordinal budget, year, duration
    if c in ('budget_categorical', 'release_year', 'clip_duration_s'):
        return 'metadata'
    # One-hot encoded metadata columns
    if any(c.startswith(p + '_') for p in
           ['targeted_emotion', 'genre_primary', 'genre_secondary', 'country
        return 'metadata'
```

```
    return 'other'

groups = {}
for c in X_all.columns:
    g = classify(c)
    groups.setdefault(g, []).append(c)

print('Feature group sizes:')
for g, cols in sorted(groups.items()):
    print(f'  {g:14s}  {len(cols):4d} features')
print(f'  {"TOTAL":14s}  {sum(len(v) for v in groups.values()):4d}')
```

```
Feature group sizes:
  metadata          31 features
  other             20 features
  physiology       258 features
  scene              9 features
  self_report       42 features
  TOTAL            360
```

In [5]:
```
CONFIGS = {
    '1. Physiology only':
        groups.get('physiology', []),
    '2. + Scene':
        groups.get('physiology', []) + groups.get('scene', []),
    '3. + Movie metadata':
        groups.get('physiology', []) + groups.get('scene', []) +
        groups.get('metadata', []),
    '4. + Self-report (current)':
        groups.get('physiology', []) + groups.get('scene', []) +
        groups.get('metadata', []) + groups.get('self_report', []),
}

print('Ablation configurations:')
for name, cols in CONFIGS.items():
    print(f'  {name:32s}  {len(cols):4d} features')
```

```
Ablation configurations:
  1. Physiology only                 258 features
  2. + Scene                         267 features
  3. + Movie metadata                298 features
  4. + Self-report (current)         340 features
```

4 configs × 2 models × 2 targets = 16 cells

In [6]:
```
def evaluate(X, y, model_factory, name=''):
    """5-fold CV with hyperparameter tuning. Returns metrics dict."""
    kf = KFold(n_splits=5, shuffle=True, random_state=42)
    pipe = model_factory()
    y_pred = cross_val_predict(pipe, X, y, cv=kf, n_jobs=-1)
    return {
        'r2':       r2_score(y, y_pred),
        'mae':      mean_absolute_error(y, y_pred),
        'rmse':     np.sqrt(mean_squared_error(y, y_pred)),
        'spearman': spearmanr(y, y_pred).correlation,
```

```
        'pearson':  pearsonr(y, y_pred)[0],
        'n_features': X.shape[1],
    }

def make_rf():
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('m',   RandomForestRegressor(n_estimators=500, max_depth=7,
                                      max_features=0.3, min_samples_leaf=1,
                                      random_state=42, n_jobs=-1)),
    ])

def make_pls(n_components=2):
    return Pipeline([
        ('imp', SimpleImputer(strategy='median')),
        ('sc',  StandardScaler()),
        ('m',   PLSRegression(n_components=n_components, scale=False)),
    ])

# Run with both default-tuned models on each config
results = {}
for cfg_name, cols in CONFIGS.items():
    if not cols:
        print(f'  Skipping {cfg_name} (0 features)')
        continue
    X_cfg = X_all[cols]
    results[cfg_name] = {}
    for tgt_name, y in [('imdb', y_imdb), ('wom', y_wom)]:
        for mdl_name, factory in [('RF', make_rf), ('PLS', make_pls)]:
            m = evaluate(X_cfg, y, factory, f'{cfg_name} / {tgt_name} / {mdl
            results[cfg_name][f'{mdl_name}_{tgt_name}'] = m
            print(f'  {cfg_name:32s}  {mdl_name}/{tgt_name}: '
                  f'R²={m["r2"]:>+.3f}  ρ={m["spearman"]:>+.3f}  MAE={m["mae
                  f'(p={m["n_features"]})')
```

```
  1. Physiology only               RF/imdb: R²=+0.479  ρ=+0.706  MAE=0.324
(p=258)
  1. Physiology only               PLS/imdb: R²=+0.485  ρ=+0.703  MAE=0.316
(p=258)
  1. Physiology only               RF/wom: R²=+0.181  ρ=+0.401  MAE=0.398
(p=258)
  1. Physiology only               PLS/wom: R²=+0.002  ρ=+0.287  MAE=0.439
(p=258)
  2. + Scene                       RF/imdb: R²=+0.486  ρ=+0.699  MAE=0.326
(p=267)
  2. + Scene                       PLS/imdb: R²=+0.479  ρ=+0.697  MAE=0.324
(p=267)
  2. + Scene                       RF/wom: R²=+0.188  ρ=+0.418  MAE=0.397
(p=267)
  2. + Scene                       PLS/wom: R²=−0.013  ρ=+0.285  MAE=0.442
(p=267)
  3. + Movie metadata              RF/imdb: R²=+0.475  ρ=+0.685  MAE=0.327
(p=298)
  3. + Movie metadata              PLS/imdb: R²=+0.544  ρ=+0.739  MAE=0.296
(p=298)
  3. + Movie metadata              RF/wom: R²=+0.211  ρ=+0.441  MAE=0.392
(p=298)
  3. + Movie metadata              PLS/wom: R²=+0.022  ρ=+0.296  MAE=0.428
(p=298)
  4. + Self-report (current)       RF/imdb: R²=+0.543  ρ=+0.677  MAE=0.304
(p=340)
  4. + Self-report (current)       PLS/imdb: R²=+0.463  ρ=+0.668  MAE=0.311
(p=340)
  4. + Self-report (current)       RF/wom: R²=+0.201  ρ=+0.424  MAE=0.387
(p=340)
  4. + Self-report (current)       PLS/wom: R²=−0.016  ρ=+0.271  MAE=0.429
(p=340)
```

Tune PLS n_components for each config

In [ ]:

```python
print('Tuning PLS n_components per configuration:')
for cfg_name, cols in CONFIGS.items():
    if not cols: continue
    X_cfg = X_all[cols]
    pls_grid = {'m__n_components': list(range(1, min(11, X_cfg.shape[1])))}
    for tgt_name, y in [('imdb', y_imdb), ('wom', y_wom)]:
        kf = KFold(n_splits=5, shuffle=True, random_state=42)
        search = GridSearchCV(make_pls(), pls_grid, cv=kf,
                              scoring='neg_mean_absolute_error', n_jobs=-1)
        search.fit(X_cfg, y)
        best_n = search.best_params_['m__n_components']
        # Re-evaluate with the best n_components via cross_val_predict
        pipe = make_pls(n_components=best_n)
        y_pred = cross_val_predict(pipe, X_cfg, y, cv=kf, n_jobs=-1)
        m = {
            'r2':       r2_score(y, y_pred),
            'mae':      mean_absolute_error(y, y_pred),
            'rmse':     np.sqrt(mean_squared_error(y, y_pred)),
            'spearman': spearmanr(y, y_pred).correlation,
```

```
            'pearson':  pearsonr(y, y_pred)[0],
            'n_features': X_cfg.shape[1],
            'best_n_components': best_n,
        }
        results[cfg_name][f'PLS_{tgt_name}_tuned'] = m
        print(f'  {cfg_name:32s}  PLS-tuned/{tgt_name}: '
              f'R²={m["r2"]:>+.3f}  ρ={m["spearman"]:>+.3f}  '
              f'(n_components={best_n})')
```

```
Tuning PLS n_components per configuration:
  1. Physiology only                PLS-tuned/imdb: R²=+0.485  ρ=+0.703  (n_
components=2)
  1. Physiology only                PLS-tuned/wom: R²=+0.184  ρ=+0.400  (n_c
omponents=1)
  2. + Scene                        PLS-tuned/imdb: R²=+0.479  ρ=+0.697  (n_
components=2)
  2. + Scene                        PLS-tuned/wom: R²=+0.182  ρ=+0.397  (n_c
omponents=1)
  3. + Movie metadata               PLS-tuned/imdb: R²=+0.544  ρ=+0.739  (n_
components=2)
  3. + Movie metadata               PLS-tuned/wom: R²=+0.098  ρ=+0.390  (n_c
omponents=10)
  4. + Self-report (current)        PLS-tuned/imdb: R²=+0.463  ρ=+0.668  (n_
components=2)
  4. + Self-report (current)        PLS-tuned/wom: R²=+0.189  ρ=+0.379  (n_c
omponents=1)
```

Comparison tables

In [8]:

```
# Table 1: R² by config × model (IMDb)
table_r2_imdb = pd.DataFrame({
    cfg: {
        'RF':        results[cfg]['RF_imdb']['r2'],
        'PLS (n=2)': results[cfg]['PLS_imdb']['r2'],
        'PLS tuned': results[cfg]['PLS_imdb_tuned']['r2'],
    } for cfg in results
}).T
print('IMDb prediction — R² by configuration × model:')
display(table_r2_imdb.round(3))

# Table 2: R² by config × model (WOM)
table_r2_wom = pd.DataFrame({
    cfg: {
        'RF':        results[cfg]['RF_wom']['r2'],
        'PLS (n=2)': results[cfg]['PLS_wom']['r2'],
        'PLS tuned': results[cfg]['PLS_wom_tuned']['r2'],
    } for cfg in results
}).T
print('\nWOM prediction — R² by configuration × model:')
display(table_r2_wom.round(3))

# Table 3: Spearman by config × model
table_spearman_imdb = pd.DataFrame({
    cfg: {
```

```
        'RF':         results[cfg]['RF_imdb']['spearman'],
        'PLS (n=2)': results[cfg]['PLS_imdb']['spearman'],
        'PLS tuned': results[cfg]['PLS_imdb_tuned']['spearman'],
    } for cfg in results
}).T
print('\nIMDb prediction — Spearman ρ by configuration × model:')
display(table_spearman_imdb.round(3))
```

IMDb prediction — R² by configuration × model:

	RF	PLS (n=2)	PLS tuned
1. Physiology only	0.479	0.485	0.485
2. + Scene	0.486	0.479	0.479
3. + Movie metadata	0.475	0.544	0.544
4. + Self-report (current)	0.543	0.463	0.463

WOM prediction — R² by configuration × model:

	RF	PLS (n=2)	PLS tuned
1. Physiology only	0.181	0.002	0.184
2. + Scene	0.188	-0.013	0.182
3. + Movie metadata	0.211	0.022	0.098
4. + Self-report (current)	0.201	-0.016	0.189

IMDb prediction — Spearman ρ by configuration × model:

	RF	PLS (n=2)	PLS tuned
1. Physiology only	0.706	0.703	0.703
2. + Scene	0.699	0.697	0.697
3. + Movie metadata	0.685	0.739	0.739
4. + Self-report (current)	0.677	0.668	0.668

Marginal contribution per feature group

In [9]:
```
# How much does each group add on top of the previous?
print('Marginal contribution to R² (using best PLS):\n')
print(f'{"Step":40s}  {"IMDb R²":>10s}  {"+Δ":>8s}  {"WOM R²":>10s}  {"+Δ":>
print('—' * 80)

prev_imdb = 0.0
prev_wom = 0.0
for cfg in results:
    r2_i = results[cfg]['PLS_imdb_tuned']['r2']
    r2_w = results[cfg]['PLS_wom_tuned']['r2']
    delta_i = r2_i - prev_imdb
```

```
    delta_w = r2_w - prev_wom
    print(f'{cfg:40s}  {r2_i:>+10.3f}  {delta_i:>+8.3f}  {r2_w:>+10.3f}  {de
    prev_imdb = r2_i
    prev_wom = r2_w

# Best-case (RF) for comparison
print('\nSame analysis with Random Forest:\n')
print(f'{"Step":40s}  {"IMDb R²":>10s}  {"+Δ":>8s}  {"WOM R²":>10s}  {"+Δ":>
print('-' * 80)
prev_imdb = 0.0; prev_wom = 0.0
for cfg in results:
    r2_i = results[cfg]['RF_imdb']['r2']
    r2_w = results[cfg]['RF_wom']['r2']
    delta_i = r2_i - prev_imdb
    delta_w = r2_w - prev_wom
    print(f'{cfg:40s}  {r2_i:>+10.3f}  {delta_i:>+8.3f}  {r2_w:>+10.3f}  {de
    prev_imdb = r2_i
    prev_wom = r2_w
```

```
Marginal contribution to R² (using best PLS):

Step                                       IMDb R²        +Δ      WOM R²
+Δ
--------------------------------------------------------------------------
----
1. Physiology only                          +0.485    +0.485      +0.184
+0.184
2. + Scene                                  +0.479    -0.006      +0.182
-0.001
3. + Movie metadata                         +0.544    +0.065      +0.098
-0.084
4. + Self-report (current)                  +0.463    -0.081      +0.189
+0.090

Same analysis with Random Forest:

Step                                       IMDb R²        +Δ      WOM R²
+Δ
--------------------------------------------------------------------------
----
1. Physiology only                          +0.479    +0.479      +0.181
+0.181
2. + Scene                                  +0.486    +0.007      +0.188
+0.007
3. + Movie metadata                         +0.475    -0.011      +0.211
+0.023
4. + Self-report (current)                  +0.543    +0.068      +0.201
-0.010
```

Visuals

In [10]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Bar chart of R² across configs
```

```python
configs_short = list(results.keys())
labels = [c.replace('. ', '.\n') for c in configs_short]

# IMDb R²
rf_imdb_r2  = [results[c]['RF_imdb']['r2'] for c in configs_short]
pls_imdb_r2 = [results[c]['PLS_imdb_tuned']['r2'] for c in configs_short]

x = np.arange(len(configs_short))
w = 0.35
axes[0].bar(x - w/2, rf_imdb_r2, w, label='Random Forest', color='#2ca02c')
axes[0].bar(x + w/2, pls_imdb_r2, w, label='PLS (tuned)', color='#1f77b4')
axes[0].axhline(0, color='black', linewidth=0.5)
axes[0].set_xticks(x)
axes[0].set_xticklabels(labels, fontsize=8)
axes[0].set_ylabel('R² (5-fold CV)')
axes[0].set_title('IMDb prediction — feature group ablation')
axes[0].legend()
axes[0].grid(alpha=0.3)

# WOM R²
rf_wom_r2  = [results[c]['RF_wom']['r2'] for c in configs_short]
pls_wom_r2 = [results[c]['PLS_wom_tuned']['r2'] for c in configs_short]
axes[1].bar(x - w/2, rf_wom_r2, w, label='Random Forest', color='#2ca02c')
axes[1].bar(x + w/2, pls_wom_r2, w, label='PLS (tuned)', color='#1f77b4')
axes[1].axhline(0, color='black', linewidth=0.5)
axes[1].set_xticks(x)
axes[1].set_xticklabels(labels, fontsize=8)
axes[1].set_ylabel('R² (5-fold CV)')
axes[1].set_title('WOM-log prediction — feature group ablation')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout(); plt.show()
```